LLM Inference Illustrated

Ted Kyi

Draft dated 2026-04-06

Table of contents

Preface v

Conventions vii

1 Introduction 1
- 1.1 Overview . . . 1

2 The Decoder-Only Transformer 5
- 2.1 From Encoder-Decoder to Decoder-Only . . . 5
- 2.2 Forward Pass Data Flow . . . 11
- 2.3 The decoder block . . . 16
- 2.4 Self-Attention Mechanics . . . 17
- 2.5 The KV Cache . . . 21
- 2.6 The Feed-Forward Network . . . 25
- 2.7 Further Reading . . . 29

3 Measuring LLM Inference 31
- 3.1 The Inference Pipeline . . . 31
- 3.2 Performance Metrics . . . 35
- 3.3 Workload Patterns . . . 41
- 3.4 The Hardware Model and Bottleneck Framework . . . 44
- 3.5 Putting Numbers on a Baseline . . . 49
- 3.6 Map of the optimizations . . . 55
- 3.7 Further Reading . . . 56

4 Model Design Choices 58
- 4.1 Model Quantization . . . 58
- 4.2 Knowledge Distillation . . . 62
- 4.3 Pruning . . . 63
- 4.4 Efficient Attention Architectures . . . 64
- 4.5 Tokenization and Vocabulary Effects . . . 69
- 4.6 Summary . . . 69
- 4.7 Further Reading . . . 70

5 Scheduling Bottlenecks 71
- 5.1 Batching Strategies . . . 71
- 5.2 Prefill Strategies . . . 76
- 5.3 Ragged batching . . . 80
- 5.4 Request Scheduling and Prioritization . . . 82

5.5 Further Reading . . . 85

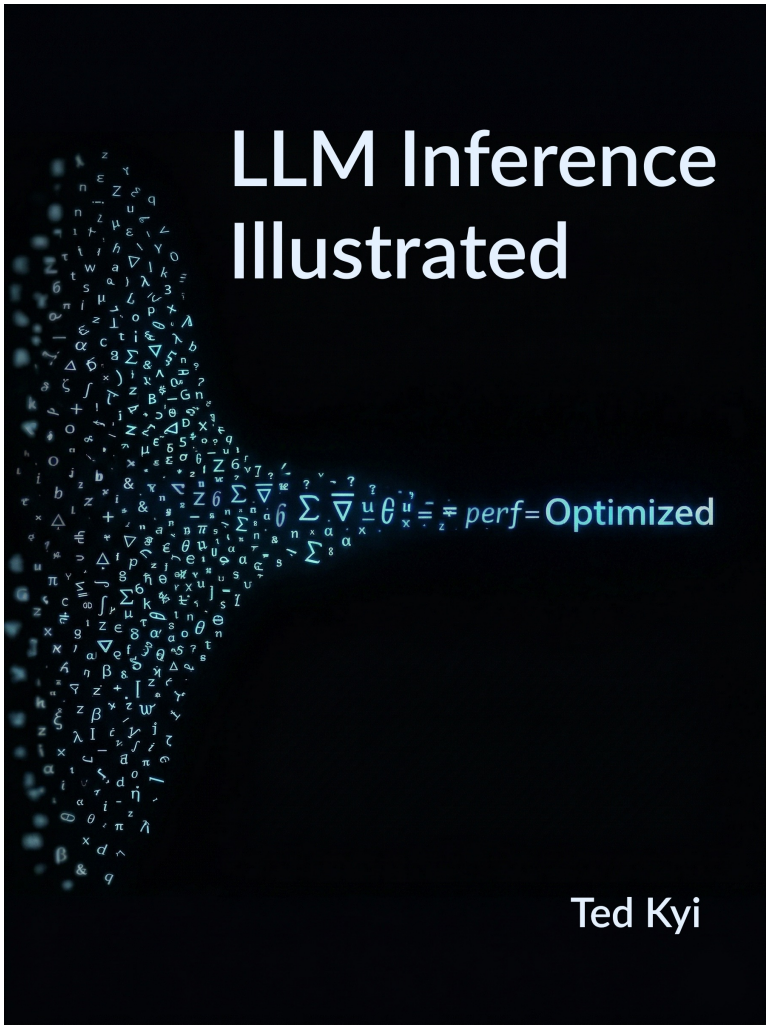
6 Request Compute, Memory, and Latency Bottlenecks 86
6.1 Memory-Aware Attention Kernels . . . 86
6.2 Kernel Fusion and Compute Graph Optimization . . . 90
6.3 KV Cache Engineering . . . 91
6.4 Prompt and Prefix Caching . . . 95
6.5 Speculative Decoding and Multi-Token Prediction . . . 96
6.6 Parallel Decoding . . . 101
6.7 Further Reading . . . 103

7 Scaling Across Hardware 104
7.1 Interconnects and Multi-GPU Topology . . . 104
7.2 Parallelism Overview and Data Parallelism . . . 106
7.3 Tensor Parallelism (TP) . . . 107
7.4 Pipeline Parallelism (PP) . . . 109
7.5 Expert Parallelism (EP) . . . 111
7.6 Context / Sequence Parallelism (CP/SP) . . . 113
7.7 Communication Cost Analysis . . . 115
7.8 Multi-Node Inference . . . 116
7.9 Further Reading . . . 117

8 Production LLM Serving Systems 119
8.1 Serving Frameworks . . . 119
8.2 Multi-LoRA Serving . . . 121
8.3 Scheduling and Orchestration . . . 122
8.4 Memory Management and Preemption . . . 123
8.5 Monitoring and Benchmarking . . . 124
8.6 Production Failure Modes . . . 125
8.7 Further Reading . . . 127

References 128

Cover



LLM Inference Illustrated

Copyright © 2026 Ted Kyi

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0).

You are free to share and adapt this material for non-commercial purposes, provided you give appropriate credit and distribute your contributions under the same license.

Typeset with Quarto.

Preface

A short book on autoregressive LLM inference and the techniques that make it fast, efficient, and scalable.

Who this book is for

This book is an intermediate guide for ML engineers (and people with similar background) who are already familiar with neural networks, the transformer architecture, self-attention, and autoregressive decoder-only LLMs. This book evolved from research into efficient ways to do LLM inference for special cases such as RAG, Writing in the Margins, and beam search. After spending time learning about attention kernels, prefix caching, and systems such as vLLM and SGLang, I decided to share the knowledge I had gathered in one convenient place.

This book contains many detailed diagrams and is best viewed on a high resolution screen.

What this book covers

This book discusses how LLM inference works, then reviews the techniques for optimizing and scaling LLM inference. This book should not be the first resource for learning about the transformer, nor will it teach how to code LLMs. It is also not a comprehensive ML Ops or system implementation book. I hope you find it useful for providing a good intuition and the necessary details about concepts like chunked prefill, ragged batching, tensor parallelism, and speculative decoding.

How this book was written

This book started as a detailed list of topics, techniques, and research papers. The idea to use data flow diagrams and the design of most diagrams was entirely my own.

Full disclosure: I used LLMs to suggest ways to organize the material and to write initial text based on my notes. I used Claude Code to speed up the implementation of the diagrams and to debug places where the appearance of text, formulas, or figures wasn't correct. If you are not interested in reading work that used AI assistance, that is your prerogative. For what it's worth, AI assistance shortened the process of getting the first release of this book from months to weeks. I probably would not have taken the time to write this book if it had required several months.

Acknowledgements

Thank you to everyone who provided feedback, starting with RC and RS.

I thank you for reporting any errata or making suggestions for how this book could be better. The easiest way is to open a GitHub issue from within the electronic book.

Conventions

Figures throughout this book use a consistent visual language for tensor shapes, dimensions, and components. This page collects those conventions in one place so you can decode any diagram at a glance.

Dimension names and abbreviations

Tensors in figures are labeled with their shapes using the single-letter abbreviations below. The full names and abbreviations match those used in the text.

Table 1: Standard dimension symbols used in figures and text.

Symbol	Full name	Example values (Llama 3 70B)
B	Batch size	1–64
D	Model dimension (d_{model})	8192
d_K	Head dimension (D/H)	128
F	FFN intermediate dimension (d_{ff})	28672 (SwiGLU)
H	Number of attention heads (n_{heads})	64
H_{KV}	Number of KV heads	8 (with GQA)
L	Number of layers	80
S	Sequence length	Prefill: prompt length (e.g., 2048); Decode: 1
V	Vocabulary size	128000

Tensor edge colors

Tensor rectangles use **colored edges** to indicate which dimension runs along that axis. Top/bottom edges represent one dimension; left/right edges represent another. This makes it possible to see at a glance how dimensions flow through a computation.

Table 2: Edge colors encode tensor dimensions.

Edge	Dimension
	Model dimension (**D**)
	Head / Q-K dimension ($\mathbf{d_K}$)
	FFN intermediate dimension (**F**)
	Number of attention heads (**H**)
	Number of KV heads ($\mathbf{H}_{\mathrm{KV}}$)
	Number of layers (**L**)
	Sequence length (**S**)
	Vocabulary (**V**)
	Default / other

Tensor fill colors

The **fill color** of a tensor rectangle indicates its role in the computation.

Table 3: Fill colors encode tensor roles.

Fill	Meaning
	Input / activations (neutral)
	Weight matrices
	Queries (**Q**)
	Keys (**K**)
	Values (**V**)
	Attention scores / weights
	KV cache entries

Component colors

Architectural block diagrams use a separate set of fill colors to distinguish transformer components.

Table 4: Component colors in architectural block diagrams.

Color	Component
	Embedding layers
	Layer normalization / Add & Norm
	Self-attention blocks
	Feed-forward blocks
	Linear projections
	Softmax

Reading a data flow diagram

To illustrate how these conventions combine, here is how a typical tensor rectangle should be read:

- The **top and bottom edge colors** tell you the dimension that runs horizontally, the number of columns.
- The **left and right edge colors** tell you the dimension that runs vertically, the number of rows.
- The **shape label** (e.g., **(S, D)**) confirms the dimensions explicitly.
- The **fill color** helps disambiguate what kind of tensor it is (query, key, weight, etc.).

When two tensors share an edge color on a matching side, it means those dimensions are compatible for the operation that connects them — a matrix multiply, a concatenation, or an elementwise operation. For matrix multiplcation, the number of columns on the lefthand tensor must match the number of rows on the righthand tensor. For elementwise operations, usually all dimensions must match, unless one of the dimensions is 1, and it is broadcast to match the other tensor.

1 Introduction

Before ChatGPT, most of the focus on language models was on how to train better, smarter models. Now that we are in a world where millions of people use **large language models (LLMs)** every day, much recent energy has gone into making serving these models more efficient and less expensive. This book summarizes many of the key advancements with LLM inference. We will attempt to provide clarity about each concept, but this is not an ML Ops book, and we will not drill down into coding each technique. In this chapter, we start by understanding the components of LLM inference. In the remainder of this book, we will look at different bottlenecks and techniques for improving performance.

1.1 Overview

LLM inference is the process of generating a text response. You give the model some input, and it predicts what comes next, one word (technically, one token) at a time. Before we get into the technical details of how inference systems are built and optimized, let's walk through a simple example to build intuition for what's actually happening.

A simple example

Suppose we give an LLM the prompt **"The quick brown fox jumps"** — five input tokens. The model's job is to predict what tokens come next.

The text generation happens in a series of steps. The first thing that happens is the LLM runs a single **forward pass** of the neural network model using all of the input tokens. After processing all of these tokens together, it produces its prediction for the next token, **over**. This initial step is shown in Figure 1.1.

Text generation continues in a series of steps, one token at a time, with the entire process shown in Figure 1.2:

1. In the next step, the predicted token **over** is appended to the end of the input text, and the model is fed **"The quick brown fox jumps over"**. In this step it predicts **the**.
2. Once again, the predicted token, which is now **the**, is appended to the end, the model is fed **"The quick brown fox jumps over the"**, and it predicts **lazy**.
3. Continuing this process, the input **"The quick brown fox jumps over the lazy"** predicts **dog**.

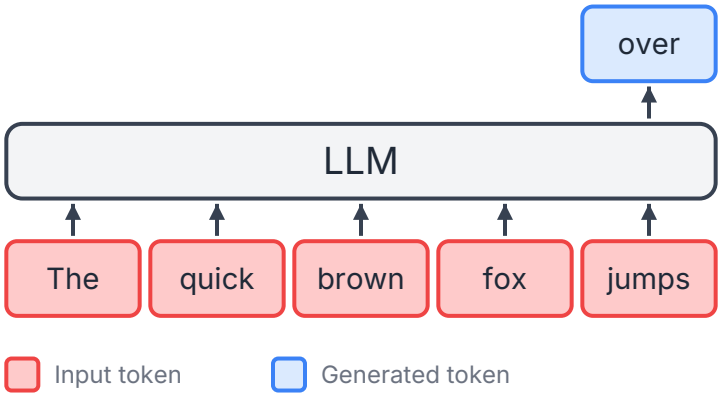


Figure 1.1: First token predicted for the prompt “The quick brown fox jumps”

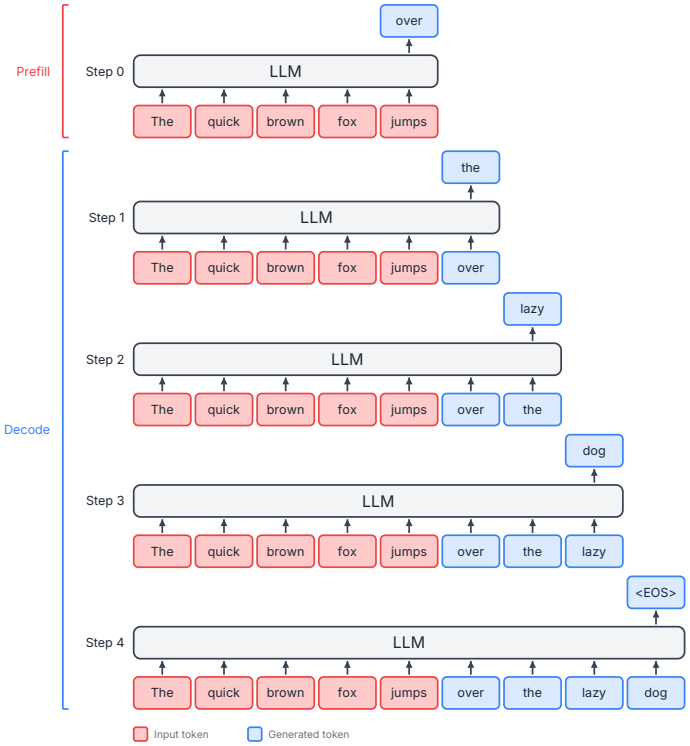


Figure 1.2: An example LLM response for the prompt “The quick brown fox jumps”

4. Then, the input **"The quick brown fox jumps over the lazy dog"** outputs a special token **<EOS>**. The special token tells the LLM to stop the generation, and the completed response is returned to the user.

In our very first step, the LLM is given all of our input tokens. In many cases, this input prompt can be quite long. Because we have so much new data to process, this initial step uses different computing resources than later steps. This first step is called the **prefill**, and it has a high ratio of computation performed to volume of data used.

The remaining steps each add only one new token to the inputs that the LLM has seen before. This has a different computing profile from the prefill, with a much lower ratio of computation performed to volume of data used. This portion is called the **decode** phase, and when a lot of text is generated, it requires many decode steps to complete generation of the response.

This iterative token loop is the core process that LLMs such as ChatGPT and Claude use to respond to requests. In this book, we will dig deeper to understand the internals of this inference process, then explore techniques for improving it.

Why inference optimization matters

That simple loop — run the model, get a token, repeat — sounds straightforward. But the raw cost of running it naively is staggering. A 70-billion-parameter model stores its weights in about 140 GB of GPU memory (at 2 bytes per parameter). Every single decode step has to, at a minimum, read through all of those weights just to produce one token. Without any optimization, a single request to a 70B model might generate only 15–20 tokens per second on a high-end GPU, and frontier models like Gemini, which are believed to be ten times larger, would be below 2 tokens per second. Run that for one user at a time with no batching, and you've got an extremely expensive system that can barely output faster than one person can read, while also leaving most of the GPU's computational capability unused.

The situation gets much better with the right techniques. Modern serving frameworks like vLLM and SGLang apply a combination of the optimizations covered in this book — smarter batching, memory management, kernel optimization, and more — and the results are dramatic. In one benchmark, an unoptimized serving setup on an A100 GPU achieved about 80 tokens per second for a 13B model. The same model served through vLLM achieved roughly 1,900 tokens per second — a 23x improvement (Anyscale 2024a). On newer hardware, optimized frameworks routinely serve models like Llama 3 8B at over 16,000 tokens per second on a single H100 GPU, with per-token latencies in the single-digit milliseconds.

These aren't gains from better hardware. They come from understanding the bottlenecks in the LLM inference pipeline and systematically eliminating them. That's what this book is about.

About this book

This book is written for ML engineers who know the LLM's neural network architecture and want to understand what else is part of serving LLM requests, and in particular how to make that process faster. We assume you're familiar with neural networks, the transformer architecture, and self-attention at a conceptual level.

With that, let's get into the technical details. We'll start by breaking down the decoder-only transformer architecture in further detail and building a precise understanding of the data and operations involved.

2 The Decoder-Only Transformer

This chapter covers the architecture of the basic decoder-only transformer used by most LLMs such as ChatGPT and Llama. We will walk through it in enough detail to be able to trace a token through the full model, seeing the parameter and activation tensor dimensions. If you aren't at all familiar with the transformer, there are many good resources, and a few are listed in the Further Reading (Section 2.7) at the end of the chapter. If you're already comfortable with multi-head self-attention, the KV cache, and gated MLPs, you can skim through this material and move ahead to the next chapter. We recommend that you examine the tip at the beginning of Section 2.2 to understand the conventions used in data flow diagrams before moving ahead to the rest of the book.

We'll use concrete numbers from the Llama family of models in this book. This is not because Llama is special, but because its architecture is representative of the core that modern LLMs share and its parameters are public. There are many newer models with better performance, but Llama has a simpler, more canonical architecture which is a good fit for this introduction. When we say "a 70B model," we mean something with roughly Llama 3 70B's dimensions: 80 layers, a model dimension of 8192, 64 attention heads, and a vocabulary of 128,000 tokens (Grattafiori et al. 2024).

2.1 From Encoder-Decoder to Decoder-Only

The original transformer architecture introduced in "Attention Is All You Need" (Vaswani et al. 2017) has two distinct halves: an **encoder** that processes the input sequence and a **decoder** that generates the output sequence. The encoder builds a rich representation of the input through layers of self-attention. The decoder then attends to that representation through **cross-attention** layers while also attending to its own previously generated tokens through **masked self-attention**.

This encoder-decoder design, shown in Figure 2.1, was built for sequence-to-sequence tasks like machine translation, where you have a clear separation between input (such as a sentence in French) and output (its English translation). The encoder on the left reads the full input and builds a bidirectional representation — each token can attend to every other token, past and future. The decoder on the right then generates the output one token at a time, attending both to the encoder's representation and to its own partial output.

The decoder-only transformer, used by GPT-2 (Radford et al. 2019), LLaMA (Touvron, Lavril, et al. 2023), and virtually all modern large language models in production today, takes this architecture and strips it down. The entire encoder is removed. The cross-attention layers in the middle of the decoder are removed. What's left is a single stack of transformer blocks,

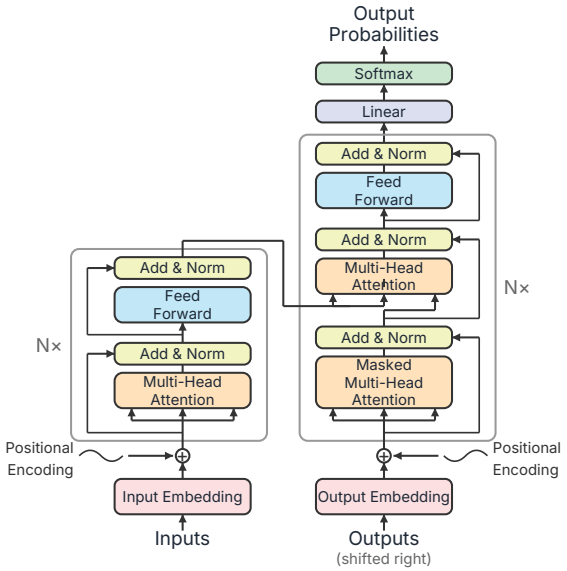


Figure 2.1: The original transformer architecture

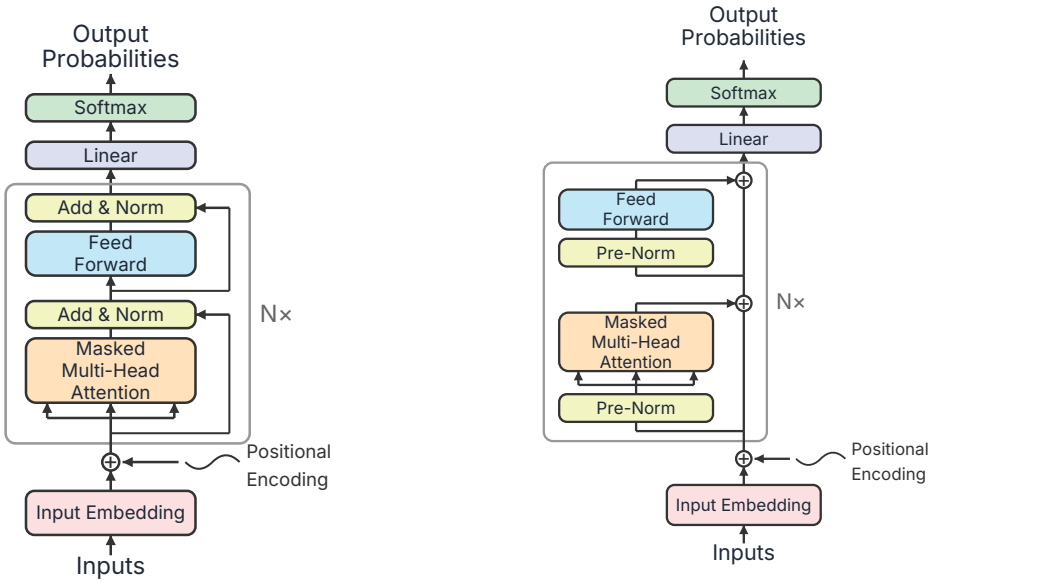
each containing masked self-attention and a **feed-forward network (FFN)**, preceded by a token embedding layer and followed by a linear language model head.

On the left of Figure 2.2 is a trimmed down decoder-only version of the original transformer architecture. Researchers also found that training larger models was more stable when the normalization happened before each attention and FFN block, instead of after. This design is referred to as the **pre-norm** architecture. On the right side of Figure 2.2 is a pre-norm decoder-only model where the attention and FFN layers have been shifted to the side, making the residual connection path, known as the **residual stream**, more obvious. Throughout this book, we will use this straight through arrangement of the pre-norm decoder-only LLM in our examples.

Why did this simpler architecture win? Three reasons stand out. First, **simplicity**: one stack of identical blocks is easier to implement, optimize, and scale than two interleaved stacks. Second, **favorable scaling behavior**: empirically, decoder-only models have shown strong performance scaling with increased parameters and data, as demonstrated by GPT-3, PaLM (Chowdhery et al. 2022), and the Llama series. Third, a **unified pretraining objective**: next-token prediction works on any text, requiring no paired input-output data. You can perform self-supervised training on the entire internet without needing aligned corpora or labeled data. Another key benefit is that the decoder-only model is faster to train than the original encoder-decoder transformer.

Layer repetition and weight sharing

Two aspects of the architecture are worth making very explicit, because they matter for understanding inference costs.



(a) Original decoder-only LLM

(b) Straight through view of pre-norm decoder-only LLM

Figure 2.2: A decoder-only LLM, shown in two variants

First, the "N×" in the architecture diagram means that the same block structure is repeated N times. Note that each layer has the same structure, shape, and size, but each layer has its own independent set of weights. For example, Layer 0's attention weights are completely separate from Layer 1's. A 70B model with 80 layers has 80 distinct sets of attention and feed-forward weights. During inference, the model must read each layer's weights from memory — there is no reuse across layers in the standard LLM.

A diagram making the N copies explicit is shown in Figure 2.3. This is what a decoder-only LLM really looks like, and if you review the PyTorch code for one, you will see this exact structure. The "N×" is shorthand, and it saves space in diagrams, but in the real world, such as on GPU memory, all of those many copies do exist and take up a lot of room. Please also note that each layer must be calculated before the next layer can be calculated. (If you are familiar with earlier work with RNNs and LSTMs, we have eliminated the sequential dependency in the sequence length direction, but we now have a sequential dependency in the layer direction.) We will also mention here that the input to the first layer is a vector of floating point numbers of length d_{model}, which we will abbreviate **(D)**.

The second architectural understanding needed is that the model is applied identically across every token position in the sequence, sharing all weights. When we process a sequence of **S** tokens, the same weight matrices are applied to every token position. Figure 2.4 shows a first intuition of the model being copied repeatedly for each token position. Unlike the layers in the vertical direction that are real, the copies of each block in the same layer at different token positions are conceptual and share weights. For example, the FFN weights in layer 2 for token 0 are identical to the FFN weights in layer 2 for token 1. There is only one FFN for

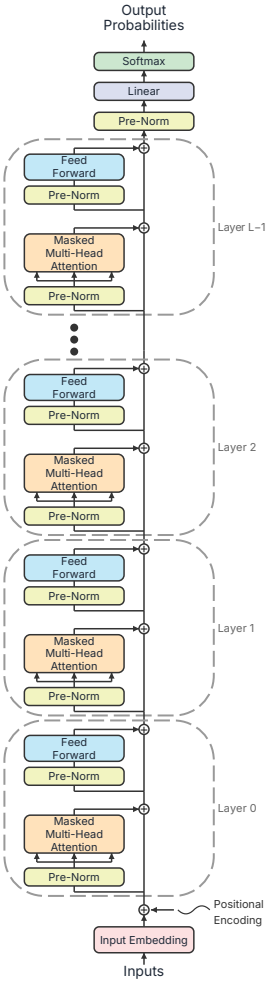


Figure 2.3: Explicit multiple layers

layer 2, and the horizontal copies are just a mental model.

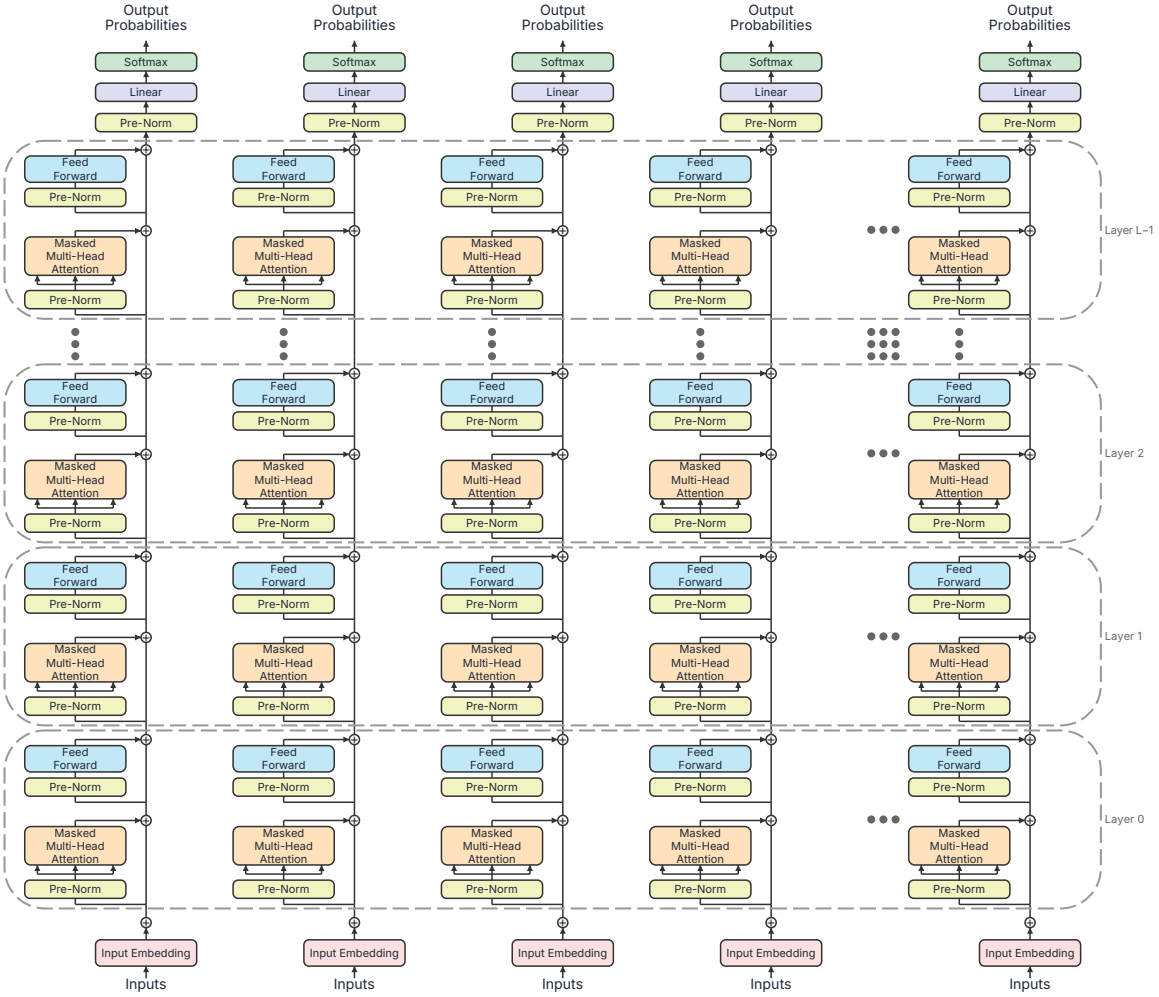


Figure 2.4: Intuition of the LLM model being copied for each token position. Horizontal copies represent the virtual model processing each token.

One aspect of the data flow that is not yet captured in this diagram is that the attention calculation processes information across token positions. All of the other blocks, such as embedding or FFN, process each token position's input in the same way, regardless of what is in other token positions. Masked multi-head attention looks at information from the other token positions, so we add an orange line in Figure 2.5 to explicitly show that data flows from earlier tokens to all later tokens. Wiring up separate vertical stacks would also require connecting attention layers in this way. We are modeling this conceptually at this time, and we will see in Section 2.5 that a fundamental optimization will implement this orange path in a data store called the KV cache.

If we were to make a separate physical copy of the model for each token position, this would give us a correct answer, but it would consume massive amounts of memory for longer sequences. This idea of multiple copies of our model is a good mental model to understand

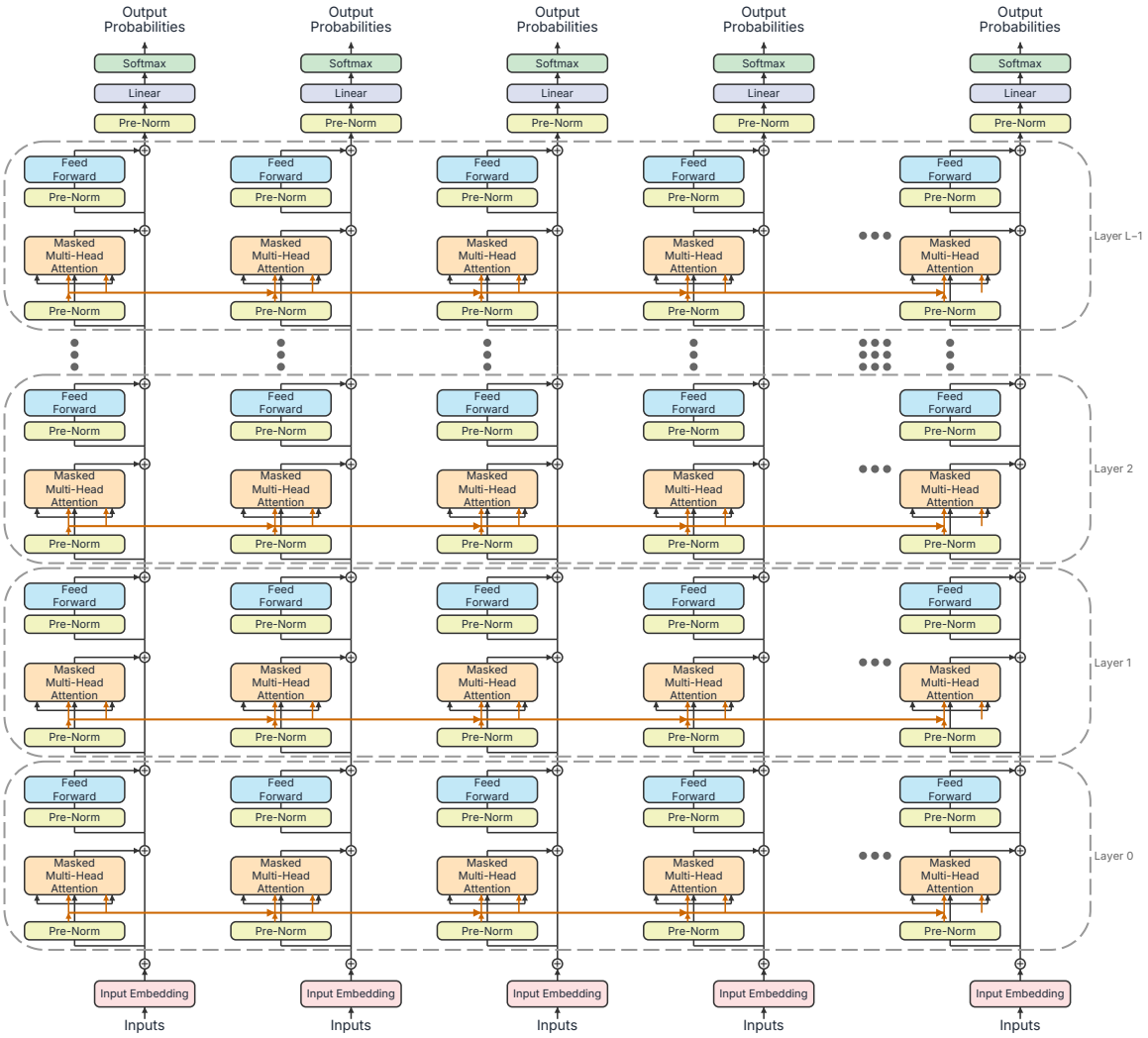


Figure 2.5: Concept of LLM model being copied for each token position, with attention communication added.

data flow across token positions, however this is not how model inference happens. Rather than making copies of the model, we simply take our vertical stack model and change the input from a 1D vector of shape **(D)** to a 2D tensor of shape **(S, D)**. Passing in this 2D tensor into a single copy of the model yields a single tensor with outputs identical to what multiple copies of the model would have produced.

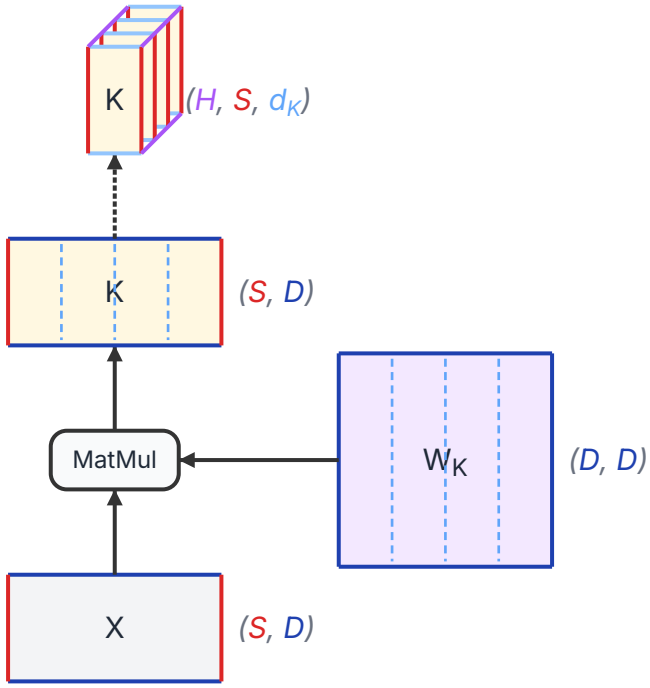
> **Note**
>
> For efficiency reasons, almost all deep learning training and inference happens in batches. This is true as well for LLM inference, and we will discuss batching in Section 5.1. For the sake of brevity, especially in diagrams, we will discuss non-batched data flow and omit the explicit mention of the batch dimension. For LLMs, the full batched shape of the input to the first layer is **(B, S, D)**, which we abbreviate to **(S, D)** for non-batched inputs. Wherever data tensor shapes are mentioned without a leading batch dimension, it is appropriate to think about batched processing adding a single dimension to the beginning of the shape. Weight tensors do not grow when processing data in batches. Rather, they are broadcast in the batch dimension. We hope this convention of using non-batched shapes simplifies diagrams and eases intuition without any loss of generality for the batched use case.

2.2 Forward Pass Data Flow

To gain a deeper understanding of how inference works, we will examine what happens when we send data through a forward pass of the model. We will start at the big picture and work our way down into the individual components. Our diagrams will trace input tensors through all of the operations until we reach the output tensor, annotating every tensor's shape along the way. We call these **tensor data flow diagrams**, and they focus on the shapes of the tensors passed into operations more than the math of the calculation being performed. The focus on tensors is so we can understand the size and shape of the data, which directly affects transfer speeds and computation costs. The purpose of the data flow diagrams is to make properties of the tensors explicit so it is easier to have an intuition about the computations being performed. We still need to be familiar with the operands, especially to know how the inputs can be partitioned, and how much data must be exchanged between partitions.

Tip

Before diving into the data flow, here are a few conventions used in the tensor data flow



diagrams.

In this example data flow diagram, the tensors are represented by rectangles, and the operand a rounded rectangle.

Dimensions and edge color: Each tensor's shape is shown next to it on the right side, unless there isn't room. The first tensor on the bottom has shape (**S**, **D**), where **S** is the number of rows and **D** is the number of columns. Notice that the left and right edges are colored red, and every tensor with a dimension that is length **S** will have that dimension colored red. Similarly, the top and bottom edges are colored blue because the number of columns is **D**, and all other dimensions length **D** will be the same color.

Fill color: The fill color of the bottom tensor is gray. For convenience, tensors holding similar semantic information will have similar fill colors. Weights are fixed model tensors, and they always have the same purple fill color.

3D tensors: The tensor at the top of the diagram is 3D with shape (**H**, **S**, d_K). Because it is 3D, it is shown with a "stacked" rectangle shape. The leading dimension, **H**, is the stacked dimension. The third dimension's purple edge color corresponds to this dimension.

Views: The middle tensor and the top tensor are connected by an arrow with a dotted line. The dotted stroke indicates that there is a change of view, but no actual operation is performed.

The input

The input to our LLM model is a list of integer token IDs. Its shape is **(S)**, where **S** is the sequence length (number of tokens). As a reminder, in this book we are using the simpler notation of non-batched processing. (For batched processing, the input shape would be **(B, S)**.) The value of **S** varies. **S** starts as the length of the prompt, and it increases as generated tokens are appended. It is reasonable that **S** might be less than **D** at the beginning of generation, and then **S** grows to be greater than **D** by the end of generating a long response.

The full model

The complete forward pass stacks **L** decoder layers between the embedding layer and the language model head, as shown in Figure 2.6. This is functionally equivalent to Figure 2.3, except this tensor data flow diagram exposes the shapes of the weight and data tensors.

We see our input with shape **(S)** at the bottom of Figure 2.6. It is converted to a sequence of one-hot vectors, and the input is now shape **(S, V)**. This flows into a matrix multiply with the embedding weights, which are a giant matrix of shape **(V, D)**. We now have a dense input matrix of shape **(S, D)**. This is fed to Layer 0, whose output is the same shape, **(S, D)**, and the layer output is added to the existing values. As the data flows through all **L** layers, the shape never changes from **(S, D)** as the layer outputs continue to get added to the residual stream. After the last layer, the data passes through a final normalization layer, retaining its **(S, D)** shape. Finally, a Linear prediction head does a matrix multiply with a language modeling output weight matrix of shape **(D, V)** and it outputs the probability logits, which are shape **(S, V)**. These can be sharpened by passing through a SoftMax, and a sampling method must be performed to finalize the next token prediction.

The most important observation to take away from Figure 2.6 is that the data along the residual stream is always shape **(S, D)**. When we drill into the decoder layer and its attention and feed forward components, our inputs and outputs will always be **(S, D)**. Most of the compute and memory access cost of a forward pass, especially for models with many layers, is associated with the decoder layers. There isn't much to be gained from modifying how the embedding or linear output projection are performed.

Positional encoding

One detail we've glossed over: how does the model know the order of the input tokens? The self-attention computation is permutation-equivariant, which means that if you shuffle the input tokens, the output gets shuffled the same way (ignoring the causal mask). For this reason, position information must be explicitly injected through a separate pathway.

The technical details and pros and cons of different position embedding techniques are beyond the scope of this section. Some approaches worth mentioning are:

Sinusoidal position embeddings: the position embeddings used in the original "Attention is All You Need" paper. A special embedding table of shape **(S, D)** is added to the token

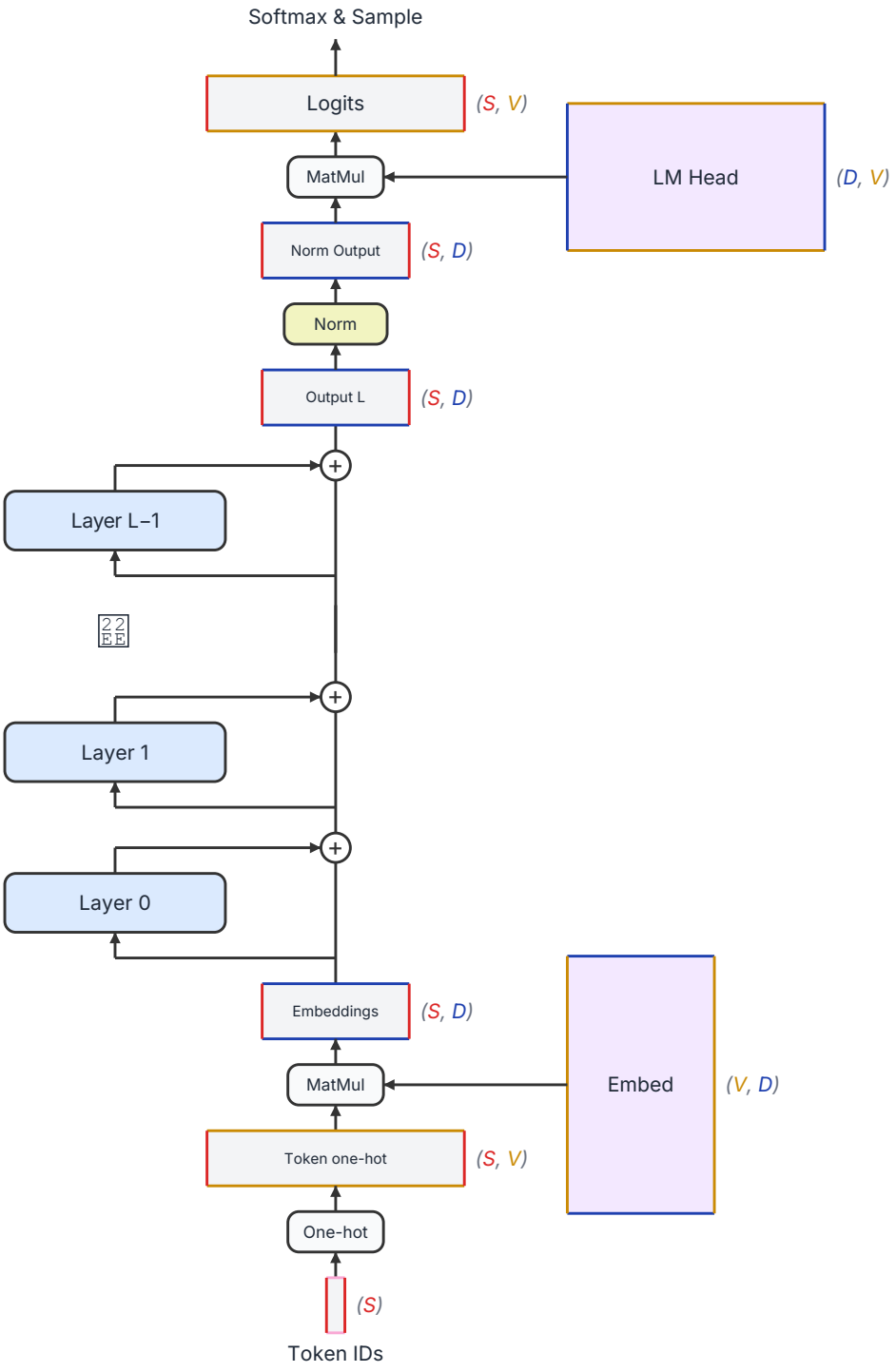


Figure 2.6: Data flow of an entire decoder-only LLM. The details within each decoder layer are hidden for now.

embeddings before they are input to the first decoder layer. This is shown at the bottom of each of Figure 2.1 through Figure 2.3. In the embedding table, position 0 gets one vector, position 1 gets another, and so on. These are called sinusoidal because the values in each length **D** position vector are based on the trigonometric functions sine and cosine.

Learned absolute embeddings (GPT-2 (Radford et al. 2019)): a variant of the embedding table of shape **(S, D)** that is added to the token embeddings, except the values for each embedding vector are learned during training along with all the other model parameters.

Rotary Position Embeddings (RoPE) (Su et al. 2021): instead of adding position information to the input, RoPE applies a position-dependent rotation to the query and key vectors inside the attention computation. The rotation is defined such that the dot product between a query at position i and a key at position j depends only on the relative distance $i - j$. This is what most modern LLMs use, including Llama and its derivatives. RoPE enables better length generalization. Applying RoPE rotations to the query and key vectors is relatively inexpensive, so we omit its details from our diagrams and discussion.

Others: Another notable technique, called **ALiBi** (Press et al. 2021) adds a value dependent on the token position to the keys. This value increases linearly with the token position number. **NoPE** (Kazemnejad et al. 2023) is a newer technique that is gaining traction when mixed with other techniques like RoPE, usually on a layer-by-layer basis. In NoPE, no positional information is added. While it is surprising that having no position information would work, research has shown that NoPE can be highly effective when mixed with other approaches.

Weight tying

Some architectures share weights between the token embedding layer, whose parameter shape is **(V, D)** and the LM head, whose parameter shape is **(D, V)** — which is the transpose of the embedding layer. When embedding tokens, the **(V, D)** weights are used, and when obtaining logits, the transpose of these weights are used. This is called **weight tying**. When the vocabulary is large, these matrices are very large. For Llama 3 70B, where **V** = 128,000 and **D** = 8192, each matrix has just over 1B parameters, so 2B combined. Tying them saves 1B parameters and ensures that the model's input and output representations are in the same space.

Weight tying doesn't affect model speed, but it does reduce the parameter count. The percentage savings is greater for small models, which is why we see weight tying used in some small LLMs. Because of the minimal impact on speed, we will keep things simple in this book and assume the weights are not tied.

2.3 The decoder block

Drilling into each layer in the model, each decoder block in a modern LLM uses the **pre-norm** formulation:

$$\hat{X} = X + \text{SelfAttention}(\text{Norm}(X))$$
$$y = \hat{X} + \text{FFN}(\text{Norm}(\hat{X}))$$

The data flow diagram for this is shown in Figure 2.7.

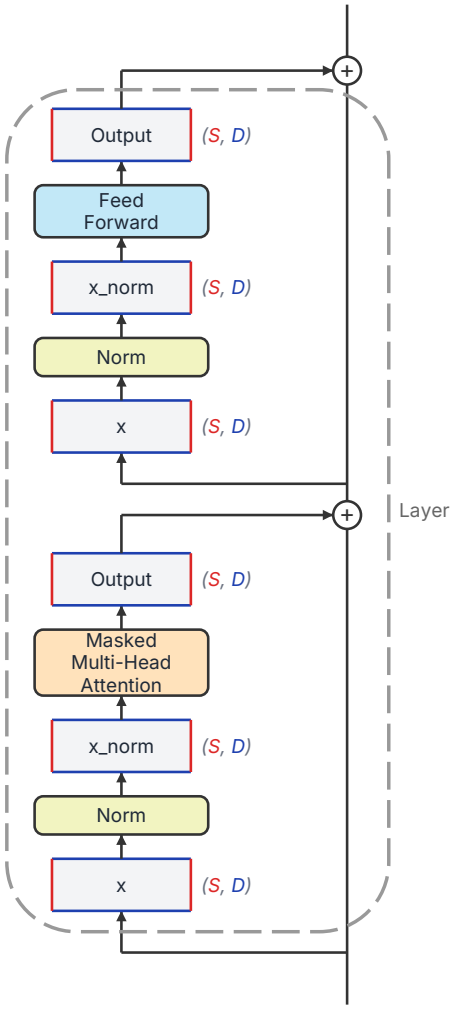


Figure 2.7: Data flow of one decoder layer

The normalization in transformers, which originally was **LayerNorm** (Ba et al. 2016), is applied before each sub-layer (self-attention and FFN), and the sub-layer's output is added to its input via the residual connection. This is the pre-norm formulation, used by GPT-2 (Radford et al. 2019) and almost all modern LLMs. Another normalization layer which has replaced LayerNorm in popularity is **RMSNorm** (Zhang and Sennrich 2019). Both of these operations calculate the magnitude of each input vector across the model dimension **D**, then

scale the input vectors. The need to calculate across **D** will impact tensor parallelism, as we will see in Section 7.3.

The residual connections are critical for training deep networks. They allow gradients to flow through the network's layers without vanishing. For inference, they're simple addition operations that don't add meaningful compute cost.

Our data flow here is all of shape **(S, D)**, and there's not much computational overhead. There isn't much to optimize being shown here. We are now ready to drill into the sub-layers where the majority of the overhead lies. We will start with attention.

2.4 Self-Attention Mechanics

Self-attention is the mechanism that lets each token's representation incorporate information from other tokens in the sequence. This is what makes a transformer more than just a position-independent feature extractor. It lets the model understand that "bank" means something different in "river bank" versus "bank account." We'll walk through the computation step by step, tracking tensor shapes throughout. We will dissect the components and data flow of the attention layer in this section. If the reader is unfamiliar with attention, we recommend some of the resources in the Further Reading (Section 2.7) at the end of this chapter.

The input

The input to a self-attention layer is a tensor of shape **(S, D)**, where **S** is the sequence length (number of tokens), and **D** is the model dimension (the width of each token's representation vector). As a reminder, in this book we are using the simpler notation of non-batched processing. For batched processing, the input shape would be **(B, S, D)**. As a concrete example, in Llama 3 70B, **D** = 8192.

Q, K, V projections

The first step of self-attention is to project the input into three separate representations: **queries** (Q), **keys** (K), and **values** (V). Each projection is a learned linear transformation:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where X has shape **(S, D)** and W_Q, W_K, W_V each have shape **(D, D)**. The outputs Q, K, and V all have shape **(S, D)**, as shown in Figure 2.8. In this data flow diagram, **S** edges are drawn shorter than **D** edges. This will be accurate for Llama 3 70B when **S** $<$ 8192. As the sequence gets longer and we have **S** $>$ 8192, then these dimensions will be longer, despite this static diagram not being able to dynamically show the change.

The intuition for Q, K, and V is roughly this: the query represents "what this token is looking for," the key represents "what this token offers to other tokens," and the value represents "the

information this token will contribute when attended to." These are loose analogies, but they help build intuition for why we need three separate projections rather than just one or two.

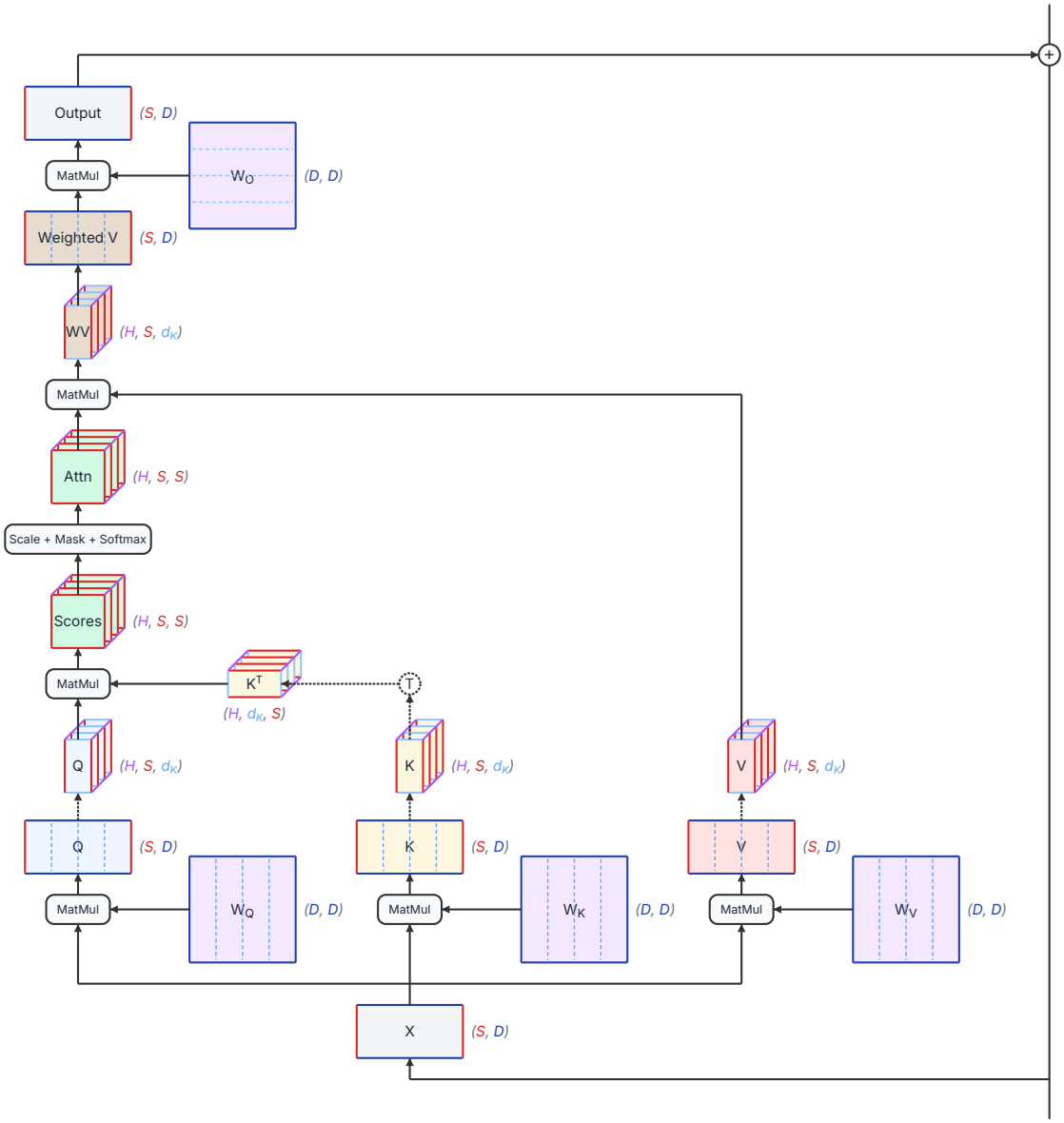


Figure 2.8: Data flow of weights and activations in a multi-head attention layer using a basic, unoptimized implementation

Multi-head attention and the reshape

Rather than computing a single attention function over the full D-dimensional space, we split the computation into **H** parallel heads, each operating on a subspace of dimension $d_K = \mathrm{D}/H$. For Llama 3 70B, **D** = 8192, **H** = 64 heads, and $d_K = 128$. This head dimension, d_K, of 128 is an extremely common choice in LLMs.

After the linear projections, we reshape Q, K, and V from (**S**, **D**) to (**H**, **S**, d_K). Mathematically, this is just reshaping the last dimension — breaking **D** into **H** groups of d_K — and transposing so that the head dimension comes before the sequence dimension. No additional computation is required. In Figure 2.8, dotted line arrows show the view operation that does the reshape. In addition, dashed lines on the weight matrices and Q, K, and V tensors show the conceptual division of those tensors into heads, even before the reshape. Note that these are divided into heads by groups of columns.

Why multiple heads? A single attention head computes a single attention pattern — one set of weights determining how much each position attends to every other position. Multiple heads allow the model to attend to different aspects of the input simultaneously. One head might focus on syntactic relationships, another on semantic similarity, and another on positional proximity. In practice, learned attention patterns are more complex than these examples, but the principle holds: multiple heads provide multiple "channels" for cross-position communication.

Attention score computation

With Q and K reshaped to (**H**, **S**, d_K), we compute the raw attention scores by taking the dot product of queries with keys:

$$\text{Scores} = \frac{QK^T}{\sqrt{d_K}}$$

The matrix multiplication QK^T produces a tensor of shape (**H**, **S**, **S**). For each head, we get a matrix shape (**S**, **S**) where entry (i, j) is the raw attention score between the query at token position i and the key at token position j. The division by $\sqrt{d_K}$ prevents the dot products from growing too large as the dimension d_K increases, which would push SoftMax into regions with vanishing gradients.

This (**H**, **S**, **S**) attention matrix is the computational signature of standard self-attention. Its size is quadratic in sequence length. For a 4096-token prompt, each head produces a 4096×4096 matrix, and when **S** = 100,000, each head's attention matrix is 100,000×100,000 and has 10 billion entries. In Figure 2.8, the score tensor looks small for small values of **S**, but if drawn to scale, for large values of **S** it would be huge compared to other tensors. This quadratic scaling is why long-context inference is expensive and why we will see that techniques like FlashAttention (Dao et al. 2022) (Section 6.1) are so important. FlashAttention computes the exact same result but avoids materializing the full **S**×**S** matrix in GPU memory by tiling the computation to fit one piece at a time.

The causal mask

In a decoder-only model used for autoregressive generation, each token should only attend to itself and to tokens that came before it — never to future tokens. This is enforced by a

causal mask: an upper-triangular matrix of $-\infty$ values added to the attention scores before SoftMax, as shown in Figure 2.9.

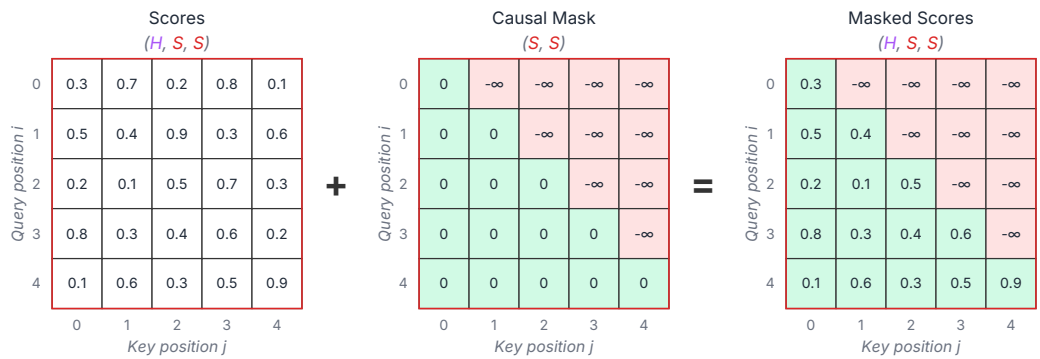


Figure 2.9: How the causal mask is applied in self-attention, shown for only one attention head for $\mathbf{S} = 5$

Note that the mask is shape (**S**, **S**), so it has to be broadcast across the **H** dimension in order to be added to the scores tensor. After adding the mask, we apply SoftMax along the key dimension (the last dimension). Any number added to $-\infty$ results in $-\infty$, and these $-\infty$ values become zero after SoftMax. This prevents the masked out positions from getting any attention weight, ensuring each position's attention weights sum to 1 over only the allowed (past and current) positions.

Weighted sum and output projection

The SoftMax output gives us the attention weights: a (**H**, **S**, **S**) tensor where each row sums to 1. We use these weights to compute a weighted sum of the value vectors:

$$\text{Weighted Values} = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_K}} + \text{mask}\right) V$$

The attention weights have shape (**H**, **S**, **S**) and V has shape (**H**, **S**, d_K), so the result has shape (**H**, **S**, d_K). We then reshape this back from (**H**, **S**, d_K) to (**S**, **D**) by concatenating the **H** heads, and apply a final output projection W_O of shape (**D**, **D**):

$$\text{Output}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \cdot W_O$$

The full self-attention block transformed an input of shape (**S**, **D**) to an output of the same shape (**S**, **D**). The shape is preserved.

Parameter count

For standard multi-head attention with $\mathbf{D} = 8192$, the parameter count per layer is:

- W_Q: $\mathrm{D} \times \mathrm{D} = 67.1\mathrm{M}$
- W_K: $\mathrm{D} \times \mathrm{D} = 67.1\mathrm{M}$
- W_V: $\mathrm{D} \times \mathrm{D} = 67.1\mathrm{M}$
- W_O: $\mathrm{D} \times \mathrm{D} = 67.1\mathrm{M}$
- **Total**: $4\mathrm{D}^2 \approx 268\mathrm{M}$ parameters per attention layer

With 80 layers, the attention parameters alone account for about 21.5B of Llama 3 70B's parameter count. The rest is mostly in the feed-forward layers, which we'll cover in Section 2.6.

> **Note**
>
> **Grouped-Query Attention (GQA) and friends.** Modern models often use fewer key-value heads than query heads. Llama 3 70B uses GQA (Ainslie et al. 2023) with 64 query heads but only 8 KV heads, reducing W_K and W_V from $(\mathbf{D}, \mathbf{D})$ to $(\mathbf{D}, 8 \times d_K) = (8192, 1024)$. This saves parameters and, more importantly, reduces the KV cache size by 8x. Multi-Query Attention (MQA) (Shazeer 2019) takes this further with a single KV head. Multi-head Latent Attention (MLA) (Liu et al. 2024) compresses the KV cache through learned low-rank projections. These attention variants are covered in Section 4.4.

2.5 The KV Cache

The KV cache is one of the most important concepts in LLM inference. It's a simple idea with far-reaching consequences for memory management, scheduling, and system design. This section explains why the cache exists, what it stores, and how much memory it consumes.

Training vs. inference

During training, the full sequence is available upfront. The model processes all $\mathbf{S}$ tokens at once, computing Q, K, and V for every position simultaneously. The causal mask ensures that position i only attends to positions $0, 1, \ldots, i$, but all positions are computed in parallel. There's no need to cache anything — the K and V tensors for all positions are computed in one pass, used immediately, and never needed again.

During standard inference, tokens are generated one at a time. At decode step t, the model generates token t based on all previous tokens $0, 1, \ldots, t-1$. The new token's query needs to attend to the keys and values of all previous positions. This creates a choice:

The naive approach: at each step, feed all t tokens through the model from scratch. This recomputes K and V for positions 0 through $t-1$, even though they haven't changed since

the last step. Each token t uses computation $O(t^2)$ for attention, and over $\mathbf{S}$ decode steps, the total computation for long sequences is $O(\mathrm{S}^3)$. We recompute increasingly longer sequences for each new token, and this is extremely wasteful.

The KV cache approach: store the K and V tensors from all token positions, accumulating one more of each with each step. At each decode step, compute K and V for *only* the new token, append them to the cache, and use the full cached K and V for the attention computation. This reduces the per token computation to $O(t)$ and the total computation to $O(\mathrm{S}^2)$ across all decode steps. The naive and KV cache approaches are contrasted in Figure 2.10.

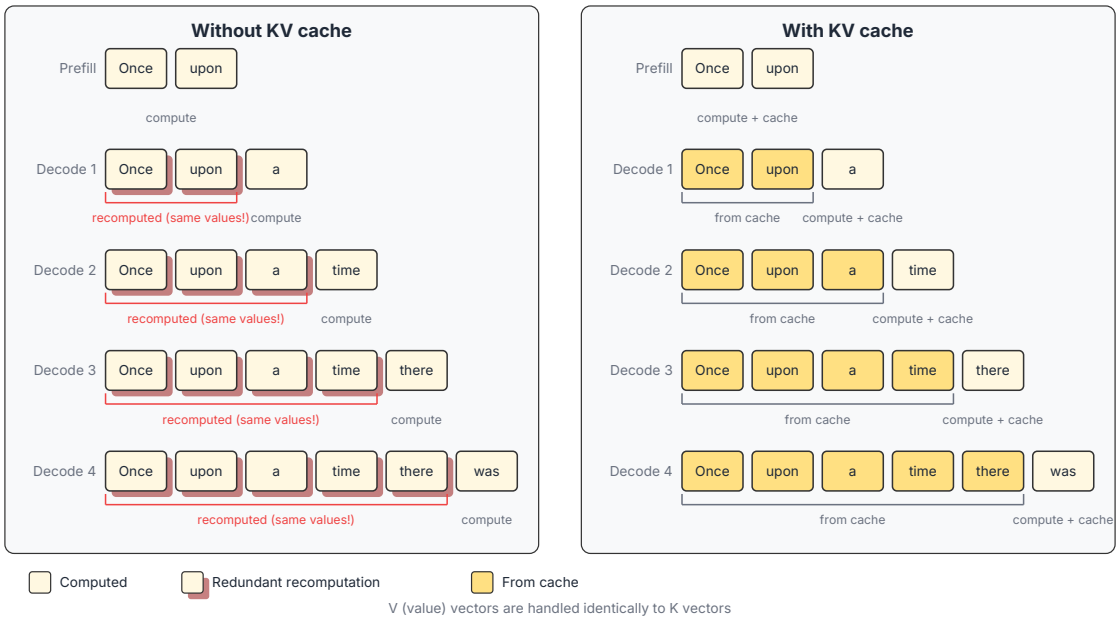


Figure 2.10: The KV cache avoids wasted recomputation of keys and values which never change over and over again

When we do inference with a KV cache, the attention calculation becomes much less computationally expensive because our query tensor shrinks from $(\mathbf{S}, \mathbf{D})$ to $(1, \mathbf{D})$. You can see this effect by comparing the original data flow for attention in Figure 2.8 to the KV cache attention data flow in Figure 2.11.

Memory consumption

The KV cache stores keys and values for every layer, every KV head, every cached position, and every element of the head dimension. The total memory for a single request is:

$$\text{KV cache memory} = 2 \times L \times H_{\text{KV}} \times S \times d_K \times b$$

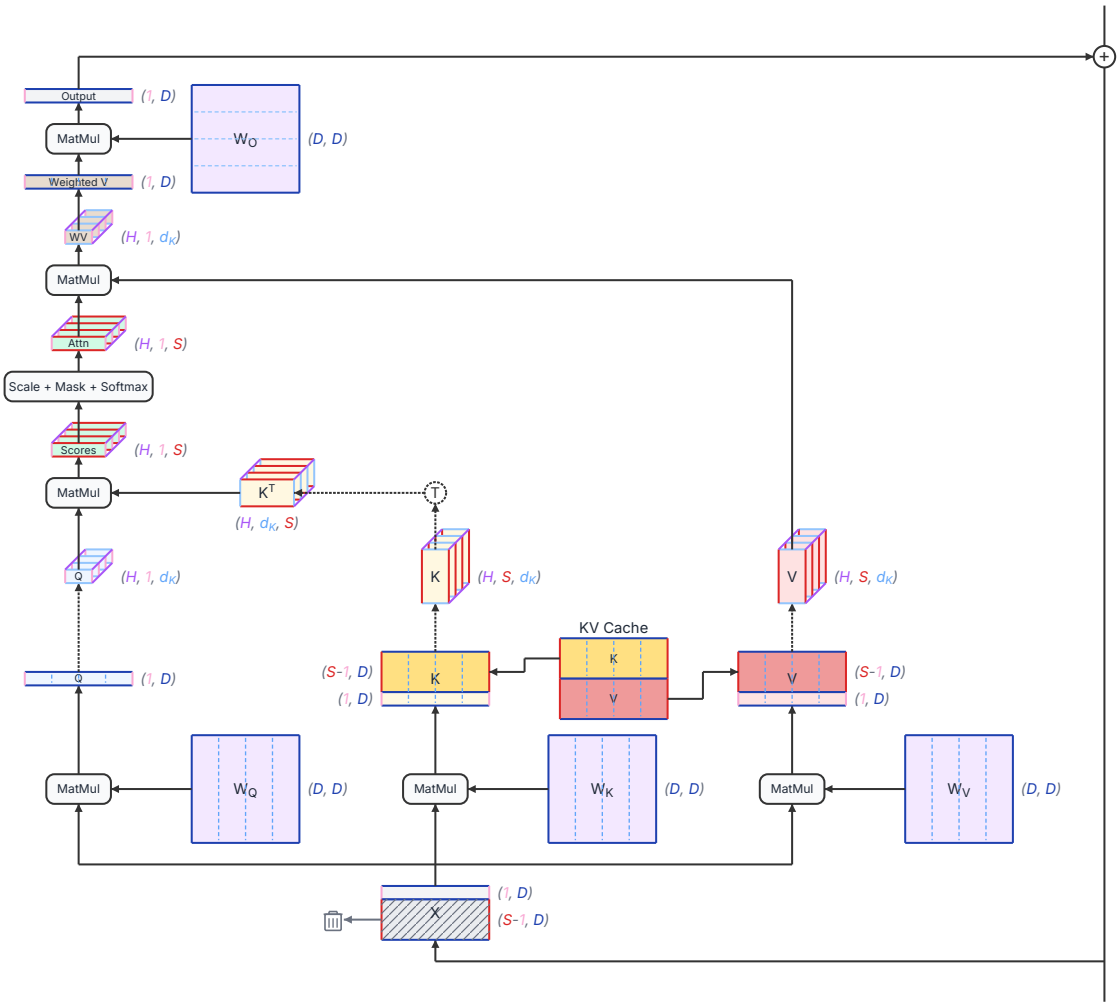


Figure 2.11: Data flow for multi-head attention when using a KV cache

where **L** is the number of layers, H_{KV} is the number of key-value heads (which is the same as **H** in a vanilla attention architecture), **S** is the current sequence length, d_K is the head dimension, and b is the number of bytes per element (2 for FP16/BF16).

For Llama 3 70B with GQA (8 KV heads), FP16 precision, and a 4096-token sequence:

$$2 \times 80 \times 8 \times 4096 \times 128 \times 2 \approx 1.34 \text{ GB}$$

That's 1.34 GB for a *single request*. If you're serving 40 concurrent requests, the KV cache alone consumes over 50 GB, which would be more than half of the memory on an 80 GB GPU that also needs to hold the model weights. This is why KV cache memory management is such a central concern in LLM serving systems.

The cache grows by a fixed amount at each decode step: $2 \times L \times H_{\text{kv}} \times d_K \times b$ bytes per token. For the Llama 70B numbers above, that's about 328 KB per token per request.

> **Note**
>
> **Forward references**: The KV cache drives many of the optimization techniques covered in the main chapters. Section 4.4 covers architectural techniques including reducing the number of KV heads, reducing the size of KV entries, and sharing KV cache entries across layers. Section 6.3 covers runtime techniques for managing the cache including PagedAttention (Kwon et al. 2023) which eliminates memory fragmentation, quantization which reduces the bytes per element, and eviction policies which remove cache entries under memory pressure.

Attention alternatives

Standard multi-head attention dominates deployed LLMs today, but it's not the only option. **State-space models** like **Mamba** (Gu and Dao 2023) and **linear attention** variants such as **Kimi Linear** (Kimi Team et al. 2025) replace the **(H, S, S)** attention matrix with a fixed-size recurrent state. Instead of caching all past keys and values (which grows linearly with sequence length), they maintain a constant-size state that gets updated at each step. This eliminates the problem of KV caches growing large, giving a fundamentally different inference profile — constant memory per step instead of linearly growing memory.

These architectures are out of scope for this book. We focus on standard multi-head attention because it remains dominant in the models that practitioners actually deploy at scale. There are model variants that reduce the number of layers running multi-head attention, but so far there are no top-performing models which exclusively use attention alternatives. It's worth knowing that alternatives exist, especially since the KV cache is a major source of inference complexity, but so far nothing less expensive has matched the capabilities of multi-head attention.

2.6 The Feed-Forward Network

Every transformer layer contains two main sub-layers: self-attention and a **feed-forward network (FFN)**, also called the **MLP** (multi-layer perceptron) block. While self-attention handles cross-position communication, the FFN operates on each position independently, transforming the representation at each position through nonlinear learned mappings. The FFN is where the majority of a model's parameters live.

Standard FFN

The original transformer (Vaswani et al. 2017) uses a two-layer FFN with a ReLU activation:

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

We can see the data flow in Figure 2.12. For simplicity, we do not show the bias terms, which could be abstracted as part of the matrix multiplication. The input x has shape **(S, D)**. The first linear layer projects up from **D** to an intermediate dimension **F**, originally **F** = 4**D**. The activation function is applied element-wise. Then, the second linear layer projects down from **F** back to **D**. When **F** = 4**D**, each weight matrix has $4\mathrm{D}^2$ parameters, for a total of $8\mathrm{D}^2$ parameters.

Gated MLP (SwiGLU)

Modern LLMs like Llama and PaLM don't use the standard FFN. They use a **gated** variant of the MLP called **SwiGLU** (Shazeer 2020), which adds a third linear layer and uses a gating mechanism:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = (\mathrm{SiLU}(xW_{\mathrm{gate}}) \odot xW_{\mathrm{up}})W_{\mathrm{down}}$$

where $\odot$ denotes element-wise multiplication and SiLU (also called Swish) is the activation function $\mathrm{SiLU}(x) = x \cdot \sigma(x)$.

The flow for the gated MLP is shown in Figure 2.13. The up and gate projections run in parallel, both mapping from **D** to a higher intermediate dimension **F**. Their outputs are combined by element-wise multiplication. The intuition is that the gate values select which features from the up-projection pass through. When an element of the gate is 0, the corresponding up projection element is zeroed out. When an element of the gate is 1, the corresponding up projection element is passed through as is. The gated up projection result is then projected down back to **D**.

Why go from two weight matrices to three? The gating mechanism has been shown empirically to improve model quality for a given parameter budget (Shazeer 2020). The intermediate dimension **F** is adjusted downward (from $4\mathrm{D}$ to roughly $\frac{8}{3}\mathrm{D}$) to keep the total parameter count approximately equal: $3 \times \mathrm{D} \times \frac{8}{3}\mathrm{D} = 8\mathrm{D}^2$, the same as the standard FFN's $2 \times \mathrm{D} \times 4\mathrm{D} = 8\mathrm{D}^2$

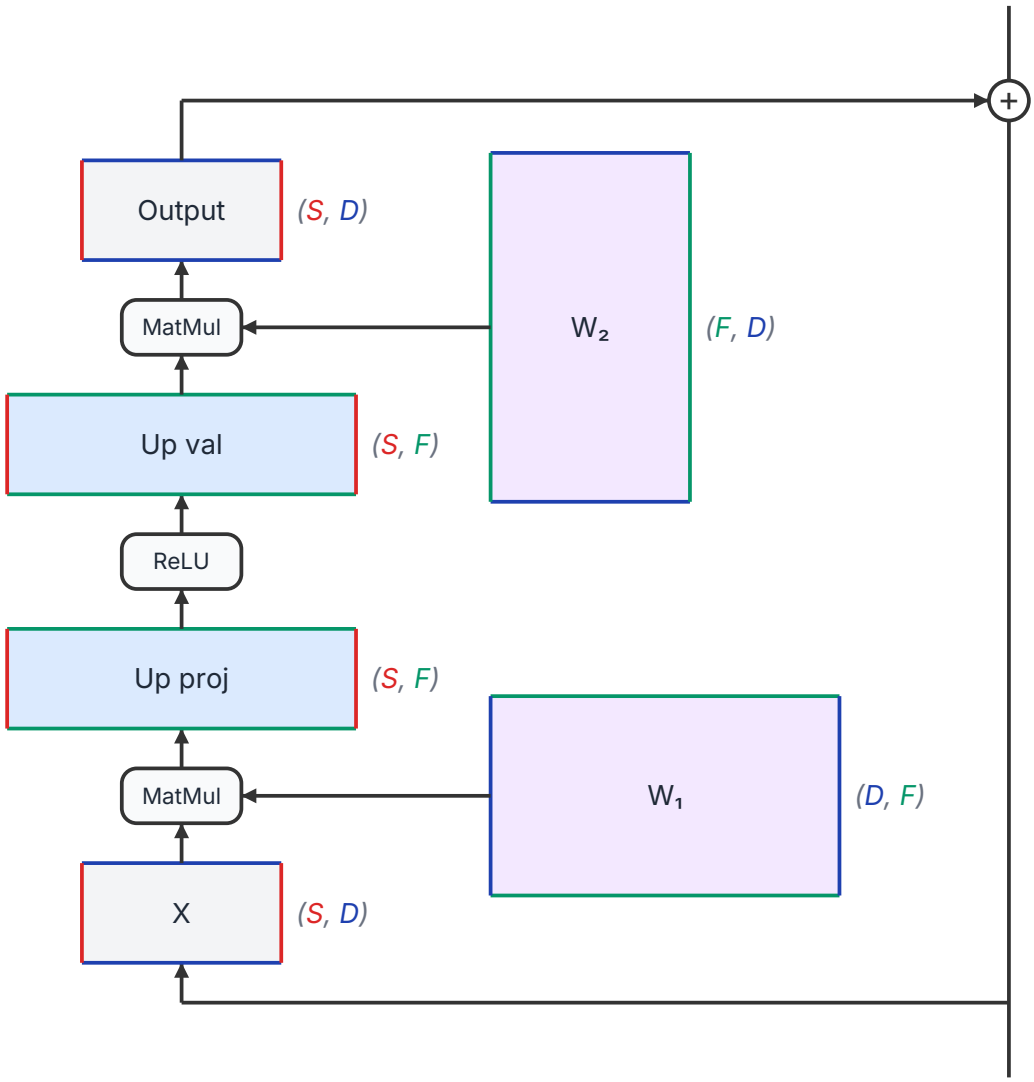


Figure 2.12: Data flow of weights and activations in a standard MLP layer

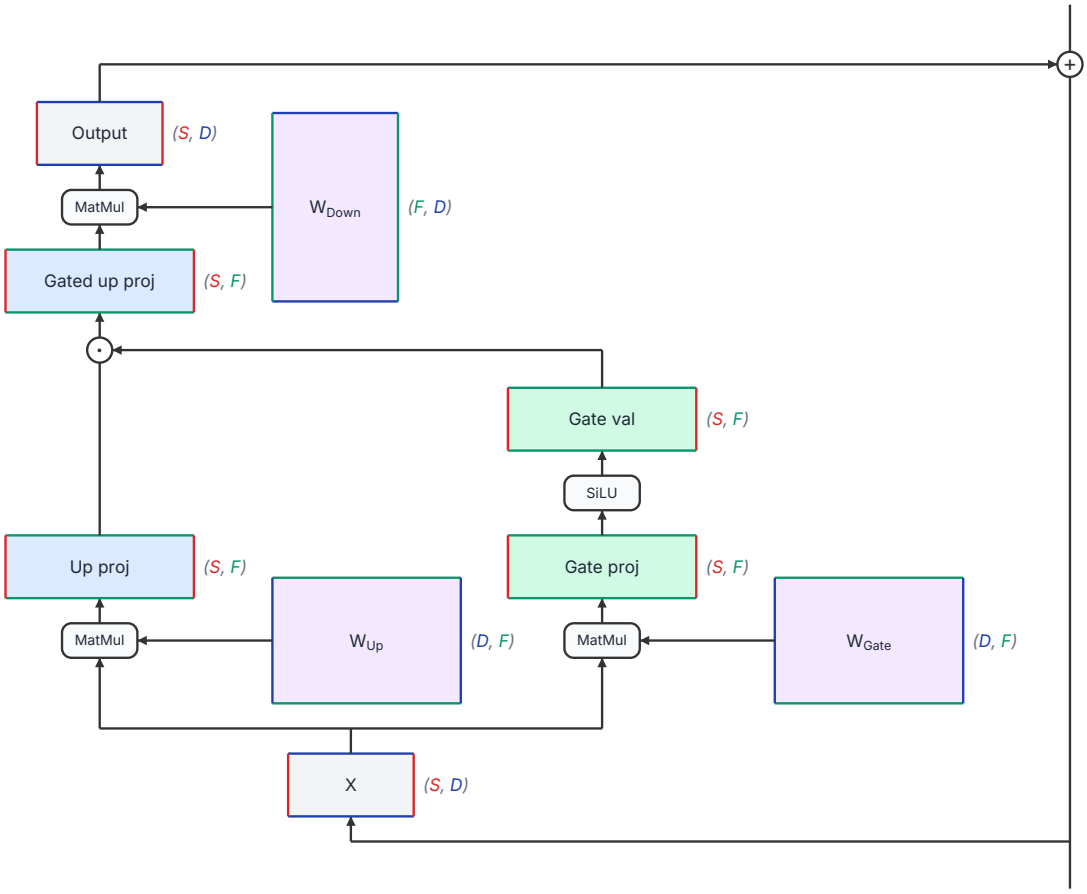


Figure 2.13: Data flow of weights and activations in a gated MLP layer

parameters. With $\mathbf{D} = 8192$, the parameter count is $8\mathrm{D}^2 \approx 537\mathrm{M}$ parameters per FFN layer. The FFN layers have double the number of parameters as the attention layers, so the FFNs comprise about two-thirds of the parameters, and the attention layers have about one-third of the parameters in a vanilla, traditional LLM.

Mixture of Experts (MoE)

Standard LLM models have one MLP per layer, and they have no choice but to apply the same MLP to every token. **Mixture of Experts (MoE)** architectures replace the single MLP with multiple MLPs called **experts** and a lightweight **router** that selects which experts to use for each token. Because not all of the MLP weights are used for each token, this is often referred to as a **sparse MLP**. In contrast, the original MLP architecture is referred to as a **dense MLP** architecture.

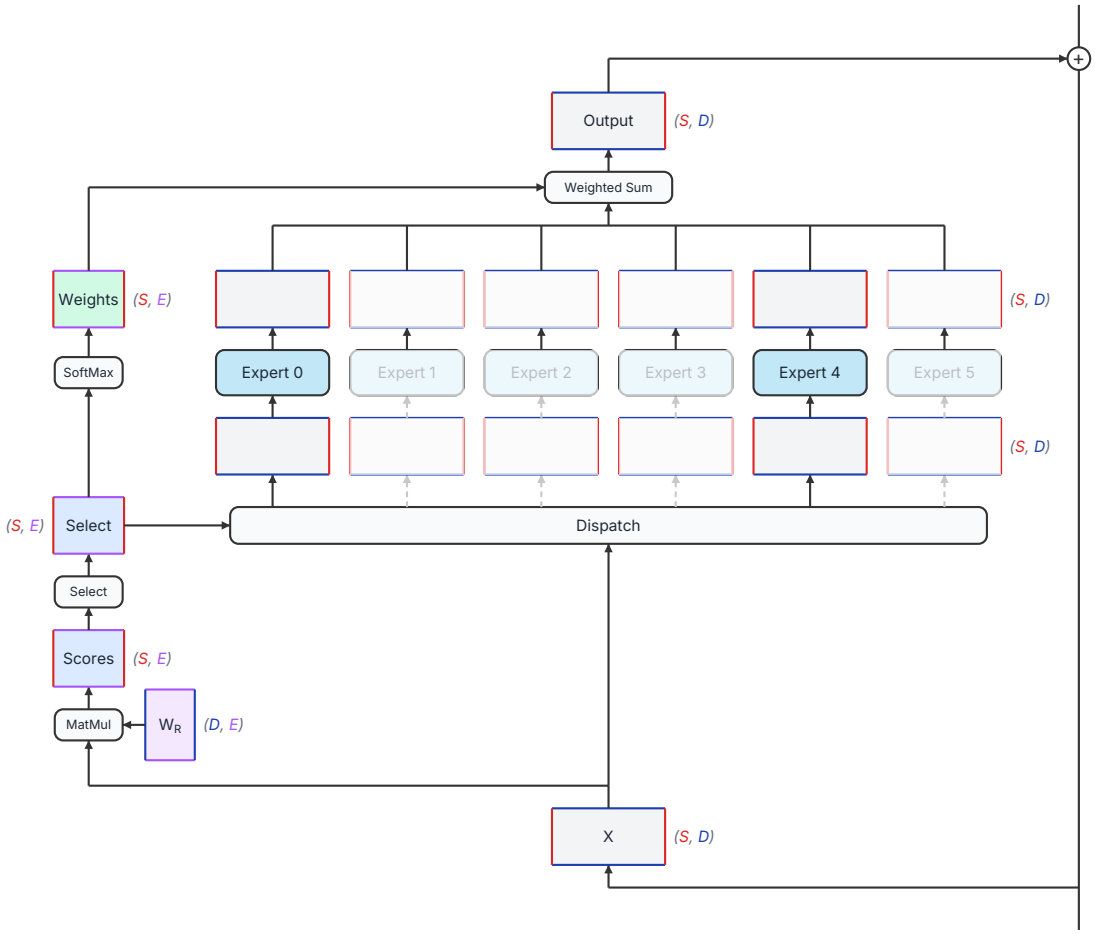


Figure 2.14: Data flow of weights and activations in an MoE MLP layer. This figure depicts a model with 6 experts, 2 of which are active.

Figure 2.14 shows the data flow for an MoE MLP layer. The router in MoE is usually a simple learned linear layer that produces a score for each expert. If there are **E** experts, the

router has a weight matrix of shape **(D, E)**. There are different designs for how the experts are chosen, and one of the simplest methods is for the router to select the top k experts with the highest scores. These top-k experts are said to be **active**. Note that there is a router in every layer, so the choice of experts differs each layer — this is not at all like having **E** separate full models which are chosen at the beginning based on the input token. The science of MoE layers is outside the scope of this book, but we will highlight a recent trend. Early MoE LLMs such as **Mixtral** (Jiang et al. 2024) had only a few experts, such as 8, and set k to a very small value, such as 2. More recent LLMs such as **Qwen 3** (Yang et al. 2025) have hundreds of experts and set k to a higher number, such as 8. They also have the concept of always-on experts called **shared experts**. Figure 2.14 shows what the vanilla data flow looks like with an MoE configuration with 6 total experts, 2 of which are active.

The outputs of the selected experts are combined using the router's SoftMax weights.

The key trade-off of MoE for inference is clear: **memory footprint is large** (all expert weights must reside in GPU memory) but **per-token compute is modest** (only k of **E** experts are active and consuming FLOPS). Mixtral 8×7B (Jiang et al. 2024) has roughly 47B total parameters but only ~13B active per token. This means the model takes up as much GPU memory as a 47B dense model but computes each token about as fast as a 13B dense model.

For multi-GPU serving, MoE models naturally lend themselves to **expert parallelism** — placing different experts on different GPUs. But this introduces all-to-all communication as tokens are routed to the GPU holding their selected expert. Load balancing across experts also matters: if the router sends most tokens to the same few experts, some GPUs do much more work than others. We'll discuss expert parallelism in Section 7.5.

Why the FFN matters for inference

The MLP layers typically account for roughly two-thirds of a model's parameters. On one hand, all these parameters require a lot of resources. All of the active parameters must be loaded from GPU memory as computational inputs, so this costs bandwidth. These active parameters are used in matrix multiplication, requiring a lot of compute. On the other hand, modern GPUs are highly optimized for this use case of loading tensors and doing matrix multiplication. GPUs perform the FFN calculations quickly, and there's not much that can be done to speed them up other than not doing them.

We wrap up the data flow conversation here. With this background into the nuts and bolts of the data and operations inside the LLM, we can now shift our attention to the bigger picture of how the model forward pass is used in inference and how we can measure and assess inference performance.

2.7 Further Reading

The original transformer paper (Vaswani et al. 2017) is the essential starting point. Even though the decoder-only variant drops half the architecture, the core ideas — scaled dot-

product attention, multi-head attention, positional encoding, and the feed-forward network — are all introduced there.

For intuitive explanations of the transformer architecture:

- Jay Alammar's "The Illustrated Transformer" (Alammar 2018) is considered a classic for explaining the original transformer architecture, with many beautiful diagrams.
- "The Annotated Transformer" (Rush 2018) walks you through Python code to understand the original transformer architecture, and is incredibly well written.
- Peter Bloem's "Transformers from Scratch" (Bloem 2019) builds the architecture from first principles with clear diagrams.
- Lilian Weng's "The Transformer Family Version 2.0" (Weng 2023) offers a more technical treatment, surveying transformer variants including the attention alternatives mentioned in Section 2.4.
- Andrej Karpathy's "Let's build GPT: from scratch, in code" (Karpathy 2023) walks through a complete implementation in about two hours.

For the specific architectural choices used in LLMs, the Llama papers (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023; Grattafiori et al. 2024) are a good reference because they clearly document the model dimensions and design decisions (SwiGLU, RoPE, GQA). The PaLM paper (Chowdhery et al. 2022) provides a thorough analysis of scaling behavior that motivates the decoder-only design. For more modern LLMs, Sebastian Raschka does a brilliant job covering all of the different architectures. A good starting point for his writings is "LLM Architecture Gallery" (Raschka 2025).

The MQA (Shazeer 2019) and GQA (Ainslie et al. 2023) attention variants are covered in Section 4.4. FlashAttention (Dao et al. 2022) is covered in Section 6.1.

3 Measuring LLM Inference

Now that we are familiar with the decoder-only LLM and how data flows through it, let's go deeper into the autoregressive nature of standard text generation and discuss how to quantify its performance. We will develop an understanding of the stages of inference, how we measure inference, and the performance model that underlies our computing hardware, in particular GPUs.

3.1 The Inference Pipeline

From the time you send a prompt to an LLM until you get a response back, a lot happens between those two moments. The full journey of a single request breaks down into a few stages. Some happen once per request and some repeat for every output token. Understanding the characteristics of the stages will help you focus your attention on the ones that actually dominate wall-clock time. In this section, as with most of the book, we will assume the LLM is running on a computer with a GPU. The principles will be the same if it is running on CPU, TPU, or other hardware.

The stages of inference

Let's walk through the stages needed to process a user request. We will break this down into four stages that have distinct properties in terms of when they happen, where they happen, and what kinds of resources they consume. These stages are shown in Figure 3.1.

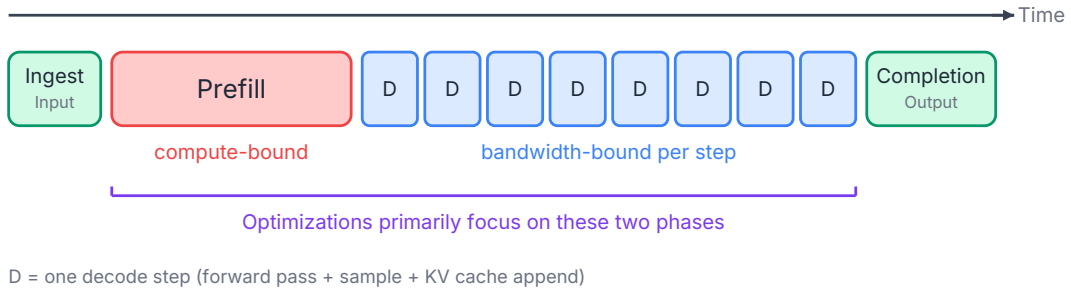


Figure 3.1: The stages of LLM inference for a single request. Ingestion and response completion are fast. Prefill and decode dominate wall-clock time.

We will assume the model is already loaded into GPU main memory. For a 70-billion-parameter model stored in FP16 (2 bytes per parameter), the weights alone are about 140 GB. Loading that volume of data from an NVMe SSD at 7 GB/s takes about 20 seconds, and from a network file system, it could take longer. Because this is so slow, on typical inference software, the weights are loaded into GPU memory at startup and they stay there for the lifetime of the serving session.

Ingestion is the entry point for each request. The system takes the user's prompt and forms a new request. The user's prompt may be merged with other content, such as a system prompt. Next, this finalized raw text is tokenized — converted from characters into the integer token IDs that the model actually works with — and any additional validation or preprocessing is applied. This stage is fast compared to the resource-intensive work involving the neural network model that follows and is typically handled on the CPU. We are skipping many details about the steps within ingestion because what they have in common is that they usually happen once and are relatively fast. One exception, which can significantly harm user experience, is request queuing. In a production system, requests may wait in a queue until there are resources available to process them, and if the system has multiple priority levels, lower priority requests may wait longer. Queueing time can be a significant contributor to overall latency, even though it doesn't consume system resources. The timing and granularity of scheduling request work can dramatically improve performance (up to the limits of system resources), and we will discuss scheduling in Chapter 5.

Prefill is where the heavy lifting begins. The model processes the entire set of input tokens in a single forward pass through the entire network — all layers, all attention heads, all FFN blocks. In particular, the attention layers calculate attention scores across all of the input tokens. Since the attention scores are a square whose sides are the number of tokens, the compute needed for attention grows quadratically with respect to the sequence length, and it becomes very expensive when there are many input tokens. Because all input tokens are known up front, the attention calculation can be done all at once using a large matrix-matrix multiplication between queries and keys. Despite how efficiently modern GPUs perform matrix multiplication, the size of the matrices involved means that prefill for all but the shortest prompts is **compute-bound** — the bottleneck is how fast the GPU can do the arithmetic, not how fast it can move data into the computational cores.

During prefill, each attention layer computes query, key, and value tensors for all of the input tokens. The keys and values will be needed again in the upcoming decoding stage. Rather than throwing these away and recomputing them during decoding, the system stores them in the **KV cache**, which we introduced in Section 2.5. Initializing the KV cache is the primary function of the prefill stage in modern LLM serving systems. The KV cache holds the key and value tensors for every attention head at every layer, for all tokens processed so far. The KV cache trades memory for compute: it uses GPU memory to store these tensors, but it eliminates the need to recompute them. The size of the KV cache created during prefill is:

$$\text{KV cache memory} = 2 \times L \times H_{\text{KV}} \times S \times d_K \times b$$

where the factor of 2 accounts for storing both keys and values, **L** is the number of layers, H_{KV} is the number of key-value heads (which may be less than the number of query heads,

as will be discussed in Section 4.4), **S** is the current sequence length, d_K is the dimension of each head, and b is the number of bytes per element (2 for FP16/BF16).

For a concrete example, consider a Llama-style 70B model with 80 layers, 8 KV heads per layer, head dimension of 128, and FP16 KV cache entries (2 bytes each). For a 4,096-token prompt:

$$\text{KV cache} = 2 \times 80 \times 8 \times 4096 \times 128 \times 2 \approx 1.34 \text{ GB}$$

That's 1.34 GB of GPU memory consumed by a single request's KV cache, just from the input tokens. This memory usage is one of the primary constraints on how many requests can be served concurrently, which we'll explore in Section 4.4, Section 6.3, and other sections.

After prefill completes, the system has two things: the KV cache populated with entries for all input tokens and the logits for the last input position's next token prediction, the latter of which are used to generate the first output token.

> **Note**
>
> For a detailed look at exactly how the key, value, and query projections are computed during the transformer's forward pass and how they relate to the KV cache, refer to sections 2.4 and 2.5.

Decode is the phase where the LLM's response is generated (except for the first token). The model typically produces output tokens one at a time, where each new token depends on all previously generated tokens. This sequential dependency comes from how the model defines the probability of an output sequence. Given input tokens x, the probability of generating output sequence $y = (y_1, y_2, \dots, y_T)$ is decomposed using the chain rule of probability, where each output token's probability distribution is conditioned on both the prompt x and all of the prior output tokens:

$$p(y \mid x) = \prod_{t=1}^{T} p(y_t \mid y_{<t}, x)$$

The model estimates each conditional probability $p(y_t \mid y_{<t}, x)$ with a forward pass. Because each token's probability depends on all previously generated tokens, the model doesn't generate token y_t until it has committed to tokens y_1 through y_{t-1}. This gives us the familiar autoregressive loop:

- Run a forward pass for one new token, reading all active weights and the full KV cache
- Obtain the logit distribution over the vocabulary
- Sample a token
- Append new key-value entries to the KV cache, and
- Repeat until the model generates a stop token or hits a length limit

Each iteration of this basic loop is called a **decode step**. The forward pass in a decode step processes just one token (or one token per batch entry, in a batch), so the query-key operations are **matrix-vector multiplications** rather than the matrix-matrix multiplications of prefill. The keys are still a matrix of **S** vectors, but now they're being multiplied by a single query vector instead of a matrix of **S** vectors.

The one-token-at-a-time nature of typical decoding is the central challenge of LLM inference. Each decode step involves reading all the model weights and the full KV cache, but only to compute a single token's worth of output. The ratio of computation to data movement is very low, which makes decode steps typically **memory-bandwidth-bound**. Here, the bottleneck is how fast the GPU can read data from GPU memory, not how fast it can do math. A large portion of this book is about techniques to make this phase less painful.

It's worth noting that the left-to-right, one-token-at-a-time autoregressive factorization is not the *only* mathematically valid way to express the joint probability of a sequence. There are other approaches that calculate the same joint distribution differently. The left-to-right decomposition is a convenient modeling choice, but it imposes a strict sequential dependency at inference time. Several techniques in Section 6.5 — speculative decoding, multi-token prediction, and Medusa heads — work by finding ways to break or shortcut this sequential dependency while preserving the same output distribution.

At the end of each decode step, **sampling** selects the next token. After the forward pass has produced a logit vector over the vocabulary, one token ID is selected. Common strategies include:

- greedy decoding (always pick the highest-probability token),
- temperature sampling (scale logits before sampling randomly),
- top-k sampling (restrict to the k most probable tokens), and
- top-p / nucleus sampling (restrict sampling to the smallest set of tokens whose cumulative probability exceeds p)

Sampling adds negligible overhead relative to the preceding forward pass. The forward pass involves billions of multiply-accumulate operations across all the model's layers. Sampling merely involves a softmax over the vocabulary (typically 30K-200K entries) and a random draw, so sampling is not where all the time goes.

Completion wraps things up. The model has finished because it either generated a stop token, hit a maximum length limit, or has been interrupted by the serving system. The accumulated response is detokenized back into text, the KV cache memory is freed, and the result is returned to the user. In most production deployments, the response is **streamed** back to the user as tokens are generated, rather than waiting for the full response to complete. Streaming doesn't change what's happening on the GPU — each decode step produces the same token whether it's streamed immediately or batched for delivery later — but it reduces how long users have to wait to see parts of the LLM response. Whether batch or streaming, completion steps are like ingestion in that they are usually fast one-time operations that happen on the CPU.

Where the time goes

Of the five stages above, the vast majority of compute and memory bandwidth is consumed by prefill and decode. Ingestion and completion are fast operations on CPU that together account for a tiny fraction of request latency. Note that we are excluding queueing time from ingestion for now, treating that as a fixed cost associated with workload. Queueing time can be long for a system under heavy load, and various techniques associated with scheduling are discussed in Chapter 5. Aside from scheduling, most of the techniques covered in this book target prefill and decode, because that's where the time goes and where the greatest opportunity for improvement lies.

Two fundamentally different regimes

While prefill and decode are both expensive, the contrast between the types of resources they need is worth emphasizing.

During prefill, we have a large set of tokens to process, so we can keep the GPU's compute units busy with very large matrix multiplication operations. During decode, we're typically generating one token at a time, so we need many forward passes which each require much smaller computations that usually involve reading a lot of data.

This means prefill and decode need different resources from the hardware. Prefill wants raw compute horsepower. Decode wants memory bandwidth. As we'll see in Section 3.4, the roofline model gives us a precise way to reason about where the boundary between these two bottlenecks falls for a given GPU or hardware accelerator.

Many of the optimizations covered in this book target just prefill or just decode, and some of the techniques span between the two of them. Understanding the fundamental difference in resource needs between prefill and decode will provide a good foundation for everything that follows.

3.2 Performance Metrics

Now that we have a mental model of what happens during inference, the natural next question is: how do we know if our system is doing it *well*? This section introduces the metrics you'll encounter in conversations about LLM serving. We'll group them by what they measure, as different concerns utilize different metrics.

User-facing latency metrics

The metrics that matter most in interactive applications are the ones users actually feel.

Time to First Token (TTFT) is the wall-clock time from when a request is submitted to when the first output token is produced. In a chat application, this is how long the user stares at a blank screen before text starts appearing. TTFT is significantly impacted by the

duration of the prefill phase because ingestion usually is fast, we don't need to decode many tokens to start streaming text to the user, and completion is also fast. Since the system has to process the entire input before it can begin generating any output, longer prompts mean longer prefill, which means higher TTFT.

Note that TTFT isn't as simple as the prefill duration plus fixed costs for the other phases. In systems under heavy load, the request may sit in a queue before being serviced. Also, in streaming deployments, the first token might be generated by the model but not immediately delivered to the user. Tokens can be buffered for various reasons, including network batching, detokenization requiring multiple tokens to resolve a word boundary, or framework-level batching of streamed chunks. So the TTFT the user experiences can be somewhat higher than the time the inference engine measures internally for prefill and generation of the first token.

Time Per Output Token (TPOT), also called **Inter-Token Latency (ITL)**, is the average time between successive output tokens during generation. This is what governs how fast text appears to stream after that first token shows up. TPOT is a function of the decode process. When each decode step produces one token, TPOT is essentially the time per decode step. For a fluent reading experience in interactive applications, you generally want TPOT below about 50–100 milliseconds (which equates to 10-20 tokens per second). Go much higher, and the output starts to feel sluggish.

TPOT is an average, and individual decode steps can vary. The KV cache grows with each token, so later decode steps read slightly more data than earlier ones. In practice, this variation is usually small relative to the total step time, but it can matter for very long sequences.

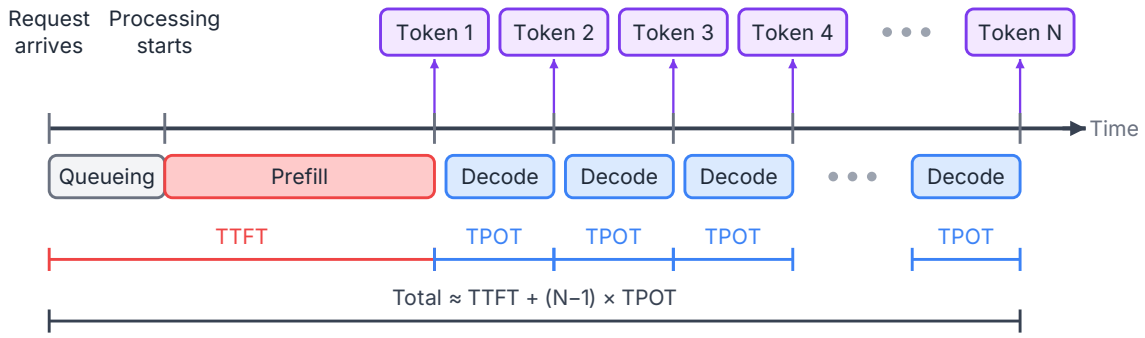


Figure 3.2: Request timeline showing TTFT and TPOT durations

Together, TTFT and TPOT define the end-to-end latency of a request, as shown in Figure 3.2. The total time to generate a response of N output tokens is approximately $\text{TTFT} + (N - 1) \times \text{TPOT}$. For short outputs, TTFT dominates. For long outputs, TPOT dominates. This is why the workload shape — the ratio of input length to output length — matters so much for understanding metrics and performing optimizations, as we'll discuss in Section 3.3.

Throughput

While latency metrics describe the experience of a single request, **throughput** measures how much total work the system gets done. The most common unit is **tokens per second (TPS)** — specifically, output tokens per second combined across all requests currently being served. You'll also see **requests per second (RPS)** and **requests per minute (RPM)**, which measure how many complete requests the system finishes in a given time window. Note that a system processing one request that generates 100 tokens per second has 100 TPS, and a system processing 100 requests that each generate 1 token per second also has 100 TPS. RPS is highly dependent on the workload.

The latency–throughput tradeoff

Latency and throughput are in fundamental tension, and this tradeoff is at the heart of inference system configuration.

One key mechanism impacting latency and throughput is **batching**. If you process requests one at a time, each request gets the lowest possible latency while being serviced, since the hardware is entirely dedicated to that single request. But the GPU is badly underutilized, especially during decode, where a single request doesn't generate enough arithmetic to keep the compute units busy.

If instead you batch multiple requests together, the GPU can do more useful work per memory read. Reading the model weights once and applying them to 32 requests in a batch is far more efficient than reading them 32 separate times, once per request. With larger batches, throughput goes up, but now each individual request might have to wait before processing begins, because the system is pausing to accumulate enough requests for a full batch. Also, each decode step takes slightly longer because there's more work per step, so with larger batches, latency goes up a little.

This tradeoff is not just theoretical. In production serving, batch configuration significantly impacts scheduler performance, as shown in Figure 3.3. A scheduler that waits longer to fill batches achieves higher throughput but increases TTFT for waiting requests. A scheduler that dispatches immediately may keep TTFT lower but hurts throughput. Finding the right operating point depends on the application's requirements, which is where the next metric comes in.

Goodput

Raw throughput doesn't tell the whole story. **Goodput** is throughput counting only responses that actually meet the application's requirements — its service-level agreement (SLA). The SLA for a chat application might specify a maximum TTFT, a maximum TPOT, a maximum end-to-end latency, or some combination of these.

A system can have impressive throughput numbers but poor goodput. If you push batch sizes so high that 30% of requests exceed the latency budget, those requests don't count toward

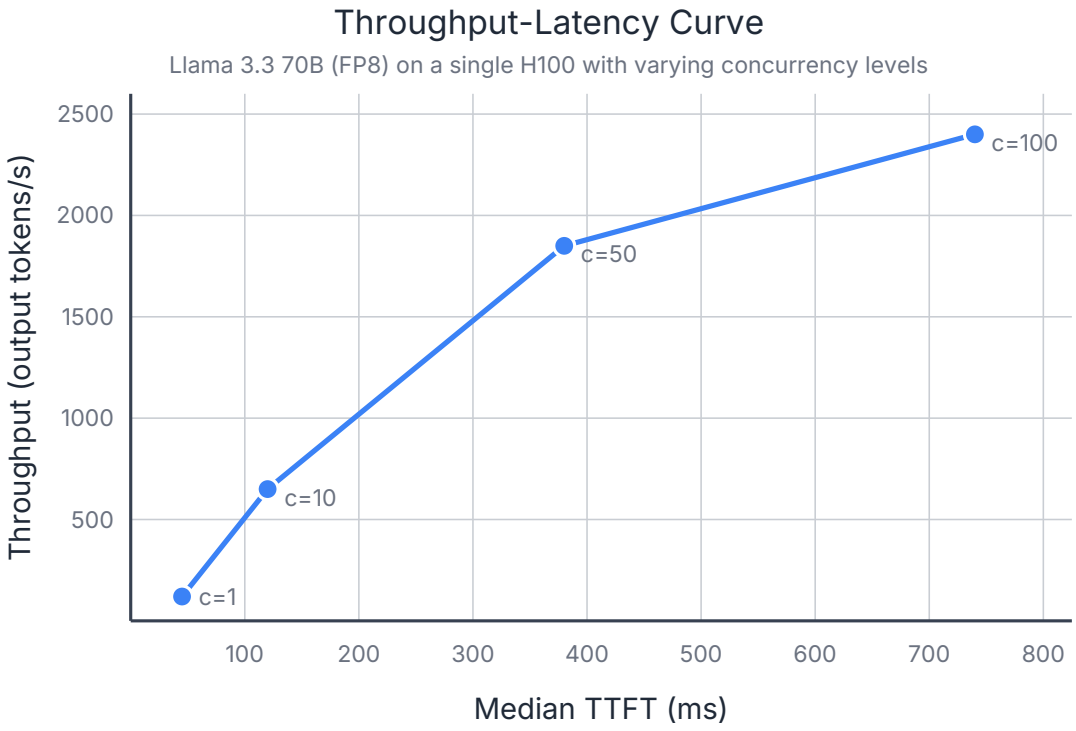


Figure 3.3: Throughput-latency curve showing how throughput gains diminish as latency increases. Data from Spheron (2025), benchmarking Llama 3.3 70B (FP8) on a single H100 system running vLLM.

goodput, even though the system did the work. Similarly, if the system is so overloaded that it starts preempting or dropping requests, the raw TPS might look fine while the actual useful output is much lower. Goodput is the metric that connects basic system performance to whether users are actually getting a good experience.

Cost efficiency

At scale, inference cost is often a key optimization target. **Cost per token** measures how much it costs in dollars (or GPU-hours) to generate output tokens. It's roughly inversely proportional to throughput — if you double your throughput on the same hardware, you halve your cost per token. This means that most of the techniques covered in this book that improve throughput will reduce cost.

The picture gets more nuanced when you factor in different hardware devices (each with different cost and performance profiles), different precision formats, and the amortized cost of hardware across many workloads. But as a first approximation, optimizing for throughput is optimizing for cost.

While cost per token is often measured in aggregate, it can also be useful to break out cost betweeen input and output tokens, or between prefill and decode. In fact, major providers usually charge separately for input and output tokens, reflecting the different costs for prefill and decode compute.

Hardware utilization metrics

The metrics above describe what the system achieves. The next two measure how efficiently it uses the hardware to get there. To diagnose whether or not you're maximizing the capabilities of your hardware, you need different metrics.

Model FLOPs Utilization (MFU) is the fraction of the GPU's peak floating-point throughput that is actually used for model computation. This metric was introduced in the PaLM paper (Chowdhery et al. 2022) and has become a standard efficiency metric. If your GPU can do 990 TFLOP/s at FP16, and your model computation actually achieves 300 TFLOP/s, your MFU is about 30%.

MFU is most useful for characterizing compute-bound workloads like prefill. A low MFU during prefill suggests that something is preventing the GPU from doing math at full speed — perhaps memory transfers, kernel launch overhead, or poorly shaped matrix multiplications. We'll see how to use the roofline model to reason about this precisely in Section 3.4.

Model Bandwidth Utilization (MBU) is the analogous metric for memory bandwidth. MBU is the fraction of peak memory bandwidth that is actually used for reading model data. During decode, the bottleneck is typically how fast you can stream model weights and KV cache entries from GPU main memory. If your GPU has 3.35 TB/s of memory bandwidth and your decode step actually achieves 2.5 TB/s, your MBU is about 75%.

MBU is a good diagnostic for decode activity. A low MBU during decode means the memory system is not being fully utilized — perhaps due to small batch sizes, kernel launch gaps between layers, or overhead from memory management. Together, MFU and MBU give you a quick read on whether your system is limited by compute efficiency, memory bandwidth efficiency, or something else entirely. Note that manufacturers often report peak compute and bandwidth numbers on very specific workloads that yield the highest numbers. You should not expect to get close to 100% MFU or MBU for real-world use cases.

Capacity metrics

These metrics describe the system's high-level resource utilization.

Concurrency is the average number of in-flight requests the system is serving simultaneously. This is primarily constrained by GPU memory: each active request needs space for its KV cache, and the KV cache grows with sequence length. If your GPU has 80 GB of HBM and your model weights consume 30 GB, you have roughly 50 GB left for KV caches (minus some overhead). How many requests fit in that budget depends on the model architecture, the precision of the KV cache, and how long the sequences are. The scheduler policy dictates whether to reserve memory conservatively or to pack it more aggressively and risk running out of memory.

GPU memory usage is the average amount of GPU memory that is being used. Usage is comprised of four components: model weights, KV cache, activations temporarily stored during the forward pass, and framework overhead (memory allocated by the CUDA runtime, the serving framework, and so on). The weights are fixed once the model is loaded. Activations are transient and relatively small. The KV cache is the variable that dominates, and it scales with both concurrency and sequence length.

Maximum sequence length supported is the longest sequence (counting both input and output tokens) that the system can handle. Note that even if the model was trained to support 128K tokens, your deployment might be limited to 32K if that's all the KV cache memory you can afford while maintaining your target concurrency. The serving framework typically sets this as a hard constraint at model load time.

Preemption metrics

When the system runs out of GPU memory to serve all active requests and their KV caches, something has to give. The scheduler may **preempt** a request, forcing it back to the waiting queue and freeing its KV cache entries to make more memory available. When the preempted request resumes, the system either recomputes its KV cache from scratch or swaps back in the entries it saved in CPU main memory at the time of preemption.

Three metrics diagnose preemption behavior: the **preemption rate** (how often preemption happens), **recomputation overhead** (the extra prefill work to rebuild evicted KV caches), and **swap latency** (the time to move KV cache data between GPU and CPU memory). These are important in high-load scenarios, because a system that preempts too aggressively

can waste significant resources on preemption activity. We will cover preemption policies and their tradeoffs in more detail when we discuss scheduling and memory management in Section 5.4.

Putting it all together

These metrics are not independent. Increasing batch size improves throughput and MBU but can hurt TTFT and TPOT. High concurrency improves throughput but can increase preemption rate. The art of inference optimization navigates these tradeoffs, trying to maximize user experience while minimizing cost.

3.3 Workload Patterns

Not all LLM requests look the same. A one-line classification question and a multi-page document summary put very different demands on the inference system, even if they're running on the same model with the same hardware. Before we get into the mechanics of how inference systems are built and optimized, it's worth stepping back and thinking about what kinds of work they're actually asked to do. The shape of the workload greatly impacts many metrics.

The input/output quadrants

The simplest way to characterize an LLM workload is by the length of the input and the length of the output. These two dimensions create four quadrants, shown in Figure 3.4, and each one has a distinct performance profile.

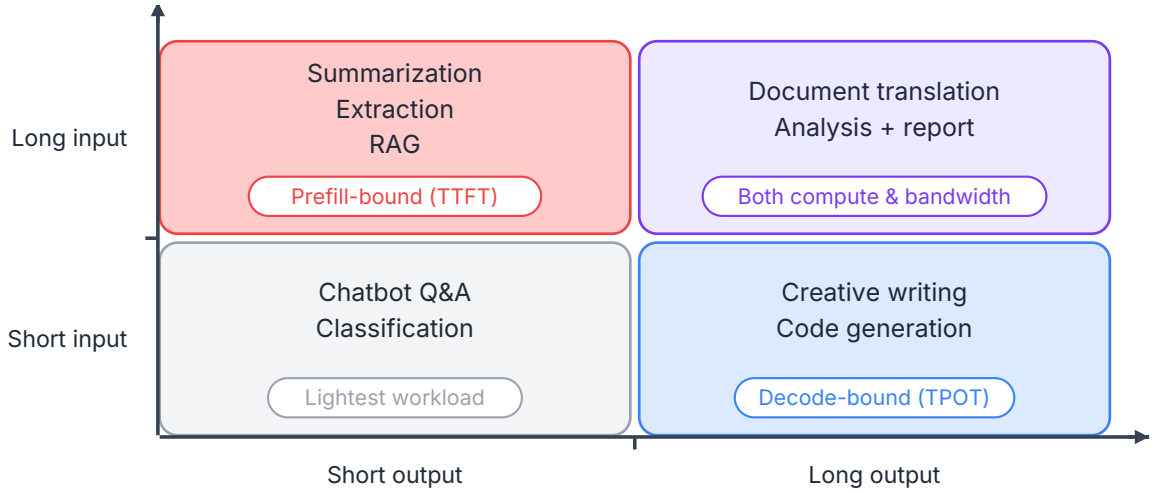


Figure 3.4: The four workload quadrants defined by input and output length, with example scenarios

Short input, short output is the lightest quadrant. Think of a simple one-off chatbot exchange or a classification task: a sentence or two goes in, and a sentence or two comes out. Prefill is fast because there aren't many input tokens. Decode is fast because there aren't many output tokens to generate. The KV cache stays small. Individually, these requests are cheap. For these requests serving systems will achieve the highest requests per minute, because they are so cheap.

Short input, long output is what you get with creative writing, open-ended code generation, or detailed explanations. Prefill is still quick, but decode dominates the request's lifetime because the model has to generate hundreds or thousands of tokens sequentially. The KV cache grows with each decode step, so memory pressure increases over the life of the request. For the user, what matters here is the sustained token generation rate — the TPOT we defined in the last section. A slow decode loop means a long wait for the full response.

Long input, short output is the pattern you see in summarization, information extraction, and retrieval-augmented generation (RAG). A large document or a long retrieved context goes in, and a relatively brief answer comes out. Prefill is expensive because the model has to process a large number of input tokens. But once prefill is done, decode finishes quickly. The KV cache is large from the start, but it doesn't grow much further. For the user, TTFT is the metric that matters most, since the wait is almost entirely in prefill.

Long input, long output is the most demanding quadrant. Document translation, detailed analysis with a lengthy report, and long multi-turn conversations all fall here. Prefill is expensive, decode is expensive, and the KV cache is large throughout. These requests consume the most GPU memory and occupy a batch slot for the most time. They're also the hardest to schedule efficiently alongside other requests, because they consume so many resources.

Why the quadrants matter

The quadrant a request falls into determines where the system's bottlenecks will show up. Long inputs mean expensive prefills and large initial KV caches. Long outputs mean many decode steps and growing KV caches. Sequence length directly affects how many requests can be batched together, because the KV cache for each active request takes up GPU memory. A batch of long-context requests might only fit a handful of concurrent requests, while a batch of short exchanges might fit many.

In practice, most production workloads are a mix of quadrants. A serving system might see a stream of short chatbot queries punctuated by occasional long summarization requests. The long requests consume disproportionate memory and compute, which can slow down the short requests sharing the same GPU. Managing this mix efficiently is one of the challenges of inference scheduling, which we'll explore in Chapter 5.

Batch vs. real-time inference

Beyond the shape of individual requests, the other major dimension of a workload is whether it runs in **batch** (offline) mode or **real-time** (online) mode. This distinction changes which

metrics you care about and how aggressively you can optimize.

In **batch inference**, you have a known set of requests to process and no one is waiting on the other end for an immediate response. Maybe the user is scoring a dataset, generating synthetic training data, or running evaluations. The metric that matters most is **throughput** — tokens per second across all requests — and its close relative, **cost per token**. Because there's no latency constraint, the goal is to take the time to plan the best way to minimize the total cost of processing these requests. You can fill batches to capacity, use cheaper or slower hardware, and let individual requests wait while the system finds the most efficient packing. Larger batches mean better GPU utilization, which means lower cost.

In **real-time inference**, a user (or another system) is waiting for a response, and how fast that response arrives directly affects the experience. Here, the metrics that matter most are **TTFT** and **TPOT**. A chatbot that takes five seconds to start responding feels broken, even if its overall throughput is excellent. This means the serving system has to balance resource utilization against latency. You can't hold requests in a queue too long to form larger batches, because that increases TTFT for every waiting request.

Some systems serve both kinds of traffic, and some offer different priority tiers — high-priority real-time requests that get scheduled immediately, and lower-priority requests that can wait to fill in the gaps. Batch inference is the easier workload to optimize for, so many of the optimizations in this book will focus on real-time latency issues. The concept of goodput really helps interpret system performance in a way that measures user experience.

Arrival patterns

The final piece of the workload picture is how requests arrive over time. Even in a real-time serving scenario, the demands on the system vary dramatically depending on the **arrival rate** and its **burstiness**.

If requests arrive at a steady, predictable rate, the system can maintain a stable batch size and keep GPU utilization relatively consistent. But real-world traffic is rarely that well-behaved. User-facing services see spikes — a sudden surge after a product launch, peak hours during the workday, or a viral moment that sends traffic through the roof. During a burst, the system has to either queue requests (increasing latency), drop them, or have enough spare capacity to absorb the spike.

The arrival pattern also affects how well the scheduling system can batch requests together. Requests that arrive close together in time can potentially share a batch. Requests that arrive in isolation may run in small, inefficient batches, wasting GPU compute. The more variable the arrival pattern, the harder it is to maintain consistently high utilization. We'll see in Section 5.1 how modern schedulers use techniques like continuous batching to adapt dynamically to changing arrival patterns, rather than waiting for a fixed batch to fill up.

In summary, understanding your workload — the mix of input/output lengths, the latency requirements, and the arrival patterns — is the first step toward choosing the right inference configuration. A system tuned for batch summarization of long documents looks very different

from one tuned for low-latency chatbot responses, even though they might be serving the same model.

3.4 The Hardware Model and Bottleneck Framework

In Section 3.1, we explained that prefill is compute-bound and decode is memory-bandwidth-bound. That's a good start, but to really be able to reason about optimizations more precisely, we need a more detailed picture of the hardware. This section covers just enough GPU architecture to build that picture, then introduces the roofline model as a framework for categorizing compute and memory bandwidth bottlenecks. This won't be a GPU deep dive or programming tutorial. We'll cover just enough to understand inference performance quantitatively.

> **Note**
>
> This book focuses on NVIDIA GPUs because they dominate the LLM inference landscape today. However, the bottleneck framework we're about to introduce applies equally to other hardware. For example, AMD's MI300X, Google's TPUs, and other accelerators all have different compute throughput ceilings and memory bandwidth limits, but the same approach to analyzing bottlenecks will work for those devices as well. We will briefly address alternative hardware in Chapter 7 when we discuss multi-device deployment.

GPU compute throughput

A modern GPU like the NVIDIA H100 is organized around **streaming multiprocessors (SMs)** that do the computing. The H100 SXM has 132 SMs, and each SM contains multiple types of execution units. For LLM inference, the most important of these are the **tensor cores**, the specialized hardware units designed to perform matrix multiplications very fast. Tensor cores are what give modern GPUs their enormous throughput for the matrix multiplications that dominate neural network models. All of the SMs on a GPU are able to simultaneously do work in parallel, which scales up their throughput.

The peak throughput of floating point operations (assuming the SMs don't ever have to wait for data) depends on the numerical precision you use. Lower precision means more operations per cycle, which translates directly into higher throughput. For the H100 SXM, relative throughput for several data types is shown in Table 3.1.

Table 3.1: Compute throughput figures for the NVIDIA H100 SXM. Source: NVIDIA (2023b).

Precision	Peak throughput (TFLOPS)
TF32	494
FP16	989
BF16	989

Precision	Peak throughput (TFLOPS)
FP8	1,987
INT8	1,987

These numbers are theoretical peaks — the maximum the hardware can deliver if every tensor core is busy on every cycle. In practice, you never hit the peak, but it sets the ceiling. You get roughly double the throughput when you halve the precision. This is one reason model quantization (covered in Section 4.1) is such a useful optimization lever. When you reduce the precision of model weights and activations, you're saving memory and also increasing compute throughput.

The memory hierarchy

Raw compute throughput only matters if you can feed data to the tensor cores fast enough. This is where the memory hierarchy comes in. An NVIDIA GPU has four levels of memory, shown in Figure 3.5. The closest levels are the fastest but also the smallest, and as the levels get larger, they get farther and slower, trading off speed for capacity.

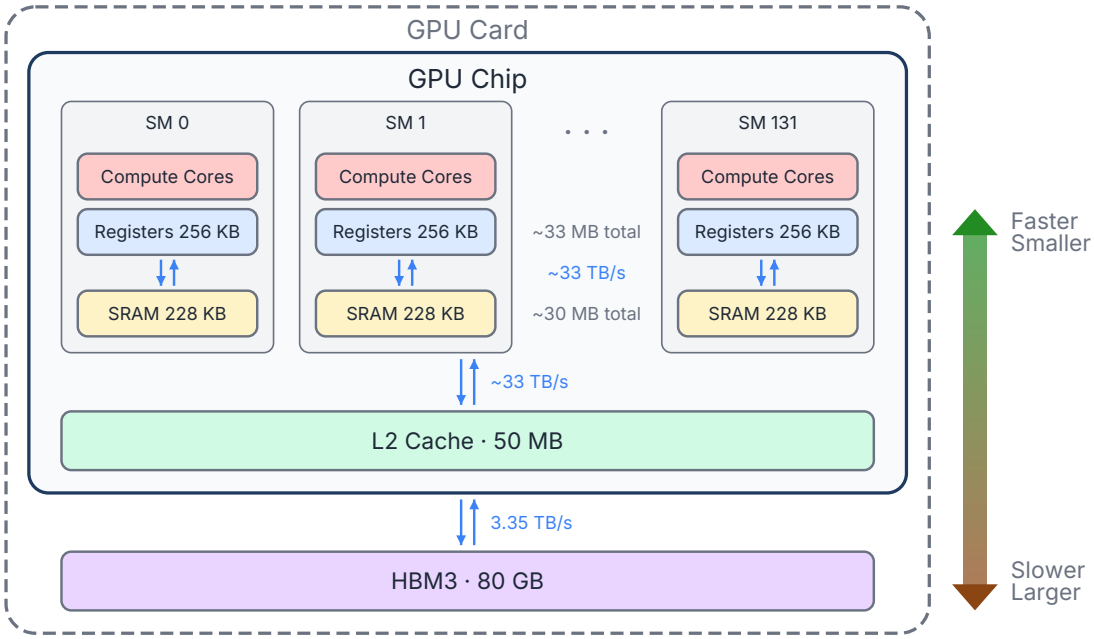


Figure 3.5: GPU memory hierarchy showing registers, SRAM, L2 cache, and HBM with capacity and bandwidth at each level

Registers are the fastest storage, private to each thread. They're plentiful in aggregate (about 256 KB per SM on the H100, totaling roughly 33 MB across all SMs) but each thread

running on a SM only sees a small portion. Data in registers is available essentially every clock cycle.

Shared memory and L1 cache (SRAM) sit on-chip, shared among threads within an SM. On the H100, each SM has 256 KB of combined shared memory and L1 cache, giving about 33 MB total across the chip. SRAM bandwidth is extremely high — on the order of tens of TB/s — but the capacity is small. This is the memory level that custom CUDA kernels like FlashAttention (Dao et al. 2022) exploit. By carefully tiling computations to fit in SRAM, they avoid repeated trips to the much slower main memory.

L2 Cache is a hardware cache that is not programmable. There is 50 MB of L2 cache total on the H100.

High Bandwidth Memory (HBM) is the GPU's main memory. This is the large off-chip memory where model weights, the KV cache, activations, and anything else that's not transient live. The H100 SXM has 80 GB of HBM3 with a peak bandwidth of approximately 3.35 TB/s. That sounds fast, and it is in absolute terms, but relative to the compute throughput, it's the bottleneck for most inference workloads. Every byte of model weights that needs to reach a tensor core has to be loaded via HBM's more limited bandwidth first.

The key insight about the GPU memory architecture is the enormous gap between what the compute units can consume and what HBM can deliver. The tensor cores can perform nearly 990 TFLOPS at FP16 (2 bytes per operand), but the HBM can only deliver 3.35 TB/s. We'll demonstrate how to compare these numbers in just a moment, when we discuss the roofline model.

CPU-GPU interaction and kernel launch overhead

There's one more piece of hardware context that matters for inference: the CPU-GPU relationship. The GPU is told what to do by the CPU. The CPU orchestrates inference by launching **kernels** — GPU functions that execute on the SMs. Each kernel launch involves the CPU sending a command to the GPU, which has a small but non-zero overhead, typically on the order of 5-10 microseconds.

For a single kernel launch, this overhead is negligible. But a single transformer layer can require multiple kernel launches (matrix multiplications, layer norm, activation functions, attention), and a large model may have 80 or more layers. During decode, where each step generates just one token, the actual computation per kernel can be very small. When you're launching hundreds of short kernels per decode step, the cumulative launch overhead starts to matter.

CUDA graphs address this problem. A CUDA graph captures a fixed sequence of kernel launches into a single object that can be replayed with a single launch command. Instead of the CPU issuing multiple individual kernel launches, it replays one graph. This amortizes the launch overhead and can noticeably reduce latency for decode steps. We'll revisit CUDA graphs in more detail in Chapter 6, where we'll also discuss their limitations — primarily that the captured graph must be static, which creates complications when sequence lengths or batch sizes change dynamically.

The bottleneck framework

With the hardware picture in place, we can now talk about bottlenecks precisely. GPU-based workloads have three potential bottleneck sources:

1. **Compute**: the GPU's arithmetic units can't perform operations as fast as the rest of the system.
2. **Memory bandwidth**: the GPU can't move data from HBM to the compute units as fast as the rest of the system.
3. **Inter-device communication**: data can't move between GPUs (or between CPU and GPU) as fast as the rest of the system.

For single-GPU inference, compute and memory bandwidth are the two relevant bottlenecks. Communication becomes critical when you distribute a model across multiple GPUs, which we'll discuss in Chapter 7. For now, we'll focus on the first two bottlenecks.

The roofline model

The **roofline model**, introduced by Williams et al. (2009), gives us a simple visual framework for understanding which bottleneck applies to a given workload. The core idea is that every compute workload has a property called **arithmetic intensity**, which is the ratio of compute operations to bytes of data moved, measured in FLOPs per byte (FLOP/byte). Arithmetic intensity tells you how much work the computational unit does for each byte being read from memory.

The roofline model says that a kernel's achievable performance is capped by whichever ceiling it hits first:

- If arithmetic intensity is low (the kernel reads a lot of data but does relatively little math per byte), the kernel is **memory-bandwidth-bound**. Performance is limited by how fast data can be read from HBM, regardless of how much compute is available.
- If arithmetic intensity is high (the kernel does a lot of math per byte read), the kernel is **compute-bound**. Performance is limited by the GPU's peak FLOP/s, regardless of how much bandwidth is available.

The crossover point – where the compute ceiling and the bandwidth ceiling intersect – is called the **ridge point**. For a given GPU, the ridge point is simply the ratio of peak compute throughput to peak memory bandwidth.

Table 3.2: Key hardware parameters for the NVIDIA H100 SXM.

Parameter	Value
Peak FP16 throughput	989.4 TFLOPS
HBM3 bandwidth	3.35 TB/s
Ridge point (FP16)	~295 FLOP/byte

Table 3.2 summarizes the key hardware parameters for the H100 SXM that we need for the roofline model. For the H100 SXM at FP16, we calculate:

$$\text{Ridge point} = \frac{\text{Peak FLOP/s}}{\text{Peak bandwidth}} = \frac{989.4 \times 10^{12}}{3.35 \times 10^{12}} \approx 295 \text{ FLOP/byte}$$

This means that any kernel with an arithmetic intensity below about 295 FLOP/byte is memory-bandwidth-bound on the H100 at FP16 precision. Any kernel above that threshold is compute-bound. The regions we've discussed are shown on the roofline plot for the H100 SXM in Figure 3.6. On the left, the attainable FLOPS equals the bandwidth times the arithmetic intensity, so the slope of the blue line is 3.35×10^{12}. On the right, the attainable FLOPS is capped at the peak of 989.4×10^{12}.

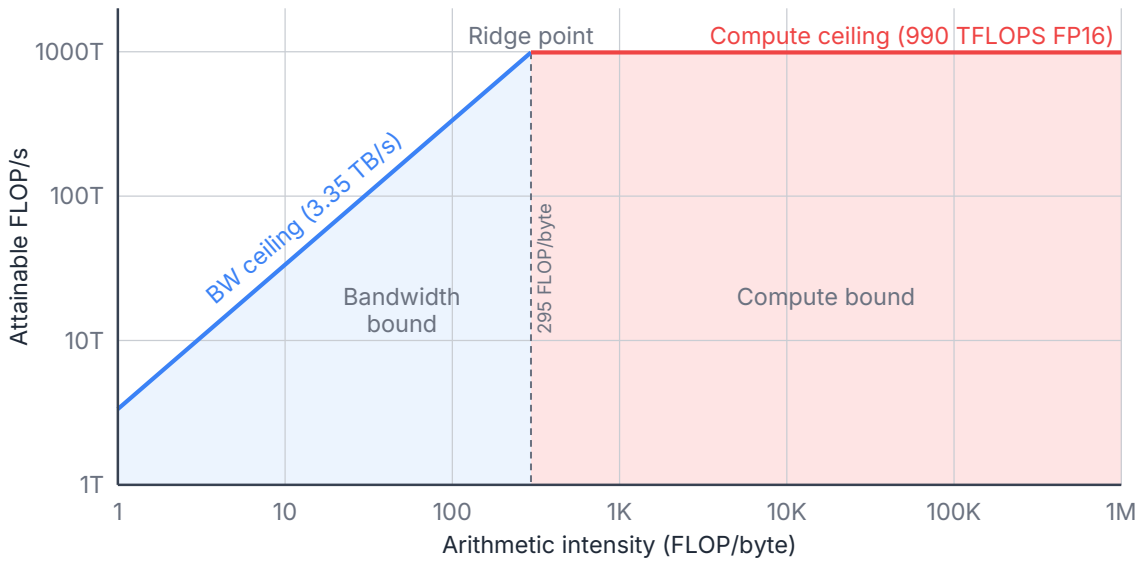


Figure 3.6: Roofline plot for the H100 SXM at FP16

If we were calculating the roofline plot for FP8, the peak bandwidth remains the same, but the peak throughput is about double at 1,979 TFLOPS. The blue line would be extended further up to the right before it hit our 1,979 TFLOPS compute ceiling, so our ridge point would be further right, at 591 FLOP/byte. When you increase compute throughput through lower precision, the bar for being compute-bound gets higher. This means that switching to FP8 can actually make some workloads that were compute-bound at FP16 become memory-bandwidth-bound at FP8. For this reason, workloads don't always run twice as fast when switching from FP16 to FP8. We will revisit this in Section 4.1 when we discuss model quantization.

3.5 Putting Numbers on a Baseline

With the roofline model in hand, let's look at real numbers for prefill and decode. We'll use the NVIDIA H100 SXM as our reference hardware and the 70 billion parameter Llama 3 model as our reference model. The goal is to establish a **baseline** for inference without any optimizations. We will start by looking at the model parameters and the computations with them.

Prefill is compute-bound

During prefill, the model processes **S** input tokens in one forward pass. A large, fixed cost is the matrix multiplications in the attention projections and MLP layers. A widely used approximation for the total FLOPs of a forward pass is:

$$\text{FLOPS}_{\text{prefill}} \approx 2 \times P \times S$$

where P is the number of model parameters and **S** is the number of input tokens. The factor of 2 comes from the fact that each parameter participates in one multiply and one add (a multiply-accumulate operation counts as 2 FLOPs). This approximation covers the linear layers and is reasonably accurate for large models where attention's quadratic component is small relative to the total.

> **Note**
>
> This $2 \times P \times S$ approximation counts the FLOPs for the linear projections (QKV projections, output projection, and MLP layers). It omits the quadratic attention score computation ($O(S^2 \times d_K \times H \times L)$) and smaller operations like layer norms and activations. For very long sequences, the S^2 in the attention cost becomes significant, and adds even more to the arithmetic intensity.

For our 70B model with $S = 4{,}096$ input tokens:

$$\text{FLOPs}_{\text{prefill}} \approx 2 \times 70 \times 10^9 \times 4{,}096 = 5.73 \times 10^{14} \text{ FLOPs} \approx 573 \text{ TFLOP}$$

Now let's compute the arithmetic intensity. The data that must be read from HBM during prefill is primarily the model weights (the input activations for the first layer come from the embedding lookup and are small by comparison). In FP16, the weights occupy:

$$\text{Weight bytes} = 70 \times 10^9 \times 2 = 140 \text{ GB}$$

The arithmetic intensity is:

$$\text{AI}_{\text{prefill}} = \frac{573 \times 10^{12}}{140 \times 10^9} \approx 4{,}093 \text{ FLOP/byte}$$

Recall from Section 3.4 that the H100 SXM ridge point at FP16 is approximately 295 FLOP/byte. Our prefill arithmetic intensity of ~4,093 FLOP/byte is more than **13x above the ridge point**. Prefill is firmly in compute-bound territory.

What does this mean for execution time? Since we're compute-bound, the time is limited by the GPU's peak compute throughput:

$$t_{\text{prefill}} \geq \frac{573 \times 10^{12}}{989.5 \times 10^{12}} \approx 0.58 \text{ seconds}$$

In practice, you won't achieve 100% of peak throughput due to overhead, memory access patterns that aren't perfectly optimized, and the attention computation. A realistic MFU of 50-70% would put the actual prefill time at roughly 0.8-1.2 seconds for this workload.

Notice what happens as the prompt gets shorter. With $S = 256$ tokens:

$$\text{AI}_{\text{prefill}} = \frac{2 \times 70 \times 10^9 \times 256}{140 \times 10^9} = 256 \text{ FLOP/byte}$$

That's just below the H100's ridge point of 295. Very short prompts could push prefill toward the memory-bandwidth-bound side, though in practice long system prompts tend to prevent requests from ever being this short.

Decode is memory-bandwidth-bound

Now consider a single decode step with batch size 1. The model runs a forward pass for just one new token. The approximate FLOPs are:

$$\text{FLOPs}_{\text{decode}} \approx 2 \times P \times 1 = 2 \times 70 \times 10^9 = 140 \times 10^9 \text{ FLOPs} = 140 \text{ GFLOP}$$

But the data movement is nearly the same as prefill — the GPU still has to read all the model weights from HBM:

$$\text{Weight bytes} = 140 \text{ GB}$$

The arithmetic intensity calculated using only the weight reads is:

$$\text{AI}_{\text{decode}} = \frac{140 \times 10^9}{140 \times 10^9} = 1 \text{ FLOP/byte}$$

One FLOP per byte. The H100's ridge point is 295 FLOP/byte. We are **295x below the ridge point**. Decode is deeply, profoundly memory-bandwidth-bound.

This is the central quantitative insight of LLM inference. During decode, the GPU reads 140 GB of weight data to perform just 140 GFLOP of computation. The tensor cores are capable

of nearly 990 TFLOP/s, but they're starved for data. The bottleneck in this scenario is how fast the GPU can stream weights from HBM.

The minimum time for a single decode step is bounded by the time to read the weights:

$$t_{\text{decode}} \geq \frac{140 \text{ GB}}{3.35 \text{ TB/s}} \approx 41.8 \text{ ms}$$

At one decode step per output token, that's a theoretical maximum of about 24 tokens per second for a single request. In practice, accounting for KV cache reads and other overhead, you might see 15-20 tokens per second for a batch-1, 70B FP16 model on an H100 — comfortably in the range where text streaming feels smooth, but not exactly fast.

> **Note**
>
> This is why quantization has such a dramatic effect on decode speed. If you quantize the weights from FP16 (2 bytes) to INT4 (0.5 bytes), the weight data drops from 140 GB to 35 GB. The minimum decode step time drops to $35/3{,}350 \approx 10.4$ ms, roughly 4x faster. The arithmetic intensity goes from 1 to 4 FLOP/byte — still far below the INT4 ridge point, so you're still bandwidth-bound, but you've reduced by 4x the bandwidth needed. So, not only does quantization help fit models into GPU memory, it also speeds up inference. We'll cover model quantization in detail in Section 4.1.

The KV cache adds to reads and writes

The analysis above only accounts for reading the model weights. But during decode, the attention computation also reads the full KV cache — all the keys and values from all previous tokens, across all layers.

For our 70B model with 80 layers, 8 KV heads, head dimension 128, and FP16 entries, the KV cache after processing a total of **S** tokens (input + output so far) is:

$$\text{KV cache size} = 2 \times 80 \times 8 \times 128 \times S \times 2 = 327{,}680 \times S \text{ bytes}$$

After a 4,096-token prefill and 512 tokens of decode ($S = 4{,}608$):

$$\text{KV cache} \approx 1.51 \text{ GB}$$

This adds to the data that must be read from HBM at every decode step. The total bytes read per decode step becomes:

$$\text{Total bytes} = \underbrace{140 \text{ GB}}_{\text{weights}} + \underbrace{1.51 \text{ GB}}_{\text{KV cache}} \approx 141.5 \text{ GB}$$

For a 70B model, this KV cache is a relatively small addition – about 1% of the total data read. But consider a smaller model. For a longer sequence, such as $S = 100{,}000$, the KV cache would be ~33 GB. At these lengths, KV cache bandwidth becomes a significant factor in decode time.

The KV cache also grows by one entry per layer per KV head per decode step. Each step must write:

$$\Delta\text{KV cache} = 2 \times L \times H_{\text{KV}} \times d_K \times b_{\text{KV}}$$

For our 70B model: $2 \times 80 \times 8 \times 128 \times 2 = 327{,}680$ bytes ≈ 320 KB per decode step. This is a small amount of data to write, but the cumulative effect is what matters: the KV cache grows monotonically, and every subsequent decode step has to read the entire thing.

Batching changes the picture

So far, we've analyzed decode with batch size 1 — a single request in isolation. This is the worst case for GPU utilization. The key insight is that **batching multiple requests together improves arithmetic intensity** by amortizing the weight reads across more tokens.

When you batch B requests together, each decode step processes B tokens instead of 1. The model weights are read once and applied to all B tokens. The FLOPs scale linearly with B:

$$\text{FLOPs}_{\text{batch}} \approx 2 \times P \times B$$

But the weight data read stays constant at $P \times b_w$ bytes (where b_w is bytes per weight). The arithmetic intensity for the weight multiplications becomes:

$$\text{AI}_{\text{batch}} = \frac{2 \times P \times B}{P \times b} = \frac{2B}{b}$$

For FP16 weights ($b = 2$), we end up with $\text{AI}_{\text{batch}} = B$ FLOP/byte.

Table 3.3: Arithmetic intensity of decode steps as a function of batch size for FP16 weights on H100 SXM. The ridge point is ~295 FLOP/byte.

Batch size	Arithmetic intensity (FLOP/byte)	Regime on H100
1	1	Bandwidth-bound
8	8	Bandwidth-bound
32	32	Bandwidth-bound
128	128	Bandwidth-bound
295	295	Ridge point
512	512	Compute-bound

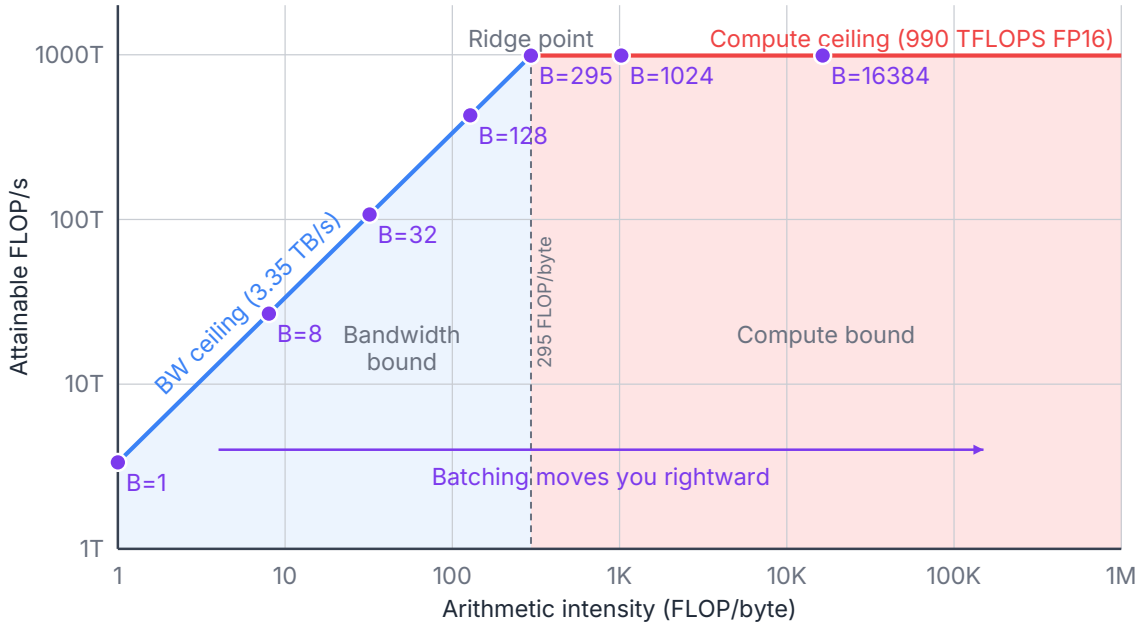


Figure 3.7: Roofline plot showing decode operating points at different batch sizes. Each point sits on the roofline at arithmetic intensity equal to the batch size (for FP16 weights). Batching moves the operating point rightward, from deeply bandwidth-bound toward the compute ceiling.

This is the fundamental reason batching helps decode throughput. With batch size 1, you're reading 140 GB to do around 140 GFLOP of work. With batch size 128, you're reading the same 140 GB but doing 18 TFLOP of work. You're moving rightward on the roofline, getting more useful computation per byte read.

The throughput gain is substantial. At batch size 1, the decode throughput is bounded by bandwidth at about 24 tokens/s. At batch size 128, you're still bandwidth-bound but now generating 128 tokens per step, so the throughput is approximately $128 \times 24 \approx 3{,}072$ tokens/s across all requests. Each individual request still sees about 24 tokens/s (the step time is the same because we're reading the same weight data), but the system throughput has increased 128x.

> **Note**
>
> This first order analysis ignores the KV cache reads, which do scale with batch size, since each request has its own KV cache. At large batch sizes with long sequences, the KV cache bandwidth can become a significant additional cost. It also ignores the memory capacity constraint: each active request's KV cache occupies HBM, so the maximum batch size is limited by how many KV caches fit in memory alongside the model weights.

To push the arithmetic intensity above the ridge point and become compute-bound, you'd need a batch size of about 295 for FP16 weights on the H100. In practice, memory capacity limits usually prevent batch sizes this large for big models. A 70B model with 140 GB of weights leaves only ~40 GB for KV caches on an 80 GB GPU. If each request's KV cache is 1.5 GB, you can fit about 26 concurrent requests, far short of the 295 needed to hit the ridge point. This is why decode for large models remains memory-bandwidth-bound even with batching, and why the optimizations for reducing KV cache memory usage in Section 6.3 and elsewhere are so important — they enable larger batch sizes, which improves throughput.

Back-of-envelope summary

Let's collect the key numbers for our reference configuration: a 70B parameter model in FP16 on an H100 SXM (80 GB HBM, 3.35 TB/s bandwidth, 989.5 TFLOP/s FP16).

Table 3.4: Back-of-envelope performance for a 70B FP16 model on H100 SXM. Prefill throughput is computed as S/t_{prefill}. Decode throughput is B/t_{decode}. These are theoretical lower bounds on step time; real-world performance will be somewhat lower.

Quantity	Prefill (S=4096)	Decode (B=1)	Decode (B=32)
FLOPs per step	573 TFLOP	140 GFLOP	4.48 TFLOP
Data read (weights)	140 GB	140 GB	140 GB
Arithmetic intensity	~4,093 FLOP/byte	~1 FLOP/byte	~32 FLOP/byte
Bottleneck	Compute	Bandwidth	Bandwidth
Min step time	~0.58 s (compute)	~41.8 ms (bandwidth)	~41.8 ms (bandwidth)

Quantity	Prefill (S=4096)	Decode (B=1)	Decode (B=32)
Tokens per step	4,096 (all input)	1	32
Throughput	~7,000 tok/s	~24 tok/s	~766 tok/s

A few things stand out from this table:

1. **Prefill processes tokens roughly 300x faster than batch-1 decode**, in terms of tokens per second. This is the direct consequence of the arithmetic intensity gap.
2. **Batching is the single most important lever for decode throughput.** Going from batch size 1 to 32 increases system throughput by 32x with essentially no increase in step time, because we're bandwidth-bound either way. The same weight data is read, just applied to more tokens. Unfortunately, total memory capacity limits how large we can push batch size.
3. **The step time for decode barely changes with batch size** (as long as we remain bandwidth-bound). This means that batching increases system throughput without significantly increasing per-request latency. This is a rare case where we can improve one metric, throughput, without a tradeoff in another metric, latency.
4. **Weight memory dominates the data read for large models.** The KV cache is a relatively small fraction of the total data read per step, though it becomes more significant for smaller models and very long sequences.

These numbers are the baseline that every optimization in subsequent chapters maps onto. When we discuss model quantization in Section 4.1, we'll see how reducing bytes per weight directly reduces the bandwidth denominator. When we discuss batching strategies in Section 5.1, we'll see how continuous batching and chunked prefill allow the system to maintain high batch sizes. When we discuss KV cache optimizations in Section 6.3, we'll see how reducing KV cache memory enables larger batches. And when we discuss speculative decoding in Section 6.5, we'll see how generating multiple tokens per decode step changes the numerator.

3.6 Map of the optimizations

The next four chapters in this book each focus on a specific class of bottleneck and the techniques that address it:

Chapter	Bottleneck class	Core lever
Chapter 4 – Model Design	Model size and compute	Reduce cost per token via decreasing model size
Chapter 5 – Scheduling and Batching	Scheduling and queuing	Reduce wasted GPU resources through smarter batching and memory management

Chapter	Bottleneck class	Core lever
Chapter 6 – Request-Level Optimizations	Request compute and memory; sequential decoding	Optimize individual requests; break the sequential decoding constraint
Chapter 7 – Multi-Device Inference	Hardware capacity	Distribute across multiple devices when one GPU isn't enough

This bottleneck framework gives you a way to evaluate each optimization you encounter: what bottleneck does it address, and does that bottleneck actually apply to your workload? An optimization that reduces compute won't help a memory-bandwidth-bound decode step much. A technique that reduces memory traffic won't help a compute-bound prefill. Keeping the roofline in mind as you read the rest of this book will help you understand not just *what* each technique does, but *why* it works and *when* it helps.

The book concludes in Chapter 8 with a discussion of production LLM serving systems.

3.7 Further Reading

The Anyscale blog post on reproducible performance metrics (Kadous et al. 2023) is a practical guide to LLM inference metrics, including many of the ones introduced in this chapter, such as TTFT, TPOT, throughput, and goodput.

The NVIDIA Hopper architecture blog post (NVIDIA 2022) provides detailed block diagrams of the H100's SM and full-chip layout, including the memory hierarchy, tensor core organization, and the Tensor Memory Accelerator. Gu and Dao (2024) from Hazy Research gives a practitioner-oriented walkthrough of GPU memory levels with concrete bandwidth numbers at each tier, which is useful context for understanding why kernel design choices matter so much for inference performance. For a detailed empirical look at the H100's L2 cache behavior and measured bandwidth at different access patterns, Chips and Cheese (2023) is an informative deep dive.

For GPU hardware specifics, the NVIDIA H100 datasheet (NVIDIA 2023b) and the CUDA Programming Guide (NVIDIA 2024a) are the authoritative references. The datasheet gives the peak throughput and bandwidth numbers used throughout this book. The programming guide covers the memory hierarchy, kernel launch mechanics, and CUDA graphs in much more detail than we do here.

The back-of-envelope analysis in this chapter follows the approach laid out by Pope et al. (2022), who provide a thorough accounting of FLOPs, memory bandwidth, and arithmetic intensity for each layer of a transformer during inference. Their paper covers both prefill and decode on TPU hardware, but the methodology translates directly to GPUs. They also cover the effects of multi-query attention, different parallelism strategies, and how the roofline picture changes as you scale model size.

For an accessible walkthrough of counting parameters and FLOPs in transformer models, Kipply's "Transformer Inference Arithmetic" (Chen 2022) is an excellent companion to this chapter. It works through the same kinds of calculations we've done here – parameter counts, memory footprints, FLOPs per layer – with clear explanations of where each number comes from.

For a broader survey of LLM inference techniques and how they fit together, Miao et al. (2023) covers the full landscape from model compression through serving system design. It's a useful map of the territory before diving into the detailed treatments in the chapters that follow.

The roofline model was introduced by Williams et al. (2009) and remains one of the clearest frameworks for reasoning about hardware bottlenecks. Their original paper uses CPU examples, but the model applies directly to GPUs – the key insight is the same: compare arithmetic intensity against the ridge point to determine whether you're compute-bound or bandwidth-bound.

A comprehensive, in-depth treatment of hardware performance is in "How to Scale Your Model" (Austin et al. 2025). This book primarily focuses on TPUs, with one specific chapter on GPUs. All of the concepts apply to whatever hardware you have.

4 Model Design Choices

Now that we have an understanding of the inference pipeline and where the bottlenecks live, starting with this chapter, we shift to doing something about them.

The optimizations in this chapter are all about the model itself, separate from request handling. They change what the hardware has to do per token by reducing the model's compute and memory footprint at the architecture and weight level. Some of these are choices made by model designers prior to training. Others are applied post-training to an existing model. Either way, the result is the same: fewer bytes to move, fewer FLOPs to compute, or both.

We'll keep the coverage here lighter than in subsequent chapters. Much of the time, you don't get a choice about the model to be used. Because of this, much of the focus of this book is on what you can do when you're given a model to serve. But understanding these techniques is still important, whether you have any say in the model or not.

4.1 Model Quantization

One can make a strong argument that **quantization** is the single most impactful model-level optimization for LLM inference. The reason is straightforward, and it connects directly to the bottleneck framework from Section 3.4.

During decode, the bottleneck is memory bandwidth. As we've discussed, each decode step reads the entire model's weights from HBM just to produce one token. If your model has 70 billion parameters stored in FP16 (2 bytes each), that's 140 GB of weight data read per token. On an H100 with 3.35 TB/s of bandwidth, reading 140 GB takes about 42 milliseconds – and that's a hard floor on your per-token latency, ignoring everything else. If you quantize those same weights to INT4 (0.5 bytes each), you're reading 35 GB instead, and the floor drops to about 10 milliseconds. That's a 4x improvement in your theoretical best-case decode speed, and you didn't change a single line of serving code.

The key insight is that quantization doesn't just reduce the amount of GPU memory needed to store the model weights (which is a pretty big benefit). Quantization also reduces the bandwidth demand that dominates decode latency, and it can speed up computation too. Let's walk through the main approaches to quantization. We will assume 16-bit weights in FP16 or BF16 are our baseline.

Lower precision datatypes

There are a number of datatypes supported, especially in the latest generation of GPUs. While higher precision datatypes, such as FP32, are needed for training, most models can run inference using FP16 without any noticeable difference. Datatypes that require fewer than 16 bits to store can save even more memory. Table 4.1 lists commonly supported datatypes.

Table 4.1: Common datatype precisions supported. The Layout column shows the number of bits allocated for the sign ("s"), exponent ("e"), and mantissa ("m").

Type	Bits	Layout	Key Properties
FP32	32	1s + 8e + 23m	Standard training precision; baseline
FP16	16	1s + 5e + 10m	2x compression; narrower dynamic range (max ~65,504)
BF16	16	1s + 8e + 7m	Same dynamic range as FP32, less mantissa precision; preferred for training stability
FP8 E4M3	8	1s + 4e + 3m	More precision; used for weights/activations in forward pass
FP8 E5M2	8	1s + 5e + 2m	More dynamic range; used for gradients
INT8	8	signed integer	Range [-128, 127]; 4x compression vs FP32
INT4	4	signed integer	Range [-8, 7]; 8x compression; requires sophisticated algorithms
NF4	4	non-uniform	16 levels at normal distribution quantiles; information-theoretically optimal for normally-distributed weights (QLoRA)
FP4	4	1s + 2e + 1m	Less common than NF4 for LLM weights

Older GPUs support 32-bit and 16-bit floating point, as well as 8-bit integer matrix multiplication. FP8 is a newer format supported on H100 and later GPUs. With the popularity of quantization, newer generations of GPUs are supporting more smaller datatypes.

Weight-only quantization

The simplest form of quantization leaves the activations in their original precision (typically FP16 or BF16) and only quantizes the weights. This is called **weight-only quantization**, and it's the most common starting point. In weight-only quantization, the weights are stored in a smaller datatype, then dequantized back to FP16 on the fly before each matrix multiplication, so the compute still happens at FP16 precision. The benefit is purely about reducing how much data needs to be read from HBM.

When quantizing below 16-bit precision, the storage space and memory read savings are usually slightly reduced because we also need to store conversion parameters. If we have both an offset and a scaling parameter, then when x is the higher precision value and $\tilde{x}$ is the quantized value, the conversion is $x = (\tilde{x} - \text{offset}) * \text{scaling}$. This dequantization step also adds a small amount of compute overhead to model inference.

LLMs tolerate quantization well. While the lower precision datatypes could cause cascading rounding errors, in practice the errors tend to cancel out. Quantizing from FP16 to INT8, known as **W8**, cuts the weight memory in half compared to FP16. The quality impact is usually negligible for large models — a 70B model at W8 will behave almost identically to its FP16 counterpart on most benchmarks. If we drop to **W4** quantization, cutting weight memory by 4x compared to FP16, this is where quality tradeoffs start to become more visible, especially for smaller models. A 7B model at W4 may show degradation on some tasks, while a 70B model at W4 often remains remarkably capable. The general pattern is that larger models are more robust to aggressive quantization.

Weight-activation quantization

Weight-only quantization reduces memory bandwidth demand but doesn't speed up the actual matrix multiplication – the compute still runs at FP16. **Weight-activation quantization** takes the next step: quantize both weights *and* activations so that the matrix multiplications themselves can run on lower-precision hardware units.

For example, weight-activation quantization using INT8, known as **W8A8**, quantizes both weights and activations to INT8, allowing the use of INT8 tensor cores. On the H100, INT8 tensor cores deliver roughly 2x the throughput of FP16 tensor cores. This means W8A8 doesn't just reduce memory traffic – it also increases the compute ceiling. As with weight-only quantization, there is some additional storage and compute overhead for the conversion parameters.

Granularity

Early attempts to quantize below 16-bit floating point took a simple approach that used the same datatype and conversion factor for all values in each weight matrix. In this scenario, one very large value distorts the conversion scale. This increases the rounding errors caused by quantization and can also cause small weights to get rounded to zero, which can cause

even more problems. To mitigate the risk of outlier values, quantization can be performed at more fine-grained granularities:

- Per channel(s)
- Per block, potentially in a hierarchy of super-blocks and sub-blocks
- Per token

Fine-grained quantization seeks to group similar values together, and is highly effective. It does, however, come at a cost of additional memory and compute overhead. For example, dividing a 2,048 by 2,048 weight matrix into groups of 32 values will require 131,072 scaling parameters and dequantization calculations. To reduce the storage of the scaling factors, they can themselves be quantized at a cost of additional dequantization overhead.

When quantization is applied

Post-Training Quantization (PTQ) is performed after the model is fully trained. This is the dominant approach since it can be applied to any model from any source, without requiring any special training. Simple PTQ only examines the weights, and more advanced techniques require processing a small set of calibration data through the model so activation information can also be analyzed. **Quantization-Aware Training (QAT)** modifies the model training procedure to minimize the rounding errors caused by quantization. While effective, the extra work required has limited adoption.

Techniques

One of the practical challenges of quantization – especially weight-activation quantization – is **activation outliers**. Large LLMs tend to develop a small number of activation channels with values that are orders of magnitude larger than the rest. If you try to quantize activations using a single scale factor per tensor, these outliers force a wide quantization range that wastes precision on the many small values. Here are a few post-training quantization methods and how they try to avoid activation outliers.

GPTQ (Frantar et al. 2022) uses approximate second-order information to quantize weights to 4 bits (or even 3 bits) with minimal accuracy loss. It works by quantizing weights one layer at a time and adjusting the remaining weights to compensate for the error introduced by each quantization step. GPTQ can be applied to a pre-trained model without any additional training data – just a small calibration set.

AWQ (Activation-aware Weight Quantization) (Lin et al. 2023) takes a different approach. It observes that a small fraction of weights are disproportionately important – specifically, the weights that correspond to large activation magnitudes. Rather than treating all weights equally, AWQ protects these salient weights by scaling them up before quantization and scaling the activations down correspondingly. This keeps the important weights at higher effective precision while allowing aggressive quantization elsewhere.

SmoothQuant (Xiao et al. 2023) addresses activation outliers by migrating the quantization difficulty from activations to weights. The idea is to apply a per-channel scaling transformation that divides the activation by a smoothing factor and multiplies the corresponding weight by the same factor. This makes the activation distribution more uniform – and therefore easier to quantize – at the cost of making the weight distribution slightly less uniform. Since weights are fixed and can be quantized offline with more care, this is a favorable tradeoff.

Connecting quantization to MBU

In Section 3.2, we introduced Model Bandwidth Utilization (MBU) as a key diagnostic for decode performance. Quantization improves the *theoretical* decode speed by reducing the bytes that need to be read per token. But it doesn't automatically improve MBU — if your serving system has overhead that prevents it from saturating HBM bandwidth, quantization reduces the numerator (bytes of model data) but the denominator (peak bandwidth) stays the same, so the time improvement is proportional to the data reduction only if MBU remains constant.

In practice, quantized models often achieve *higher* MBU because the smaller data footprint means fewer cache misses, better memory access patterns, and the possibility of fitting the entire model in a single GPU's memory rather than splitting across devices. These secondary effects can make the real-world speedup even better than the raw byte reduction would suggest.

> **Note**
>
> Quantization affects the roofline model in an important way. When you use FP8 or INT8 tensor cores, you raise the compute ceiling (from ~990 TFLOPS to ~1,979 TFLOPS on the H100) while the bandwidth ceiling stays constant. This shifts the ridge point higher, which means some workloads that were compute-bound at FP16 can become memory-bandwidth-bound at FP8. Keep this in mind when choosing quantization strategies for prefill-heavy workloads.

4.2 Knowledge Distillation

Knowledge distillation is a training technique where a smaller **student** model is trained to mimic the behavior of a larger **teacher** model. In the original distillation formulation (Hinton et al. 2015), the student learns not just from the ground-truth labels, but from the teacher's full output distribution – the probability the teacher assigns to every token in the vocabulary, not just the top-1 prediction. These soft targets carry richer information than hard labels, which is why a distilled student can often outperform a model of the same size trained from scratch. Newer techniques vary the approach by training on sample outputs from the larger model or using reinforcement learning to impart some of the larger model's capability into the smaller model.

From an inference perspective, the result is simple: you get a smaller model. Fewer parameters means less data to read from HBM per decode step, a smaller KV cache (if the model also has fewer layers or a smaller hidden dimension), and less memory consumed overall. The inference system doesn't need to know or care that the model was distilled — it just sees a model with fewer parameters that performs better than you'd expect for its size.

The tradeoff is that distillation requires significant training compute, and you need the larger, more capable teacher model. This may not be an option for a frontier model, but distillation is popular in research and in use for smaller models.

4.3 Pruning

Pruning removes weights or structures from a model that contribute little to its output. The motivation is similar to quantization: fewer parameters means less data to move, which speeds up bandwidth-bound decode. But while quantization keeps all the parameters and reduces their precision, pruning eliminates parameters entirely.

Unstructured pruning

Unstructured pruning zeroes out individual weights based on some importance criterion (typically magnitude). A pruned model might have 50% of its weights set to zero, theoretically cutting compute and memory in half. The problem is that these zeros are scattered throughout the weight matrices in an irregular pattern. If enough of the weights can be pruned, good speedups can be achieved for CPU inference. Standard GPU hardware and matrix multiplication kernels still process the full dense matrices, so they can't exploit this sparsity efficiently, and sparse matrix multiplications are much slower than their dense counterparts, so they need extremely sparse matrices before they are faster than full dense matrix multiplication. To get real speedups from unstructured pruning, you need specialized sparse compute support, such as NVIDIA's 2:4 structured sparsity format on Ampere and later GPUs, and it is more difficult to prune weights into this pattern.

Structured pruning

Structured pruning takes a coarser approach: instead of zeroing individual weights, it removes entire structural units – attention heads, entire layers, or neurons in MLP blocks. The result is a smaller dense model that runs on standard hardware with no special sparse computation support required.

Removing an attention head reduces the KV cache proportionally. Removing an entire layer reduces both parameters and the depth of the forward pass. These are straightforward wins for inference, and the cost model from Section 3.5 applies directly — fewer parameters means proportionally less time reading weights during decode.

The challenge with structured pruning is identifying which structures can be removed with minimal quality impact. Attention heads in the later layers of a model often contribute less than early or middle layers, but this varies by model and task. Structured pruning typically requires some retraining or fine-tuning after removal to recover quality, which limits its use as a purely post-training optimization.

In practice, pruning is less widely adopted for LLM inference than quantization. Quantization offers a more predictable quality-performance tradeoff and is easier to apply post-training.

4.4 Efficient Attention Architectures

The sections above are post-training techniques you can apply to an existing model. This section covers architecture choices – decisions made during model design and training that affect inference efficiency from the start.

MLP layers account for roughly two-thirds of a typical transformer's parameters, but they operate independently on each token. Their cost scales linearly with sequence length, and they are straightforward to parallelize. Attention is the more interesting bottleneck. Its KV cache grows with sequence length, creating a per-request memory cost that directly limits concurrency. For long sequences, the quadratic cost of full attention also becomes significant during prefill. This makes attention the primary target for architectural optimization.

Reducing KV cache size

The KV cache is one of the largest consumers of GPU memory during inference – and unlike model weights, it scales with the number of active requests and the sequence length of each. Reducing the KV cache per request lets you serve more concurrent requests in the same memory budget. We'll cover KV cache management techniques in Section 6.3; here we focus on the architectural choices that determine how large the cache is in the first place.

Multi-Head Attention (MHA) is the original design from the transformer paper (Vaswani et al. 2017). Each attention head has its own key and value projections, so the KV cache stores separate K and V vectors for every head, every layer, and every token in the sequence. For a model using FP16 with 64 heads and a head dimension of 128, each token adds $2 \times 64 \times 128 \times 2 = 32{,}768$ bytes to the KV cache per layer. This is the most expensive option. The top row of Figure 4.1 shows how queries align with keys and values in MHA.

Multi-Query Attention (MQA) (Shazeer 2019) goes to the opposite extreme: all attention heads share a *single* set of key and value projections, as shown on the third row of Figure 4.1. Each head still has its own query projection, so the heads can attend to different aspects of the input, but the KV cache only stores one K and one V vector per layer per token. This reduces the KV cache by a factor equal to the number of heads – from 32 sets of K/V down to 1 in the example above. The quality impact is generally small for large models, and the inference speedup – especially for long sequences with many concurrent requests – can be substantial.

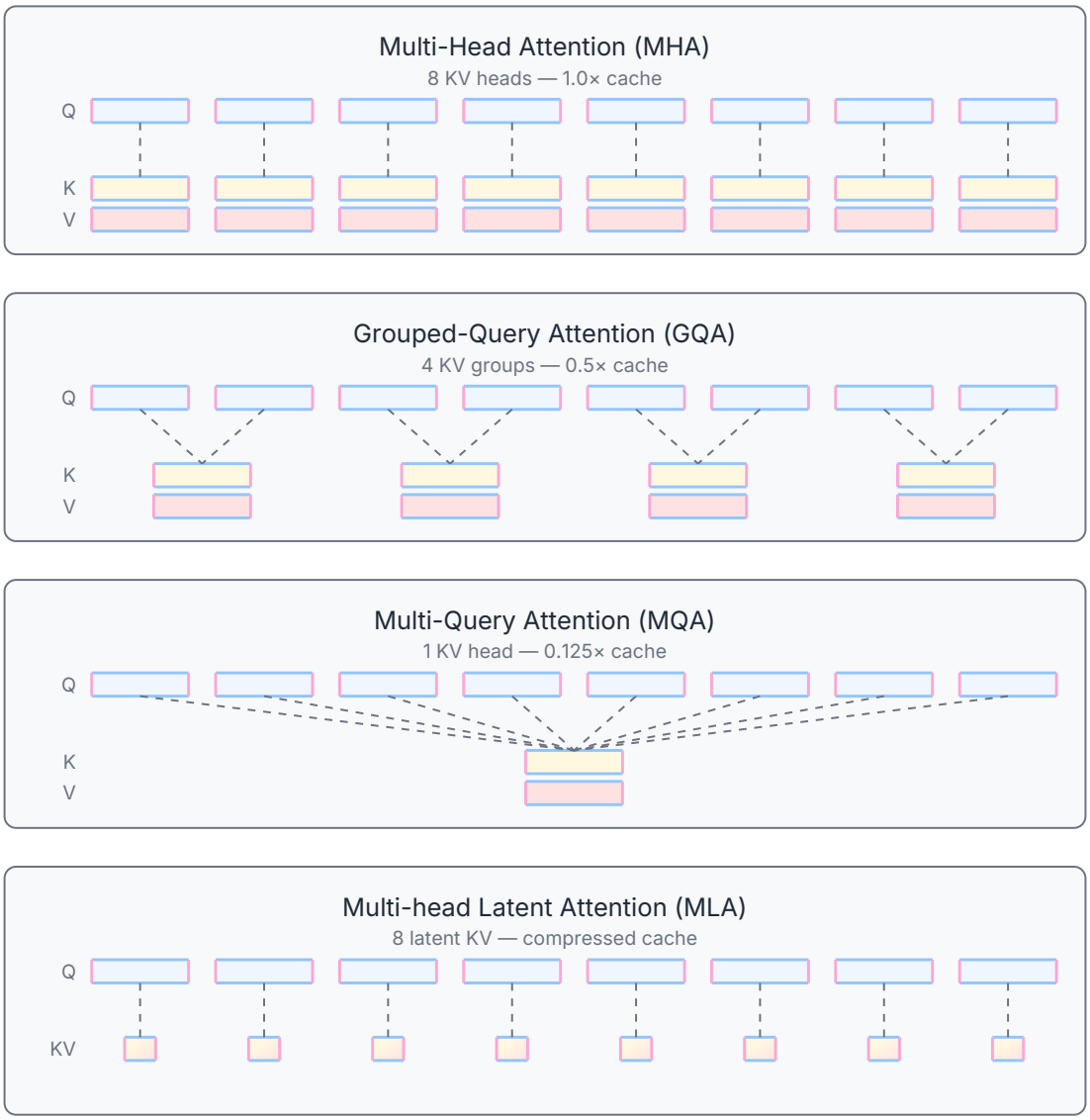


Figure 4.1: Techniques for reducing the KV cache size of a given layer. GQA and MQA share keys and values with multiple query heads. MLA compresses key and value information together to reduce size.

Grouped-Query Attention (GQA) (Ainslie et al. 2023) is a middle ground between MHA and MQA. Instead of one shared KV set for all heads, GQA uses **G** groups, where each group of heads shares one set of keys and values. With 32 heads and **G** = 8 groups, the KV cache is reduced by 4x compared to MHA. GQA has become the dominant choice in recent models – LLaMA 3, Mistral, and Gemma all use it. It offers most of MQA's memory savings with less quality risk. A GQA example with 8 query heads and 4 groups is shown on the second row of Figure 4.1.

Multi-head Latent Attention (MLA) (Liu et al. 2024) is DeepSeek's approach to KV cache compression. Rather than storing full key and value vectors, MLA compresses them into a low-rank **latent vector** at each layer. At attention time, the full keys and values are reconstructed from the latent representation. The compression ratio depends on the latent dimension, but MLA can achieve significant cache reductions – comparable to or better than GQA – while preserving model quality. The tradeoff is additional compute to decompress the latent vectors during attention, but this compute is typically small relative to the bandwidth savings, and there are tricks to reduce the compute. MLA is depicted on the bottom row of Figure 4.1.

Cross-Layer Attention (CLA) (Brandon et al. 2024) takes a different angle: rather than sharing KV across heads within a layer, it shares KV across adjacent layers. If every pair of consecutive layers shares the same KV cache, the total cache size is halved. Figure 4.2 compares standard attention with CLA, where the K and V vectors are shared between pairs of layers. CLA can be combined with GQA or MQA for even greater reductions.

Reducing attention compute

Beyond the KV cache, the attention computation itself can be a bottleneck, particularly during prefill for long sequences where the cost is quadratic in sequence length.

Sliding window attention (Beltagy et al. 2020) restricts each token to attend only to a fixed-size window of nearby tokens rather than the full sequence. This reduces the attention computation from $O(n^2)$ to $O(n \times w)$, where w is the window size. Figure 4.3 illustrates the difference between full attention and sliding window attention with a window size of 4. For inference, sliding window attention also bounds the KV cache size per layer to the window size, regardless of the total sequence length. Several recent models – including Mistral – use sliding window attention in some or all layers, often combined with a few full-attention layers to preserve the model's ability to attend to distant tokens.

Parallel attention and MLP blocks is a simple architectural trick used in some models, including PaLM (Chowdhery et al. 2022). Instead of computing attention and MLP sequentially within each transformer block, they are computed in parallel and their outputs are summed. This doesn't reduce total FLOPs, but it reduces the serial dependency within each layer, which can improve GPU utilization and reduce latency — especially when the attention and MLP computations can be overlapped on different hardware units. A schematic of this parallel design is shown in Figure 4.4.

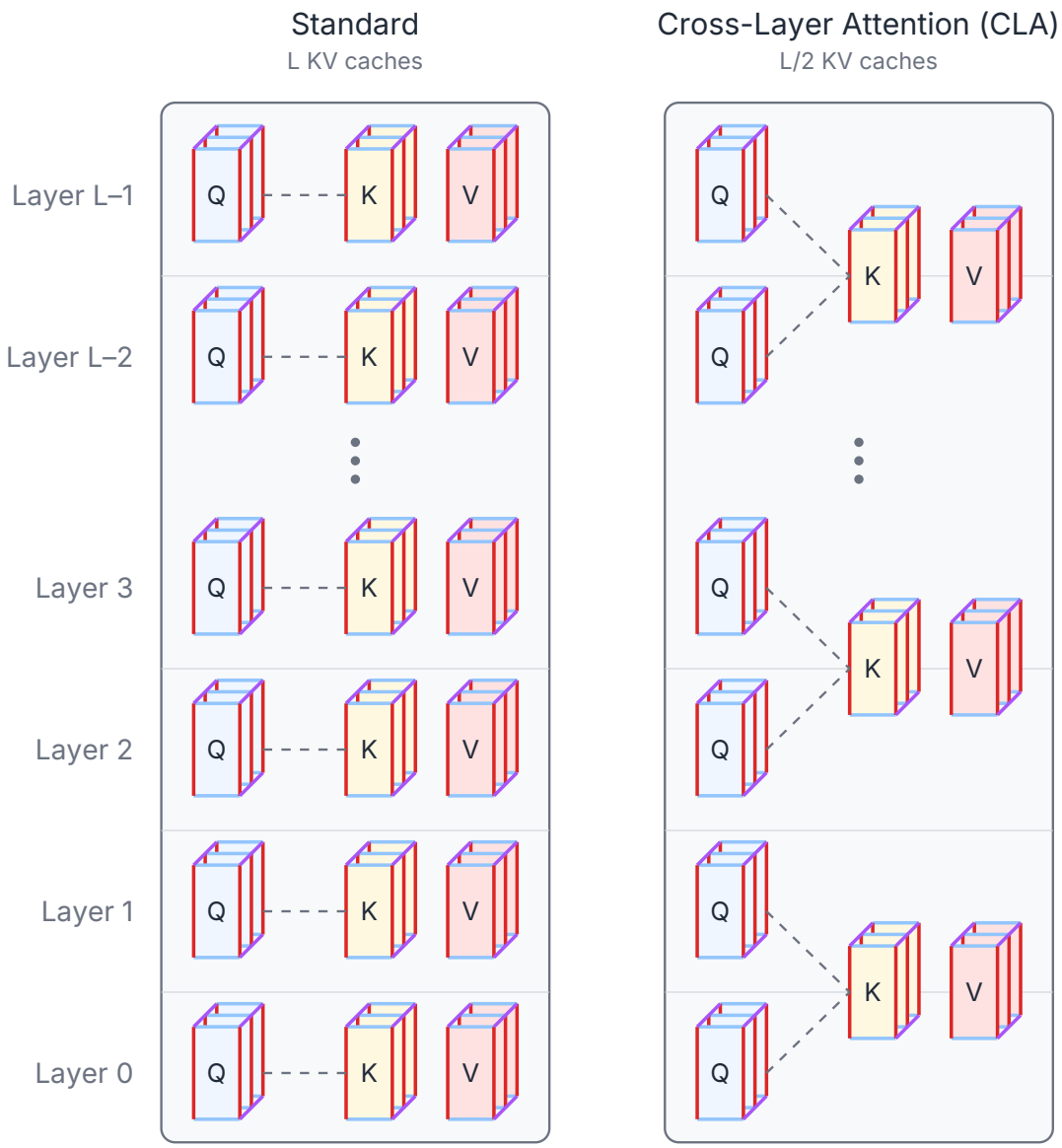


Figure 4.2: Cross-layer attention with a factor of 2 KV sharing between layers

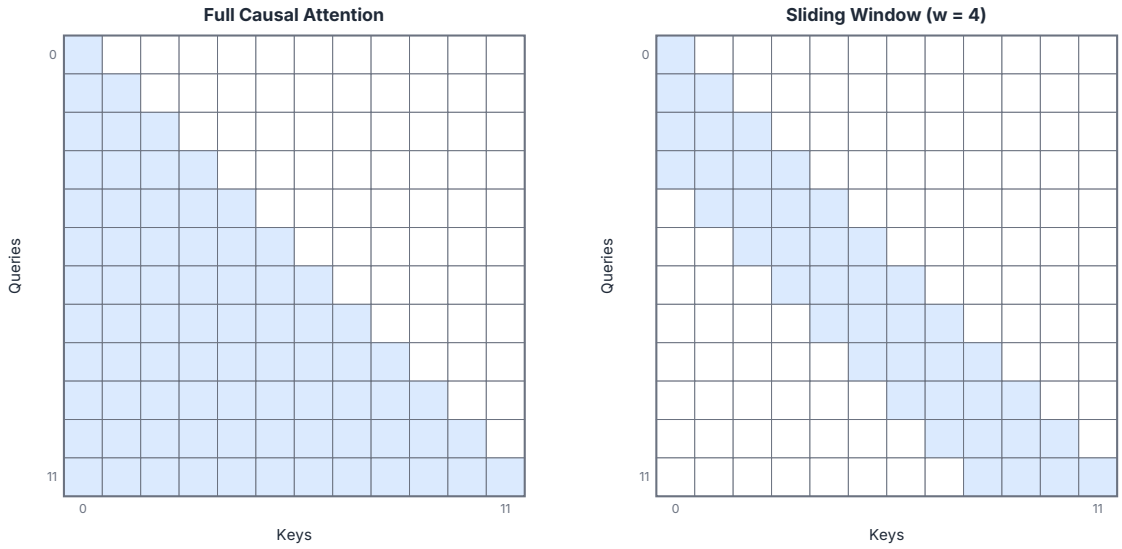


Figure 4.3: Full causal attention (left) vs. sliding window attention with window size w=4 (right). Each shaded cell indicates a query-key pair included in the attention computation.

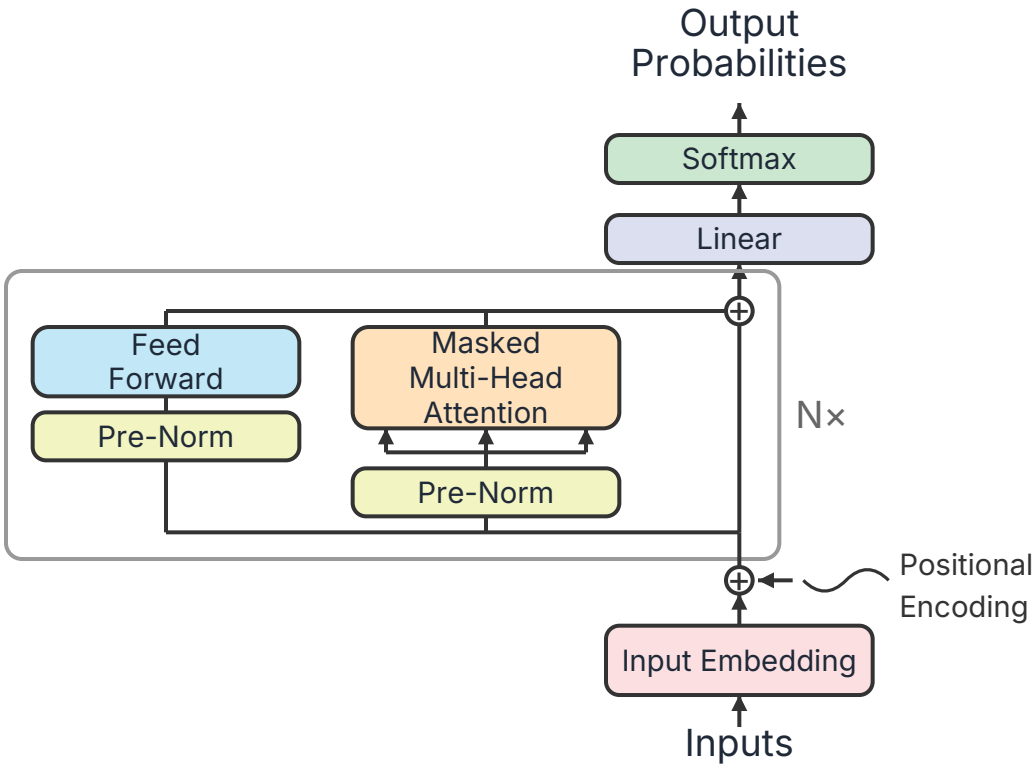


Figure 4.4: Parallel attention and MLP blocks

> **Note**
>
> This book focuses on the dominant transformer architecture with softmax attention. Alternative approaches like Mamba and other state-space models, as well as linear attention variants, replace the attention mechanism entirely with recurrent-style computations that have constant-size state rather than a growing KV cache. These are an active area of research, but the vast majority of deployed LLMs today use softmax attention, so we'll focus our attention there.

4.5 Tokenization and Vocabulary Effects

One model-level lever that's easy to overlook is the **tokenizer** and its vocabulary size. The tokenizer determines how many tokens a given piece of text is broken into, and that token count directly drives the number of decode steps – each of which requires a full forward pass through the model.

A larger vocabulary means each token covers more text on average – common words or subwords that would otherwise be split into multiple tokens get their own single token. Fewer tokens means fewer decode steps, which means faster end-to-end generation. For latency-sensitive applications, this is a meaningful win. Going from a 32K vocabulary to a 128K vocabulary might reduce the average token count for a given text by 10-20%, which translates directly to 10-20% fewer decode steps.

The tradeoff is that a larger vocabulary increases the size of two model components: the **embedding table** (which maps token IDs to vectors at the input) and the **LM head** (the output projection that produces logits over the vocabulary). For a model with a hidden dimension of 4,096 and a vocabulary of 128K tokens, the LM head alone has $4{,}096 \times 128{,}000 \approx 500$ million parameters. At FP16, that's about 1 GB – not negligible, but typically a small fraction of the total model size for large LLMs. The compute cost of the final softmax over a larger vocabulary also increases, though this is usually a minor contributor to overall latency.

For most practical scenarios, the efficiency gains from fewer decode steps outweigh the increased embedding and LM head size. This is why the trend in recent LLMs has been toward larger vocabularies.

4.6 Summary

The techniques in this chapter all reduce what the hardware has to do for each token. Quantization reduces the bytes-per-parameter, directly attacking the bandwidth bottleneck that dominates decode. Distillation and pruning produce smaller models with fewer total parameters. Efficient attention architectures reduce the KV cache footprint and, in some cases, the attention compute itself. Even vocabulary design affects inference cost by changing how many decode steps are needed.

These are the model-level knobs. Once they're set – once you have a quantized, architecturally efficient model ready to serve – the next set of optimizations operates at the system level: how to schedule requests, batch them together, and manage memory. That's where we turn next in Chapter 5.

4.7 Further Reading

Quantization is a deep field with a large body of work. The GPTQ (Frantar et al. 2022) and AWQ (Lin et al. 2023) papers are essential reading if you want to understand the mechanics of post-training weight quantization beyond the overview given here. GPTQ's use of approximate second-order information to minimize quantization error is elegant, and AWQ's insight that activation-aware scaling protects the most important weights is a useful design pattern that shows up in other contexts. SmoothQuant (Xiao et al. 2023) is the key reference for weight-activation quantization, particularly its technique of migrating quantization difficulty from activations to weights. For an earlier take on INT8 inference that pioneered mixed-precision decomposition, see Dettmers et al. (2022). The most popular quantization library is `bitsandbytes` (Dettmers et al. 2021), also by Tim Dettmers.

For a comprehensive survey of quantization methods applied to LLMs, Zhu et al. (2024) covers the full taxonomy – from uniform to non-uniform quantization, weight-only to weight-activation, and post-training to quantization-aware training – with empirical comparisons across model sizes.

On the attention architecture side, the progression from standard multi-head attention to MQA (Shazeer 2019), GQA (Ainslie et al. 2023), and MLA (Liu et al. 2024) is well documented in the original papers. Shazeer's MQA paper is particularly worth reading because it clearly explains the inference motivation – reducing KV cache bandwidth – rather than just reporting accuracy numbers. The Llama 3 paper (Grattafiori et al. 2024) provides a good case study of GQA adopted in a production-grade model, with ablations showing the quality-efficiency tradeoff.

For knowledge distillation applied to LLMs specifically, Gu et al. (2024) covers the challenges of distilling autoregressive language models, including the mismatch between training-time and inference-time token distributions that makes naive distillation less effective for generative models.

5 Scheduling Bottlenecks

The previous chapter focused on making the model itself smaller and faster – fewer parameters, lower precision, more efficient attention mechanisms. Those techniques change what gets computed. This chapter drills into how requests flow through the system. Even with a well-optimized model, a naive scheduler can leave the GPU sitting idle while requests wait in line, waste compute on padding tokens that contribute nothing, or let a single long prefill block dozens of shorter requests from making progress.

The techniques here — batching strategies, prefill management, and request scheduling — act on the serving system layer. They're about keeping the GPU busy doing useful work as much of the time as possible, regardless of what model is being used.

5.1 Batching Strategies

Why batching matters

In Section 3.4, we established that decode is deeply memory-bandwidth-bound. Each decode step reads the entire set of model weights from HBM to produce just one token per request. On an H100 SXM with a 70B parameter model in FP16, that's roughly 140 GB of weight data read through 3.35 TB/s of bandwidth – about 42 ms just to stream the weights, and all of that work produces a single token.

Batching changes this equation. When you process **B** requests in a single decode step, you still read the weights once, but you apply them to **B** token vectors instead of one. The compute scales linearly with **B**, but the data movement stays roughly constant (dominated by the weight read). This directly increases the **arithmetic intensity** – the ratio of FLOPs to bytes moved – pushing decode toward the compute-bound side of the roofline.

Let's put concrete numbers on this. For a single linear layer with weight matrix of shape **(D, D)** in FP16:

- **Data moved**: $2D^2$ bytes (reading the weight matrix)
- **Compute**: $2BD^2$ FLOPs (B matrix-vector multiplications)
- **Arithmetic intensity**: $2BD^2/2D^2 = B$ FLOP/byte

Recall Figure 3.7.

With batch size 1, the arithmetic intensity is about 1 FLOP/byte – far below the H100's FP16 ridge point of ~295 FLOP/byte. The GPU's tensor cores are almost entirely idle, waiting for data to arrive from HBM. With batch size 64, the intensity rises to 64 FLOP/byte. Still

below the ridge point, but now the GPU is doing 64x more useful work per byte read. With batch size 256, we're approaching the ridge point and the GPU's compute units are finally getting busy.

The throughput gain is dramatic. A system decoding one request at a time on an H100 might produce ~30 tokens/s. The same system decoding a batch of 64 requests might produce ~1500 tokens/s total – roughly the same latency per step, but 50x the throughput. This is why batching is the single most important lever for inference cost efficiency.

The memory cost of batching

There's a catch, and it's a big one. Each request in the batch needs its own **KV cache** – the stored key and value tensors from every layer, accumulated across every token processed so far. As we'll cover in detail in Section 6.3, the KV cache for a single request can consume several gigabytes for long sequences with large models.

If your model weights occupy 30 GB on an 80 GB GPU, you have roughly 50 GB left for KV caches (minus activation memory and framework overhead). How many requests fit depends on the model architecture, the KV cache precision, and the sequence lengths involved. A batch of 64 requests, each with 2,048 tokens of context, might consume far more than 50 GB of KV cache – at which point you simply can't fit that many requests.

This is the central batching tradeoff: **larger batches improve GPU utilization and throughput, but each additional request in the batch consumes KV cache memory**. Push the batch size too high and you'll run out of GPU memory. Keep it too low and you're leaving performance on the table. The memory management techniques in Section 6.3 and the preemption policies we'll discuss at the end of this chapter exist to navigate this tradeoff.

Example workload

To compare the batching strategies that follow, we'll use the same small workload throughout: five requests arriving at staggered times, each needing 2 prefill steps and a varying number of decode steps.

Table 5.1: Example workload used in the batching diagrams below.

Request	Arrival time	Decode steps
A	0	4
B	1	6
C	1	3
D	4	4
E	5	3

This is a toy example. In practice, prefill time depends on input length and can vary enormously across requests, queuing delays can stretch to seconds under heavy load, and decode runs for

hundreds or thousands of steps. We've flattened all of that to keep the diagrams readable. The point is to show how each strategy handles staggered arrivals and variable-length outputs, not to depict realistic timescales.

Static batching

The simplest batching strategy is **static batching**: collect a fixed number of requests, process them all together, and don't start the next batch until every request in the current batch has finished generating.

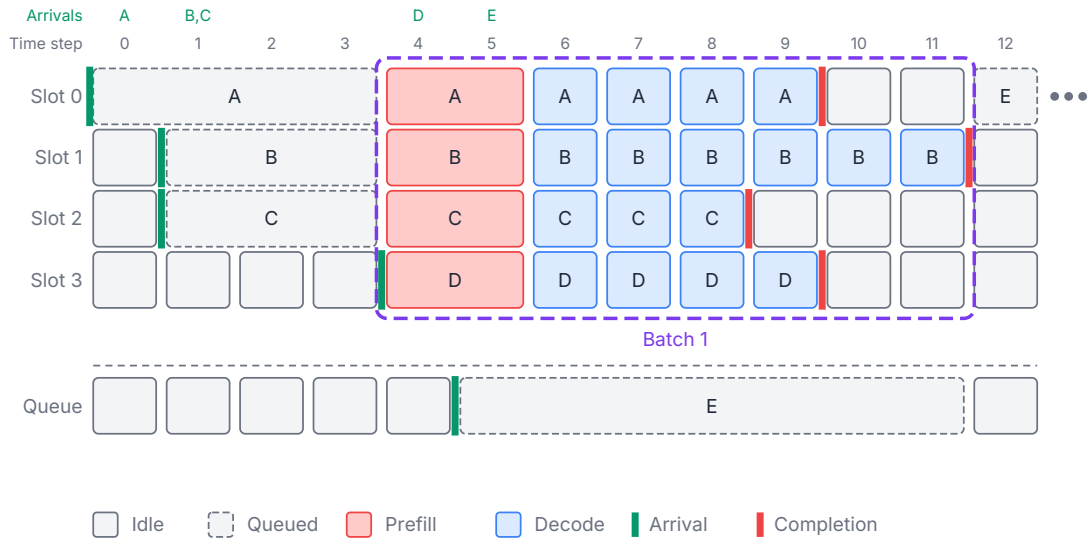


Figure 5.1: Static batching

Figure 5.1 shows the results of our workload with a static batch size of 4 requests. Requests A through D start work at time step 4 and finish at time step 11. At time step 12, Request E is still waiting for three more requests to arrive, and could finish much later. There are obvious problems with these results. First, the GPU sits idle for the first four time steps even though we have requests in hand, because we don't have a full batch yet. Next, Requests A, C, and D finish earlier but their batch slots sit empty. The GPU does no useful work for those slots while continuing to work just on Request B to complete. Request E is stuck in the queue even though there are free batch slots starting at time step 9. This problem where a single slow request holds up the entire batch is known as **head-of-line blocking**. Imagine how much worse this problem would be if B were a very long request generating thousands of tokens. There are also issues with prefill, which are not so obvious from our simplified diagram. If requests have different input lengths and prefill is done with all of them at the same time (as shown here with A, B, C, and D), then the shorter inputs need to be padded to match the longest one, wasting compute on dummy tokens during prefill.

Static batching works fine for offline workloads where all inputs and outputs have uniform length – say, scoring a dataset of fixed-length classification examples. Having the entire dataset available at the beginning, instead of dealing with unknown arrival times, makes scheduling much easier. For anything with variable-length inputs, variable-length outputs, and stochastic arrival time, which is the common case in online LLM generation, it wastes too many GPU cycles.

Dynamic batching

Dynamic batching improves on static batching by allowing the batch size to vary. Instead of fixing the batch size up front, the scheduler collects requests from a queue and forms a batch based on what's available, subject to a maximum batch size and a timeout. If the batch size is 32 and only 5 requests are waiting, once the timeout elapses, a batch with 5 requests will start processing, rather than waiting for a full batch of 32. This reduces queuing delay and adapts to variable arrival rates.

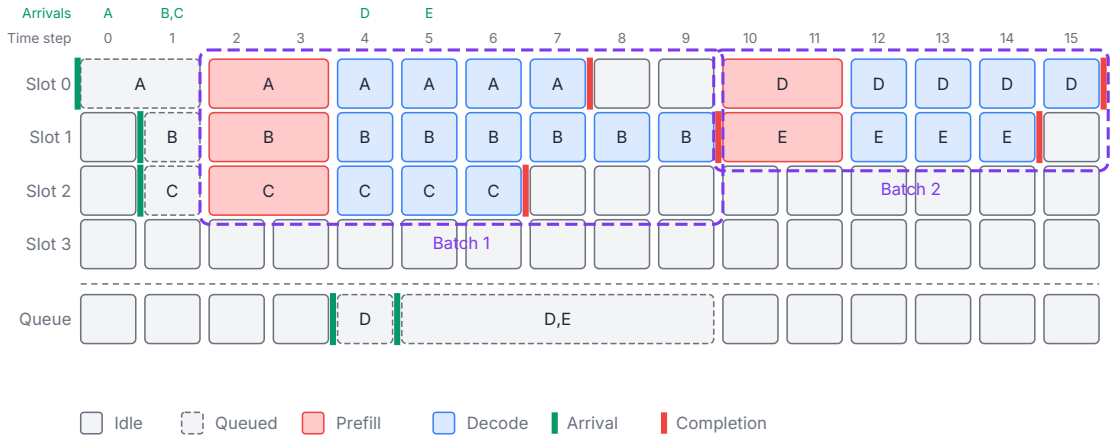


Figure 5.2: Dynamic batching

Figure 5.2 shows the results of dynamic batching on the sample workload in Table 5.1 for a maximum batch size of 4 and a timeout of 2 time steps. Now, the first batch starts at time step 2 and finishes requests A, B, and C by time step 9. Requests D and E start immediately after Batch 1 finishes. These two requests start at time step 10 and finish at time step 15. Compared to static batching, we have reduced the amount of waiting before Batch 1 starts. But it still has the same fundamental problem as static batching during decode: the batch doesn't complete until the last request finishes generating. Short-output requests leave batch slots that sit idle while they wait for long-output requests to wrap up, so a really long Request B still wastes a lot of GPU capacity. In our example, requests D and E stay queued until time step 10 even though the GPU has capacity starting at time step 7.

Continuous batching

The breakthrough that modern serving systems are built on is **continuous batching**, also called **in-flight batching**, introduced by Orca (Yu et al. 2022). The key insight is to operate at the granularity of individual decode iterations rather than complete requests.

In continuous batching, the scheduler makes a decision at every decode step: which requests should be in this step's batch? When a request finishes generating (by hitting a stop token or a length limit), its slot is immediately freed and can be filled by a new request from the queue, without waiting for the rest of the batch to finish. This eliminates head-of-line blocking.

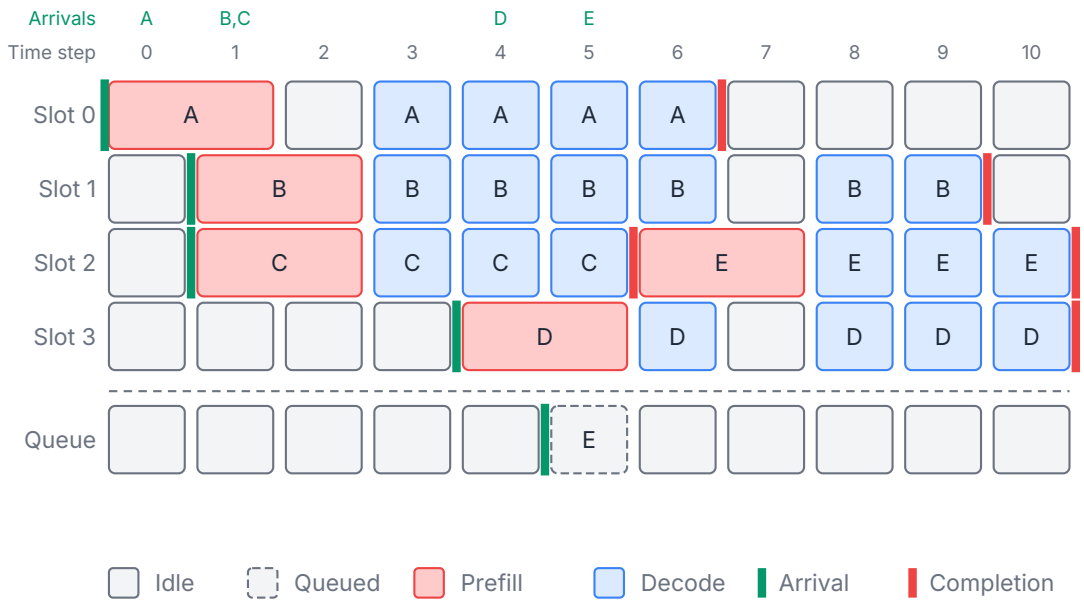


Figure 5.3: Continuous batching

On our workload from Table 5.1, we see in Figure 5.3 that continuous batching completes all five requests by time step 10! Requests A, B, C, and D each start as soon as they arrive. Request E does have to wait, but it starts processing as soon as the first batch slot is available, at time step 6. Continuous batching has eliminated unnecessary GPU idle time waiting for batches to be full as well as waiting for every request in a batch to complete.

The throughput improvement from continuous batching can be substantial. Under workloads with high variance in output length, continuous batching can improve throughput by 2-3x or more compared to static batching, simply by keeping batch slots occupied with useful work. This is why continuous batching is the foundational scheduling technique in serving frameworks like vLLM (Kwon et al. 2023) and SGLang (Zheng et al. 2023).

> **Note**
>
> Continuous batching requires the serving system to manage per-request state (KV cache, position counters, stop conditions) independently for each slot, and to handle the bookkeeping of inserting and removing requests mid-generation. This is more complex than static batching but is well worth the implementation effort for any production system.

Continuous batching has eliminated most of the GPU idle time, but it has created a new problem with long prefills causing requests that are doing decoding to wait. In a simple implementation where the prefill for each input is processed all at once, we don't have the ability to insert multiple decode requests during the middle of a prefill. And it's worthwhile to remember that the prefill of a long input can take many times longer than a decode step (not just double as we've depicted).

This is less of a GPU utilization problem, which would show in MFU, than it is a problem with increasing TPOT and overall request completion latency. In the next section, we will unpack more of the issues with prefill and decode, and ultimately solve this issue with holes in decoding.

5.2 Prefill Strategies

The prefill–decode tension

In Section 3.5, we characterized prefill as compute-bound and decode as memory-bandwidth-bound. This difference creates a scheduling headache when the system has to do both at the same time.

Consider a serving system running continuous batching with 30 requests in the decode phase. A new request arrives with a 4,096-token prompt. The system needs to prefill this request before it can start decoding. But the prefill for a 4,096-token prompt is a large, compute-intensive operation that can take hundreds of milliseconds – during which the 30 decode requests are blocked from making progress.

From those 30 requests' perspective, this is terrible. Each one is generating text for a user, and they all just stalled for the duration of someone else's prefill. Their TPOT spikes, the streaming output freezes, and the user experience degrades. This is the **prefill–decode tension**: serving a new request's prefill comes at the cost of stalling existing requests' decode steps.

The tension gets worse as prompt lengths increase. With the trend toward longer contexts – 32K, 128K, or even longer – a single prefill can take multiple seconds, creating an unacceptable stall for all active decode requests.

Chunked prefill

Chunked prefill, introduced by Sarathi (Agrawal et al. 2023), resolves this tension by splitting a long prefill into smaller chunks and interleaving them with decode steps.

Instead of processing all 4,096 input tokens in one large forward pass, the system might break the prefill into 8 chunks of 512 tokens each. At each iteration, the scheduler runs one prefill chunk alongside the batch of decode tokens. The decode requests make progress every iteration, and the new request's prefill completes over 8 iterations instead of blocking everything for one long pass.

The chunk size is a tuning parameter. Smaller chunks mean smoother decode progress but increase the number of iterations needed to complete the prefill, adding overhead from additional kernel launches and attention computations across chunk boundaries. Larger chunks reduce this overhead but create longer pauses in decode progress. In practice, chunk sizes of 256–2,048 tokens are common, depending on the model size, the capability of the GPU (or other hardware accelerator), and the latency sensitivity of the workload.

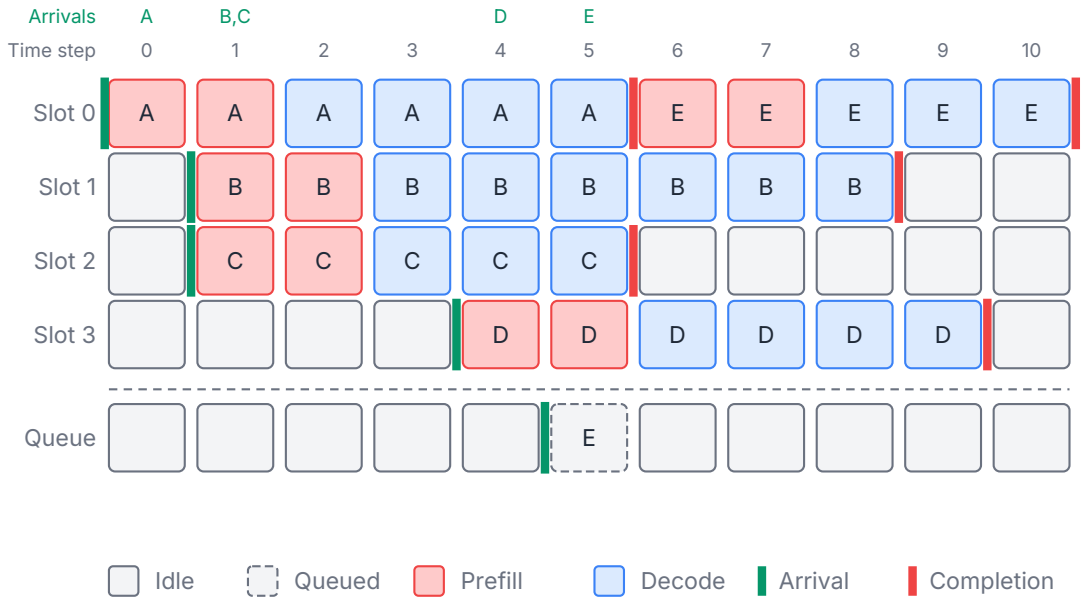


Figure 5.4: Chunked prefill

For our sample workload from Table 5.1, we can now improve upon continuous batching by breaking up our prefills (which we simplified to all be exactly twice as long as a decode step) into separate chunks. Figure 5.4 shows Requests A, B, and D all complete one time step sooner than with vanilla continuous batching. (In reality, if prefill was dozens of times slower than decode, the savings would be even greater.)

Chunked prefill has a nice bonus: it improves GPU utilization during decode iterations. The prefill chunk is compute-heavy and the decode tokens are bandwidth-heavy, so combining them in the same iteration gives the GPU a mix of both types of work. The prefill chunk tokens keep the compute units busy while the decode tokens are heavily utilizing GPU bandwidth to feed the compute units. It doesn't take the GPU any longer to complete a prefill chunk with other decode steps than it would to do the prefill chunk alone. This piggybacking effect can be as significant as the reduced stalling.

> **Note**
>
> Chunked prefill interacts well with continuous batching. The scheduler treats each prefill chunk as a partial request that occupies a "prefill slot" for one iteration. When the last chunk completes, the request transitions to the decode phase and joins the regular decode batch. The system never has to choose between serving new requests and making progress on existing ones – it does both simultaneously.

Disaggregated prefill

Chunked prefill is effective, but it's still a compromise – prefill and decode share the same GPU and compete for the same resources. **Disaggregated prefill** takes a more radical approach: run prefill and decode on entirely separate hardware instances (Zhong et al. 2024).

The idea is straightforward. Prefill is compute-bound, so prefill instances should be optimized for compute throughput. Decode is memory-bandwidth-bound, so decode instances should be optimized for memory bandwidth and capacity. By disaggregating the two phases onto separate hardware, each can be independently optimized and scaled without interfering with the other.

The workflow looks like this:

1. A request arrives and is routed to a **prefill instance**
2. The prefill instance processes the full input and generates the KV cache
3. The KV cache is transferred over a fast interconnect (such as NVLink or InfiniBand) to a **decode instance**
4. The decode instance generates tokens autoregressively using the transferred KV cache

A comparison of co-located and disaggregated prefill on a sample workload is shown in Figure 5.5.

The main cost is the KV cache transfer. For a large model with a long prompt, the KV cache can be several gigabytes, and this data needs to move between machines before decoding can begin. The transfer latency adds directly to TTFT, so disaggregated prefill needs a fast interconnect to be practical. With NVLink or high-speed InfiniBand, this transfer can be completed in tens of milliseconds, which is often acceptable.

The benefits go beyond eliminating interference. Disaggregated prefill lets you scale prefill and decode capacity independently. If your workload has long prompts but short outputs, you

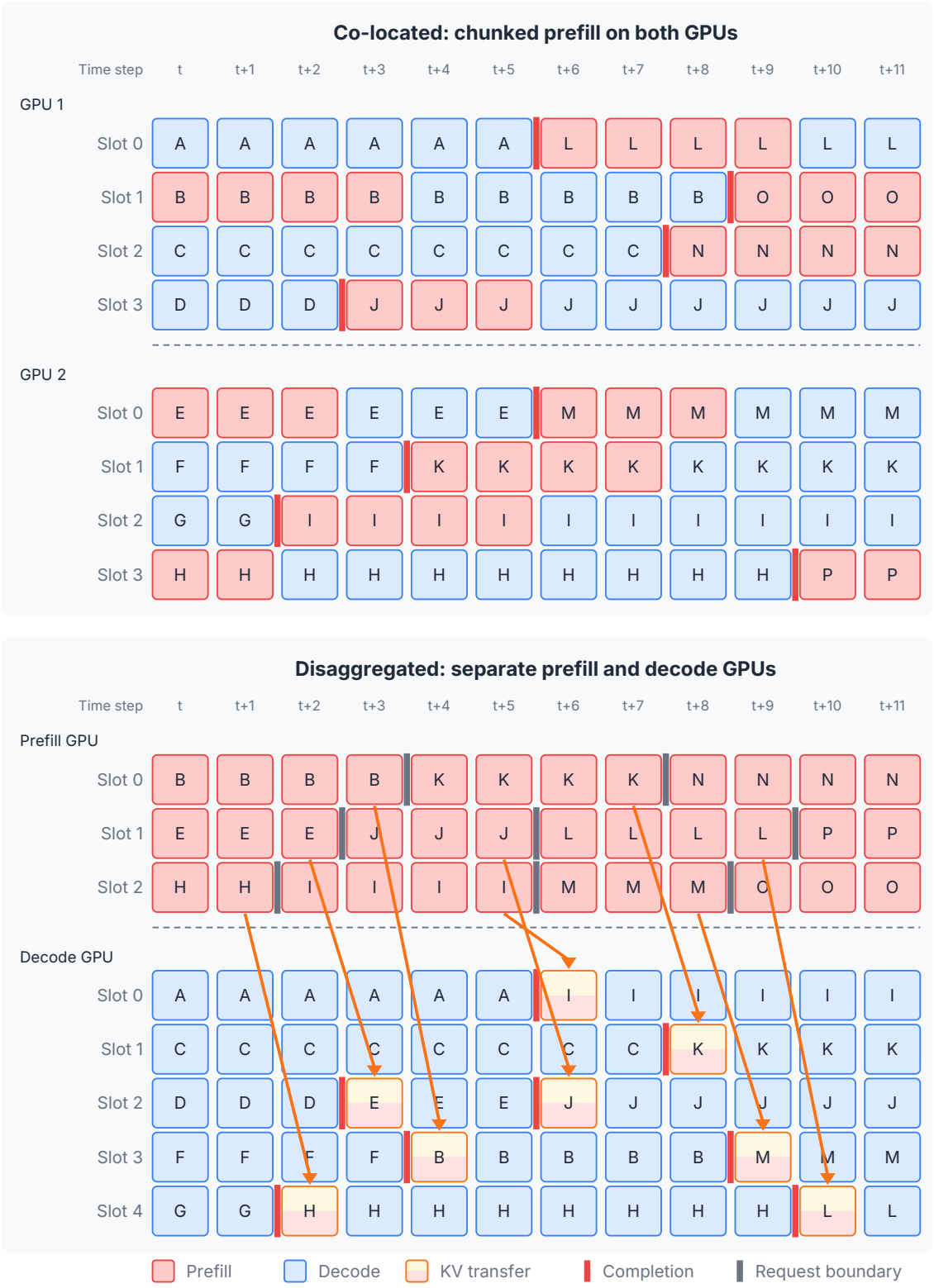


Figure 5.5: Disaggregated prefill

might need more prefill capacity and less decode capacity. If you're serving a chatbot with short prompts but long responses, the ratio flips. With disaggregated serving, you allocate hardware to match the workload rather than being stuck with a fixed ratio.

Disaggregated prefill is a more complex deployment architecture, and the KV cache transfer adds latency and requires fast networking. For many deployments, chunked prefill provides enough isolation between the phases. But at large scale, where the workload is diverse enough and the infrastructure supports fast interconnects, disaggregation can provide significant efficiency gains.

5.3 Ragged batching

Even with continuous batching, there's still a source of waste: when requests in a batch have different sequence lengths, the shorter sequences need padding to match the longest one in the batch for the attention computation. **Ragged batching** eliminates this by concatenating all the tokens from different requests into a single flat sequence, with appropriate attention masks to prevent cross-request attention.

Instead of a padded batch tensor of shape $[B, S_{\text{max}}]$ where shorter sequences waste compute on pad tokens, ragged batching creates a packed tensor of shape $[T_{\text{total}}]$ where T_{total} is the sum of all actual token counts. An index array tracks where each request's tokens begin and end, and the attention mask ensures that tokens from one request can't attend to tokens from another. A comparison of simple padded batching and ragged batching is shown in Figure 5.6 and Figure 5.7.

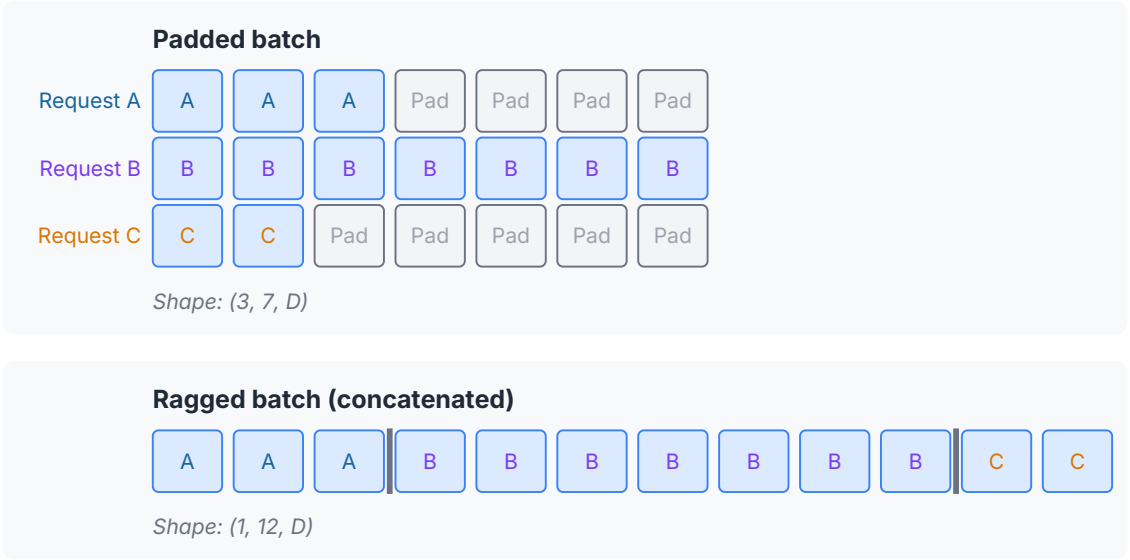


Figure 5.6: Ragged batching compared to simple batching with padding

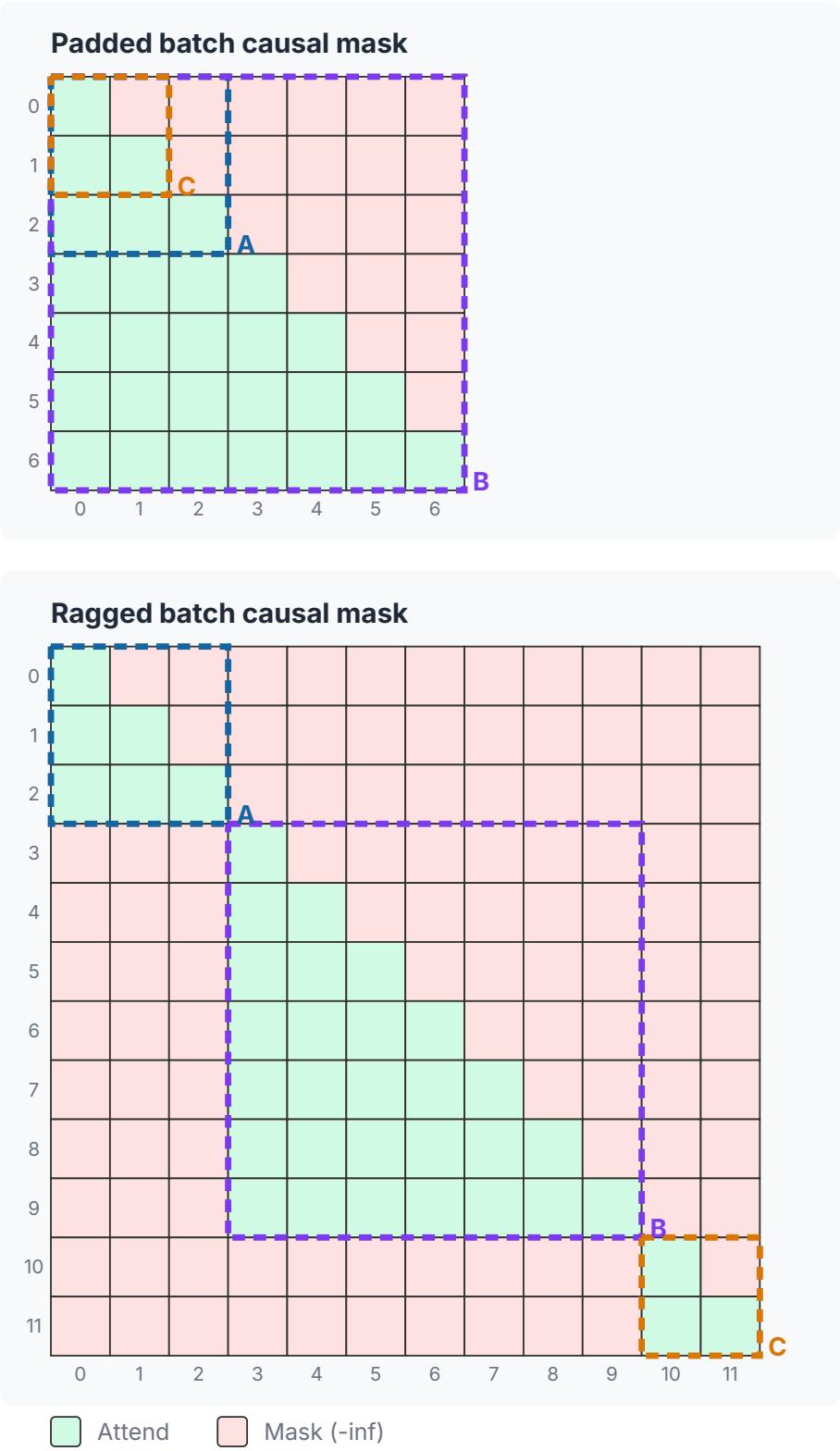


Figure 5.7: Causal mask for ragged batching. The dashed squares show the portions applicable to each of the requests.

This matters most during prefill, where the input lengths across requests can vary widely. A batch with one 2,048-token prompt and three 128-token prompts would waste enormous compute on padding without ragged batching. With ragged batching, the compute is proportional to the actual number of tokens: $2{,}048 + 3 \times 128 = 2{,}432$ tokens instead of $4 \times 2{,}048 = 8{,}192$ tokens.

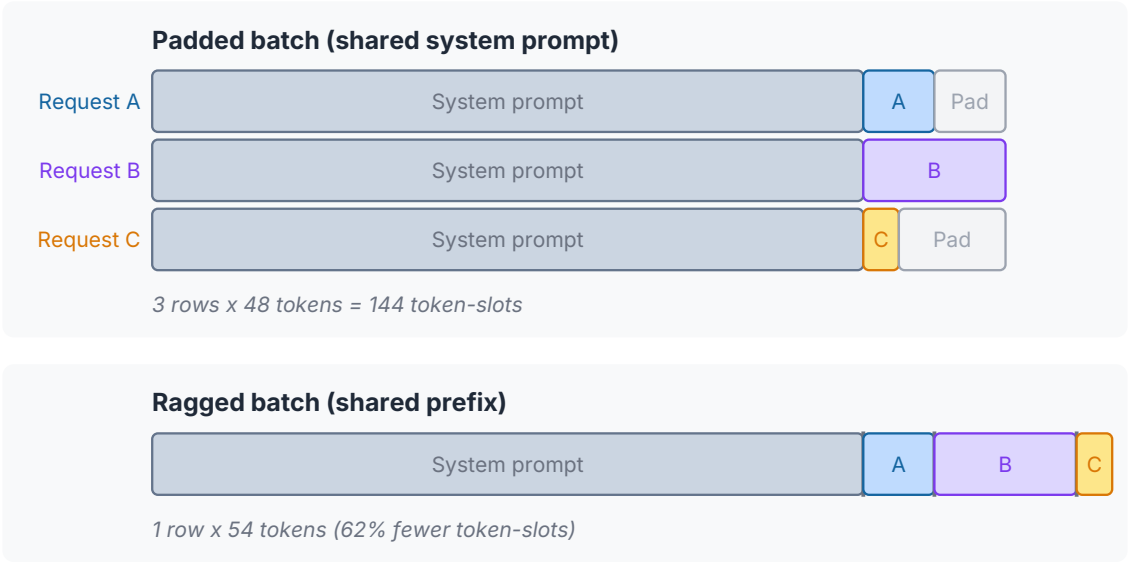


Figure 5.8: Ragged batching example with a shared prefix

The benefits of ragged batching combine really well with prefix caching, which we will discuss in Section 6.4. Figure 5.8 briefly shows how sharing the prefix in ragged batching can save huge amounts of compute with long shared prefixes, such as a long system prompt.

5.4 Request Scheduling and Prioritization

Granularity of scheduling

TODO: write about how scheduling at the step-level gives more control. Where does this go relative to the other sections?

Priority-based scheduling

Not all requests are equally urgent. A real-time chatbot response that a user is waiting on matters more – at least from a latency perspective – than a background batch job scoring a dataset. **Priority-based scheduling** lets the serving system express this difference.

In a priority-aware scheduler, each incoming request is assigned a priority level. High-priority requests are placed at the front of the scheduling queue, while lower-priority requests wait until there's spare capacity. The scheduler decides at each iteration which requests to include in the batch, and high-priority requests get scheduled first.

This enables a common production pattern: a system serves both real-time and batch traffic on the same hardware. Real-time requests get low TTFT because they jump the queue. Batch requests fill in the gaps when real-time traffic is light, keeping the GPU busy during off-peak periods. The result is better overall utilization without sacrificing the latency SLA for interactive users.

SLA-aware scheduling takes this a step further. Instead of simple priority levels, the scheduler tracks each request's deadline – for example, "this request must produce its first token within 500 ms." The scheduler then makes admission decisions based on whether it can meet the deadline. If admitting a new request would cause an existing request to miss its deadline, the scheduler may defer the new request or take other action.

> **Note**
>
> The details of how serving frameworks like vLLM and SGLang implement their iteration-level schedulers – deciding which mix of prefill chunks and decode tokens to process each step – are covered in Section 8.1. The scheduler is the beating heart of a serving system, and getting it right has a large impact on goodput.

Preemption

What happens when the system runs out of KV cache memory? With continuous batching, the scheduler is constantly admitting new requests into the batch. Each new request needs KV cache memory, and the existing requests' KV caches grow with each decode step. Eventually, something has to give.

Preemption is the mechanism for handling this. When KV cache memory is exhausted and a new high-priority request needs to be served, the scheduler can preempt one or more existing requests – suspending them and freeing their KV cache memory to make room. When resources become available later, the preempted requests resume.

There are two strategies for handling the KV cache of a preempted request:

Swap evicts the preempted request's KV cache blocks from GPU memory to CPU memory. When the request resumes, the blocks are swapped back. This preserves the work already done – the request picks up exactly where it left off without any recomputation. The cost is the time to transfer data over the PCIe bus. For a KV cache of several hundred megabytes, a swap to CPU memory at PCIe Gen5 speeds (~64 GB/s) takes a few milliseconds. This is usually acceptable, but the CPU needs enough free memory to hold the evicted blocks, and the PCIe bandwidth is shared with other transfers.

Recomputation takes a simpler but more expensive approach: discard the preempted request's KV cache entirely. When the request resumes, the system reruns prefill from scratch

to rebuild the KV cache. This frees GPU memory immediately and requires no CPU memory, but it wastes the compute that went into the original prefill plus all the KV cache entries accumulated during decode. For a request with a long prompt, recomputation can be very costly.

Table 5.2: Comparison of preemption strategies. See Section 8.4 for production policies.

Strategy	GPU memory freed	CPU memory needed	Resume cost	Best when
Swap	Yes	Yes	Transfer latency	Plenty of CPU memory, short pauses
Recomputation	Yes	No	Full prefill re-run	Limited CPU memory, rare preemption

The choice between swap and recomputation depends on the situation. If the system has ample CPU memory and preemptions are brief, swapping is clearly better – you avoid redoing work. If CPU memory is tight or the preempted request would sit idle for a long time (making the cached data stale anyway), recomputation may be simpler and more predictable. Some systems use a hybrid approach: swap when possible, fall back to recomputation when CPU memory is full.

OOM handling and graceful degradation

Preemption is ideally a rare event – it happens only when the system is under heavy load and the scheduler can't fit all active requests into memory. But in adversarial conditions – a sudden burst of long-context requests, for example – preemption alone may not be enough. The system can run out of both GPU and CPU memory for KV caches.

At this point, the system needs a last-resort strategy. Options include:

- **Rejecting new requests**: return an error and let the client retry later. This protects existing requests but reduces throughput.
- **CPU offloading**: proactively keep some fraction of KV cache blocks on CPU memory, streaming them to the GPU as needed. This trades PCIe bandwidth for GPU memory capacity.
- **Reducing batch size**: admit fewer concurrent requests, prioritizing the ones already in progress. New requests queue up and wait.

The details of how production systems handle these failure modes – and how to configure policies that balance throughput, latency, and reliability – are covered in Section 8.6.

Putting scheduling together

The scheduling techniques in this chapter form a stack. At the bottom, the batching strategy determines how requests are grouped for each forward pass. Continuous batching provides the foundation, keeping batch slots full at all times. Ragged batching eliminates padding waste within each batch. On top of that, chunked or disaggregated prefill manages the tension between serving new requests and making progress on existing ones. And the priority and preemption policies govern which requests get resources when demand exceeds supply.

None of these techniques change the model or its weights. They don't reduce the number of FLOPs needed to process a token or shrink the KV cache per token. What they do is ensure that the FLOPs the GPU performs are useful FLOPs – not wasted on padding, not blocked by long prefills, not sitting idle while a slow request holds up the batch. In the next chapter, we'll turn to techniques that operate on individual requests: optimizing the attention computation, managing the KV cache efficiently, and breaking the one-token-at-a-time constraint of autoregressive decoding.

5.5 Further Reading

The Orca paper (Yu et al. 2022) is the foundational reference for continuous batching. It introduced **iteration-level scheduling** – the idea that the scheduler should make decisions at every decode step rather than waiting for an entire batch to complete. If you read one paper from this chapter, make it this one. The system design is clean, and the evaluation clearly shows the throughput gains over static batching.

Sarathi (Agrawal et al. 2023) introduced chunked prefill and the idea of piggybacking decode tokens onto prefill chunks to keep the GPU busy during long prefills. The paper includes a clear analysis of how unchunked prefills create latency bubbles for co-scheduled decode requests, and how chunk sizing trades off between prefill throughput and decode latency stability.

DistServe (Zhong et al. 2024) takes the opposite approach to managing the prefill-decode tension: rather than interleaving them on the same GPU, it disaggregates them onto separate hardware. The paper's key contribution is showing that the optimal hardware configuration for prefill (maximize compute) is fundamentally different from decode (maximize memory bandwidth), and that separating them improves goodput. Splitwise (Patel et al. 2024) independently explored a similar disaggregation idea, with additional analysis of how to balance capacity between prefill and decode pools as workload mix changes.

For a broader treatment of how request scheduling interacts with memory management and SLA compliance, the vLLM paper (Kwon et al. 2023) covers not just PagedAttention but also the preemption and swap policies that sit on top of the memory allocator. The scheduling policy discussion in sections 4-5 of that paper complements the treatment here.

6 Request Compute, Memory, and Latency Bottlenecks

The previous chapter tackled scheduling — how to keep the GPU busy by managing how requests are handled. This chapter examines the efficiency with which individual requests are handled. The first two sections address memory-bandwidth bottlenecks at the kernel level – optimizations that apply to both prefill and decode. The remaining sections target the sequential decoding constraint, exploring techniques that generate multiple tokens per step or amortize the cost of each one.

6.1 Memory-Aware Attention Kernels

Why standard attention wastes bandwidth

To understand why attention is expensive in practice, you need to think about where data lives during computation. Recall from Section 3.4 that the GPU has small, fast on-chip registers and SRAM, and a large, slow off-chip HBM. The standard implementation of self-attention is depicted in Figure 6.1 and works roughly like this:

1. Read the Q and K tensors, each shape (**H**, **S**, d_K) — reminder, this would be (**B**, **H**, **S**, d_K) with batched processing but we're avoiding the clutter of repeatedly listing the batch dimension.
2. Compute the score matrix Scores $= QK^T$ and write Scores, shaped (**H**, **S**, **S**), to HBM.
3. Read Scores shaped (**H**, **S**, **S**) back from HBM.
4. Apply SoftMax to get the attention weights and write Attn, also shaped (**H**, **S**, **S**), to HBM.
5. Read Attn shaped (**H**, **S**, **S**) back from HBM, compute the weighted values Weighted Values $=$ Attn$V$.
6. Write the weighted values, shaped (**H**, **S**, d_K), back to HBM

The read of Q and K at the beginning needs to happen in some form, and the writing of the weighted values also needs to happen at the end. But the writing and reading back of the (**H**, **S**, **S**) Scores and Attn tensors in the middle wastes bandwidth, especially as **S** gets large. For a sequence of length 4,096, the score matrix alone is $4096 \times 4096 = 16.7$ million entries per head. The actual computations of the matrix multiplications and softmax are fast. What's slow is the round-trip to HBM for these intermediate results.

This is a textbook example of a memory-bandwidth-bound operation. The GPU spends most of its time waiting for data to move to and from HBM, not doing arithmetic. And the problem

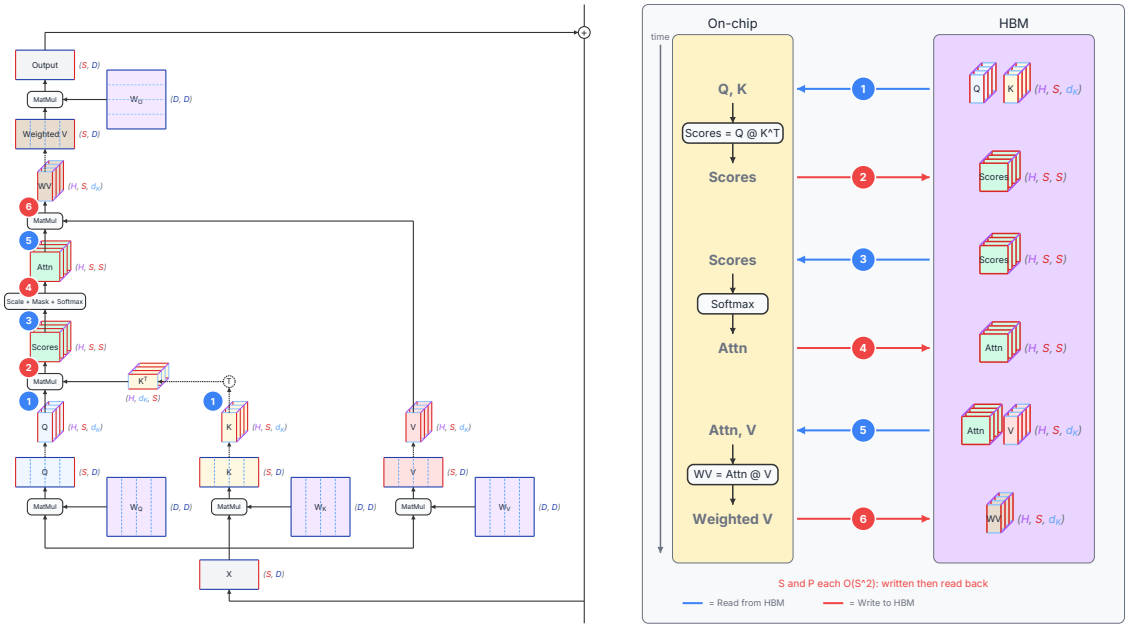


Figure 6.1: Memory access pattern for a straightforward implementation of self-attention

scales quadratically with sequence length — double the sequence length and you quadruple the intermediate memory traffic.

It's worth noting that even a single matmul like Scores $= QK^T$ doesn't happen as one monolithic operation. The Q and K matrices are far too large to fit in SRAM, so the GPU kernel **tiles** the computation. Consider Q with shape **(S,** d_K**)** and K^T with shape **(**d_K**, S)**. A standard tiled matmul loops over row-blocks of Q (groups of query positions) and column-blocks of K^T (groups of key positions). For each pair, the kernel loads a block of Q rows and a block of K^T columns into SRAM, computes their partial product, and writes the result into the corresponding tile of Scores in HBM. Since d_K is small (typically 64 or 128), the inner dimension often fits in a single pass. But the outer dimensions don't: each block of Q rows is re-read from HBM once for every column-block of K^T, and each column-block of K^T is re-read for every row-block of Q. This means the total HBM reads exceed the theoretical minimum of reading each matrix exactly once — the GPU re-reads portions of Q and K multiple times to tile through the computation. But that extra read traffic is modest and scales with **S**, not S^2. The real waste in standard attention isn't within any single matmul — it's the round-trips *between* operations: writing the **(H, S, S)** Scores matrix to HBM just so softmax can read it back, then writing the attention weights just so the next matmul can read them.

FlashAttention

FlashAttention (Dao et al. 2022) solves this by never materializing the full attention matrix in HBM. If we had enough SRAM, we could store these attention matrices there, and we

would avoid the round trip to HBM, but we don't. The core idea with FlashAttention is **tiling**: break the Q, K, and V matrices into small blocks that fit in SRAM, then compute the attention output block by block, keeping all intermediate results on-chip.

The algorithm works in two nested loops. The outer loop iterates over blocks of K and V. The inner loop iterates over blocks of Q. For each pair of blocks, the kernel computes the local attention scores, applies a local softmax, and accumulates the weighted sum – all in SRAM. The tricky part is that softmax requires the maximum and sum over the full row of scores, not just the current block. FlashAttention handles this with an **online softmax** technique that maintains running statistics and corrects the accumulated output as new blocks are processed.

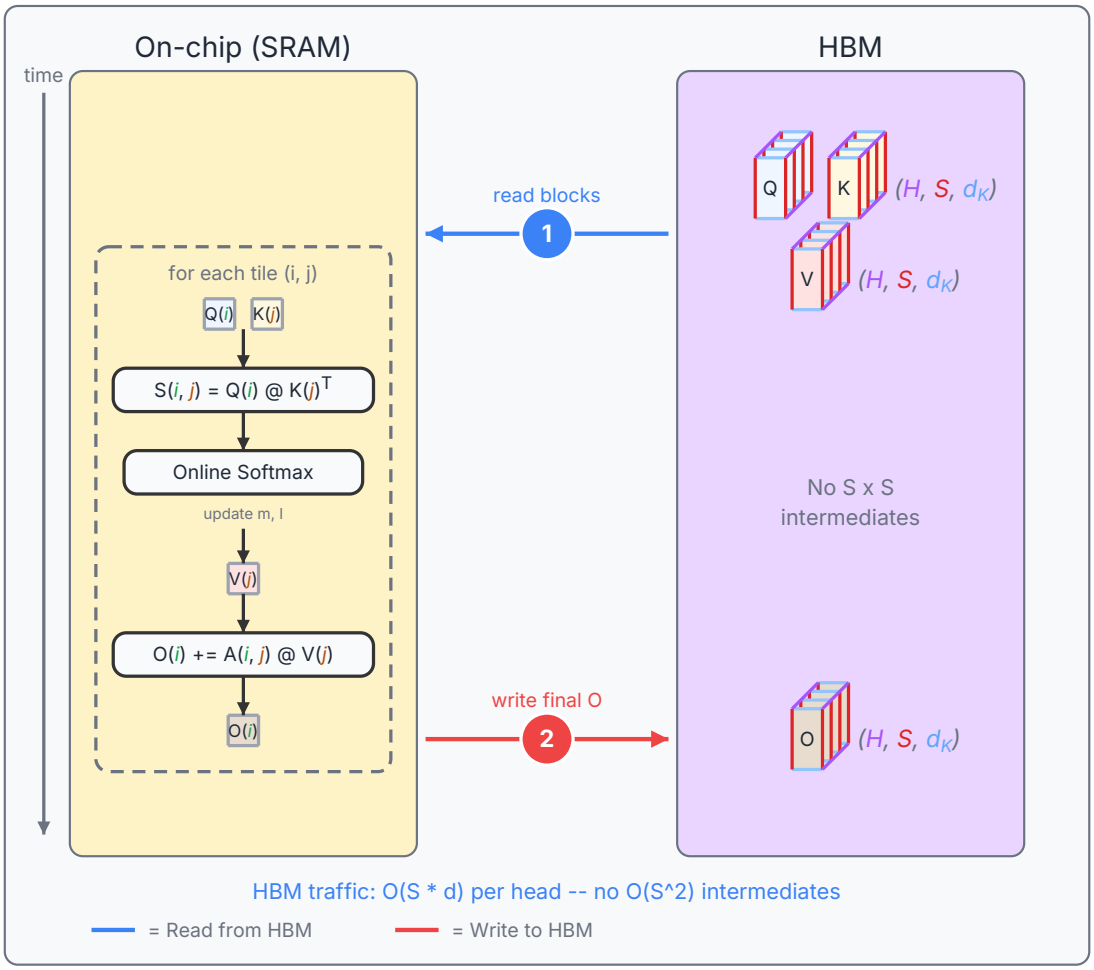


Figure 6.2: FlashAttention keeps all intermediate results on-chip — only two HBM transfers instead of six

The result is depicted in Figure 6.2. HBM traffic drops from $O(S^2)$ to $O(S \cdot d_K)$ for the attention matrix, where d_K is the head dimension (typically 64 or 128). For a 4K sequence

with $d_K = 128$, that's a reduction from 16.7 million elements to about 524K elements – roughly a 32x reduction in memory traffic for the attention intermediates.

Here's the counterintuitive part: FlashAttention actually performs *more* FLOPs than standard attention. The online softmax rescaling adds extra multiply-accumulate operations. But wall-clock time drops substantially because the bottleneck was never the arithmetic – it was the HBM traffic. By trading a small increase in compute for a large decrease in memory movement, FlashAttention lands in a much better spot on the roofline.

FlashAttention applies to both prefill and decode. During prefill, the sequence length $\mathbf{S}$ is large (the full prompt), so the $O(\mathrm{S}^2)$ savings are dramatic. During decode, $\mathbf{S}$ is just 1 for the query (the new token), but the key and value sequences span the entire context, so the tiled approach still avoids unnecessary intermediate writes.

FlashAttention v1, v2, and v3

The original FlashAttention (Dao et al. 2022) established the tiling approach. Subsequent versions improved the implementation substantially:

FlashAttention-2 restructured the algorithm to improve GPU parallelism. The key change was swapping the inner and outer loops – iterating over Q blocks in the outer loop and K/V blocks in the inner loop. This reduced the amount of shared memory communication between warps (groups of 32 threads that execute together on the GPU) and improved occupancy on modern GPUs. The result was roughly a 2x speedup over v1.

FlashAttention-3 targeted the H100 specifically, exploiting its new hardware features. It introduced **warp specialization** – dedicating some warps to data loading while others perform computation, overlapping memory access and arithmetic within the same kernel. It also leveraged the H100's FP8 tensor cores and hardware-assisted asynchronous memory operations. These changes pushed FlashAttention closer to the theoretical peak throughput on H100 hardware.

The progression across versions illustrates a broader principle: writing a correct tiled attention kernel is one thing, but extracting maximum performance from a specific GPU architecture requires deep hardware-aware engineering. This is why FlashAttention has become a de facto standard rather than something every team reimplements.

> **Note**
>
> FlashAttention handles the tiling of attention computation within a single GPU. When the KV sequence is very long, you may also want to parallelize the attention computation *across* the sequence dimension using multiple GPU cores or even multiple GPUs. This is the role of FlashDecoding, which we'll cover in Section 7.6.

6.2 Kernel Fusion and Compute Graph Optimization

FlashAttention is a specific instance of a more general optimization strategy: reducing the number of times data makes a round trip to HBM by combining operations into fewer GPU kernels. This section covers the general techniques.

Kernel fusion

Every time the GPU finishes one kernel and starts the next, there's an implicit data handoff through HBM. The output of the first kernel gets written to HBM, and the input of the second kernel gets read from HBM. If those two kernels operate on the same data, this round trip is pure waste.

Kernel fusion combines multiple operations into a single GPU kernel so that intermediate results stay in registers or SRAM. Common fusion opportunities in transformer inference include:

- **Fused QKV projection**: instead of three separate matrix multiplications for Q, K, and V, fuse them into a single kernel that reads the input activations once and produces all three outputs
- **Fused softmax + mask**: apply the causal mask and softmax in one pass, avoiding a separate write-then-read for the masked scores
- **Fused add + LayerNorm**: combine the residual addition and layer normalization, keeping the intermediate sum in registers
- **Fused RoPE + attention**: apply rotary position embeddings to Q and K within the attention kernel itself

Each fusion eliminates one HBM round trip. For a single transformer layer, there might be 10 or more such opportunities. Across 80+ layers in a large model, these small savings add up to a meaningful reduction in total memory traffic and kernel launch overhead.

CUDA graphs

We introduced CUDA graphs briefly in Section 3.4. Here's the full picture.

During decode, the GPU executes the same sequence of kernels at every step – the same transformer layers in the same order with the same shapes (for a given batch size). But the CPU doesn't know that. At each step, it launches each kernel individually, waits for the GPU to be ready, sends the launch command, and moves on to the next kernel. Each launch takes roughly 5–10 microseconds, which doesn't sound like much, but a single decode step through a large model might involve hundreds of kernel launches. At 5 μs each, that's hundreds of microseconds of pure overhead – time the GPU spends idle waiting for the CPU to tell it what to do next.

A **CUDA graph** captures a fixed sequence of kernel launches into a single replayable object. You run the decode step once in "capture mode," which records every kernel launch, memory

copy, and synchronization point. Then on subsequent steps, you replay the entire graph with a single command. The CPU issues one launch instead of hundreds, and the GPU executes the full sequence without waiting.

The main limitation is that CUDA graphs are static. The captured graph assumes fixed tensor shapes and memory addresses. If the batch size changes, or a request finishes and leaves the batch, the graph is invalidated and must be recaptured. Serving frameworks handle this with a small pool of pre-captured graphs for common batch sizes and sequence lengths, falling back to eager execution for uncommon shapes.

Computation graph compilation

Rather than manually fusing kernels, you can let a compiler do it. **Computation graph compilers** analyze the full model graph, identify fusion opportunities, optimize memory layouts, and emit optimized GPU code.

`torch.compile` in PyTorch traces the model's computation graph and applies a series of optimizations including operator fusion, memory planning, and code generation through backends like Triton. For inference workloads, it can deliver significant speedups with minimal code changes.

TensorRT-LLM takes a more aggressive approach. It converts the model into NVIDIA's TensorRT format, which applies extensive graph-level optimizations – layer fusion, precision calibration (auto-selecting FP16 or FP8 for each operation), kernel auto-tuning, and memory-efficient execution planning. The compilation step takes longer, but the resulting engine is heavily optimized for the specific model and GPU.

The tradeoff is flexibility versus performance. Eager execution (no compilation) is the most flexible but the slowest. `torch.compile` offers a middle ground. TensorRT-LLM produces the fastest code but requires a more involved build step and is harder to modify. Most production serving frameworks use some form of graph compilation, often TensorRT-LLM for NVIDIA hardware.

6.3 KV Cache Engineering

The KV cache is one of the biggest memory consumers during inference, and managing it well has a direct impact on how many requests you can serve concurrently and how long sequences can be. This section covers the core techniques for reducing KV cache memory pressure.

KV cache memory formula

For a model with $\mathbf{L}$ layers, n_{kv} KV heads per layer, and head dimension d_K, each token adds the following to the KV cache:

$$\text{bytes per token} = 2 \times \mathrm{L} \times n_{kv} \times d_K \times b_{\text{dtype}}$$

The factor of 2 accounts for both keys and values. b_{dtype} is the bytes per element (2 for FP16/BF16, 1 for FP8/INT8).

For a concrete example, consider a 70B parameter model with $\mathrm{L} = 80$ layers, $n_{kv} = 8$ GQA heads (see Section 4.4), $d_K = 128$, stored in FP16:

$$\text{bytes per token} = 2 \times \mathrm{L} \times n_{kv} \times d_K \times b_{\text{dtype}}$$

On an 80 GB GPU that's already holding 35 GB of model weights (a 70B model in INT4), you're already over budget.

This arithmetic is why KV cache management is critical. Every technique in this section – paging, quantization, compression, and eviction – is aimed at fitting more requests into the same memory budget.

PagedAttention

In a naive implementation, you allocate a contiguous block of GPU memory for each request's KV cache sized for the maximum possible sequence length. This wastes memory in three ways, illustrated in Figure 6.3: **reserved** slots held for tokens that haven't been generated yet, **internal fragmentation** (allocated slots that will never be used because the sequence won't reach the maximum length), and **external fragmentation** (gaps between allocations that are too small to be useful).

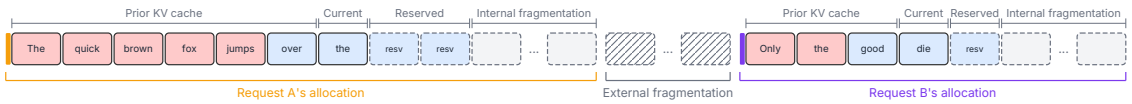


Figure 6.3: Memory waste with contiguous KV cache allocation. Each request pre-allocates for the maximum sequence length, leaving most of the memory unused.

PagedAttention (Kwon et al. 2023), introduced in vLLM, borrows the virtual memory model from operating systems. Instead of one big contiguous allocation per request, the KV cache is divided into fixed-size **pages** (also called blocks), typically holding 16 or 32 tokens each. A request's KV cache is a linked list of pages that can be scattered anywhere in GPU memory. The same requests from Figure 6.3 are shown in Figure 6.4 with a paged memory model.

The benefits are significant:

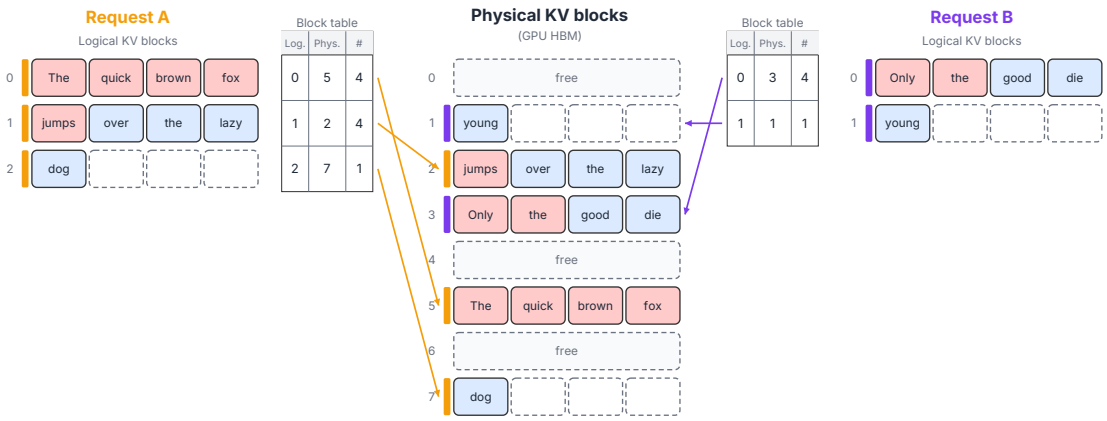


Figure 6.4: Paged attention

- **Near-zero fragmentation**: the only wasted space is the partial last page of each request. With 16-token pages, average waste is 8 tokens per request – negligible compared to the old approach of reserving memory for the maximum sequence length.
- **Dynamic allocation**: pages are allocated on demand as the sequence grows, so a short request doesn't tie up memory it will never use.
- **Sharing**: two requests that share a common prefix (same system prompt, for instance) can point to the same physical pages for the shared portion, storing the data once. This is the foundation for prefix caching, which we'll cover in Section 6.4.
- **Fine-grained preemption**: if the scheduler needs to free memory (Section 5.4), it can evict individual pages rather than killing an entire request. Pages can be swapped to CPU memory and swapped back later.

PagedAttention has become the standard approach for KV cache memory management. Nearly every major serving framework – vLLM, SGLang, TensorRT-LLM – uses some variant of it.

KV cache quantization

The KV cache formula above assumed FP16 storage. If you store keys and values in INT8 or FP8 instead, you immediately halve the memory per token. For our 70B model example, that drops per-request KV cache memory from 1.28 GB to 640 MB at 4K sequence length.

KV cache quantization is separate from weight quantization (Section 4.1). The model weights can be in one precision while the KV cache is stored in another. In practice, many deployments run model weights in INT4 or FP8 while keeping the KV cache in FP8 or INT8 – different components have different sensitivity to precision loss.

The quality impact is generally small. Keys and values are intermediate activations, not learned parameters, and small quantization errors in individual KV entries tend to average out across the many entries involved in an attention computation. The tradeoff is straightforward: half the memory for a small and often negligible reduction in output quality.

One research paper garnering a lot of interest is **TurboQuant** (Zandieh et al. 2025). Rather than simply choosing a new datatype, they use techniques like rotations and vector quantization to compress entries. They report success squeezing entries down to 3.5 bits without hurting LLM performance. Given that decoding is bandwidth bound, reductions in KV cache size provide close to linear speedups.

KV cache compression

Rather than quantizing individual elements, you can compress the KV cache using a learned low-rank representation. The idea is to store a compressed version of the keys and values that uses fewer dimensions, then reconstruct the full representation when needed for attention.

Multi-Head Latent Attention (MLA), introduced in DeepSeek-V2 (Liu et al. 2024), is the most prominent example. Instead of caching separate key and value tensors for each head, MLA compresses them into a single low-rank latent representation. This can reduce KV cache size by 5–10x compared to standard multi-head attention, at the cost of additional compute to decompress during attention. The key difference from the attention architecture changes covered in Section 4.4 (like GQA and MQA) is that MLA achieves compression without reducing the number of effective attention heads – it maintains full representational capacity while storing less data.

Selective KV cache and token eviction

Not all tokens in the context are equally important for generating the next token. In many attention patterns, a small fraction of tokens receive the vast majority of attention weight, while most tokens contribute very little. This observation motivates **selective KV cache** strategies that drop or compress low-importance entries.

Keyformer (Adnan et al. 2024) identifies important tokens based on accumulated attention scores. Tokens that consistently receive high attention weight across layers and heads are retained, while low-scoring tokens are evicted. The retained set is refreshed periodically as the model generates more tokens and attention patterns shift.

Native Sparse Attention (NSA) (Yuan et al. 2025) from DeepSeek takes a different approach, designing the sparsity pattern into the model architecture itself. Rather than using a learned or heuristic eviction policy at inference time, NSA defines structured sparse attention patterns that combine compressed coarse-grained tokens, selected high-relevance tokens, and a sliding window of recent tokens. This approach avoids the quality risk of post-hoc eviction because the model is trained to work with the sparse pattern.

The general tradeoff with token eviction is that it reduces memory but introduces risk. If the eviction policy drops a token that turns out to be important for a later generation step, the output quality degrades. Conservative eviction (keeping more tokens) is safer but saves less memory. The most robust approach – as NSA demonstrates – is to build the sparsity into the model during training rather than bolting it on at inference time.

6.4 Prompt and Prefix Caching

The shared prefix opportunity

In many real-world deployments, a large fraction of requests share the same prefix. A chatbot application might prepend a 2,000-token system prompt to every user message. A RAG pipeline might prefix retrieved context that is identical across multiple queries. A coding assistant might include the same repository-level context for every completion request.

Without caching, the system runs prefill on those 2,000 shared tokens for every single request – computing the same key-value pairs over and over. If you're serving 100 requests per second, that's 200,000 tokens of redundant prefill per second. The wasted compute shows up directly as higher TTFT and lower throughput.

RadixAttention

RadixAttention (Zheng et al. 2023), introduced in the SGLang framework, stores KV cache entries in a **radix tree** keyed by token sequences. A radix tree is a prefix-compressed trie: shared prefixes are stored once, and branches diverge only where token sequences differ. An example radix tree is shown in Figure 6.5.

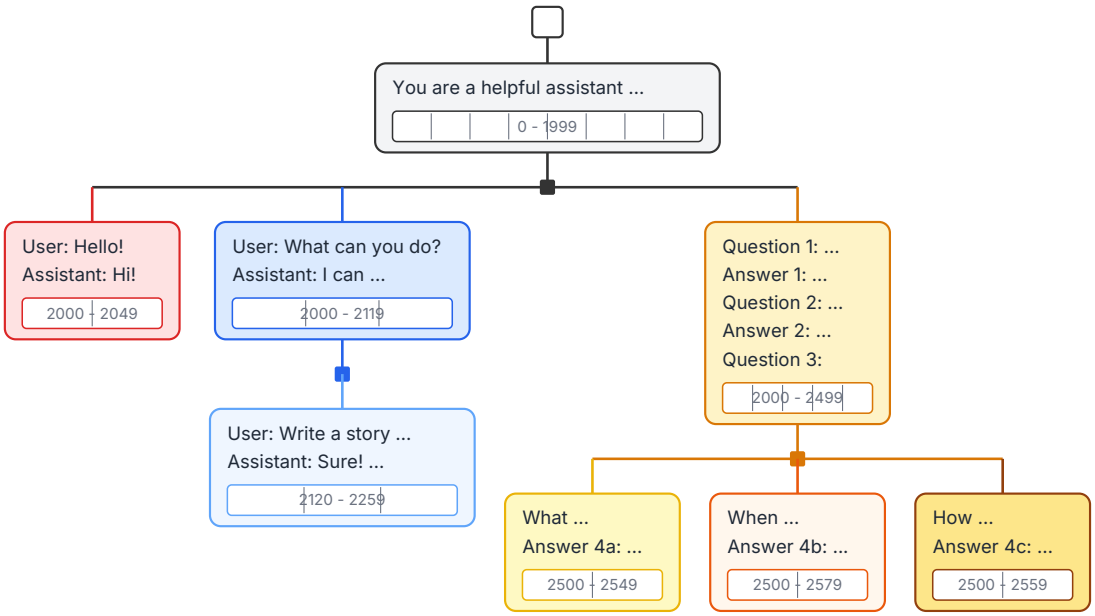


Figure 6.5: Radix attention: an example radix tree with several different prompts and responses sharing different common prefixes.

When a new request arrives, the system walks the radix tree matching its token sequence. If the first 2,048 tokens match a cached prefix, those KV entries are reused directly – no prefill

needed for them. The system only runs prefill on the remaining tokens that diverge from the cached prefix. For a request with a 2,000-token system prompt and a 200-token user message, this cuts prefill work by roughly 90%.

The radix tree structure enables fine-grained prefix matching that goes beyond simple exact-prefix caching. Two requests that share a system prompt but have different few-shot examples will still reuse the system prompt portion. A multi-turn conversation reuses all KV entries from previous turns.

Prompt caching in production APIs

The radix tree approach is what happens inside the inference engine. At the API level, prompt caching is often exposed as an explicit feature. When prefix caching is enabled, a provider hashes the prefix tokens and looks up cached KV entries. If found, the cached entries are reused and the customer pays reduced cost for the cached portion.

This is a win for both sides: the user gets faster TTFT and lower cost, and the provider avoids redundant compute. The effectiveness depends on how much prefix overlap exists in the workload – applications with long, stable system prompts benefit enormously, while applications with unique prompts for every request see little benefit.

Implementation dependencies

Prefix caching depends heavily on the paged KV cache from Section 6.3. In the PagedAttention model, the shared prefix pages are simply referenced by multiple requests through their page tables. The pages themselves are stored once in GPU memory, and a reference count tracks how many requests are using each page. When the last request using a shared prefix completes, the pages can be freed or kept for future reuse based on an eviction policy (typically LRU).

Without paged memory, sharing would require contiguous memory layouts to be identical across requests, which is impractical. PagedAttention makes prefix sharing a natural consequence of the memory model.

6.5 Speculative Decoding and Multi-Token Prediction

Everything up to this point has optimized the cost of individual prefill or decode steps. But there's a more fundamental constraint we haven't attacked yet: the autoregressive loop itself. Standard decoding produces one token per forward pass, and each token depends on the previous one. For a 500-token response, that's 500 sequential forward passes through the entire model. Even if each pass is perfectly optimized, you're still bound by the serial dependency chain.

This section covers techniques that break or amortize that sequential constraint – generating multiple tokens per model invocation.

The core idea of speculative decoding

Speculative decoding (Leviathan et al. 2022; C. Chen et al. 2023) is based on a simple but powerful observation: verifying a sequence of tokens is cheaper than generating them one at a time.

Here's the setup. You have a large **target model** – the model you actually want to sample from – and a small, fast **draft model**. The draft model generates k candidate tokens autoregressively (cheap, because the draft model is small). Then the target model processes all k candidates in a single forward pass, checking whether it would have generated the same tokens. Accepted tokens are kept. The first rejected token is resampled from the target model's distribution. Then the cycle repeats. A comparison of standard decoding and speculative decoding is shown in Figure 6.6.

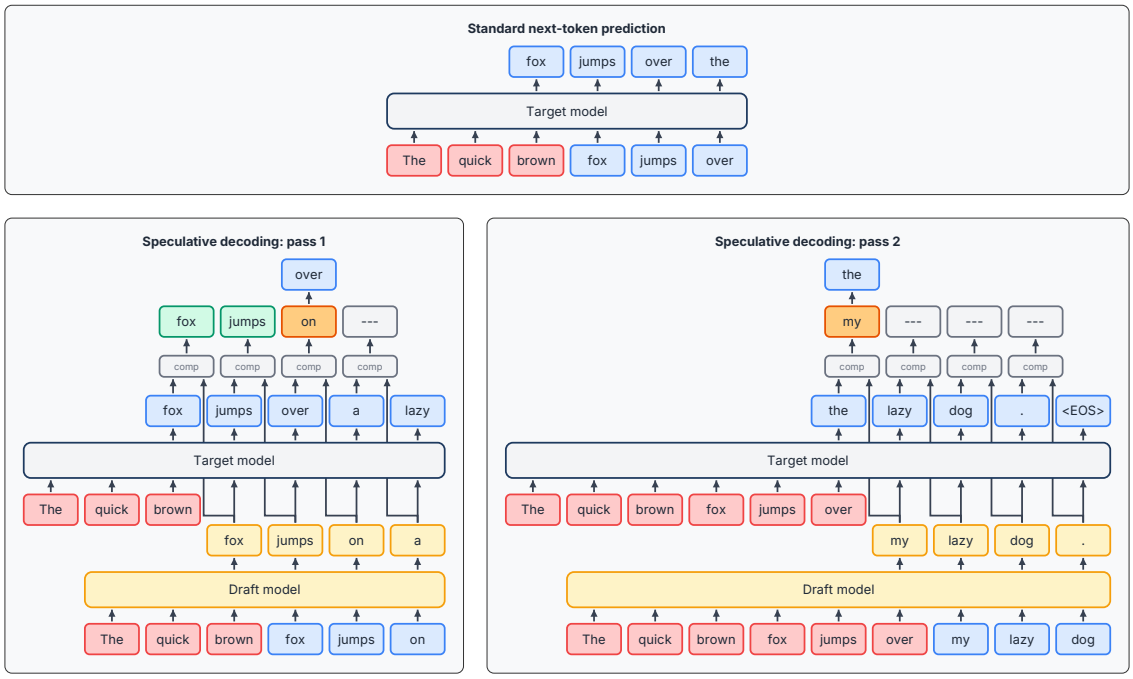


Figure 6.6: Speculative decoding

The reason this works is the asymmetry between generation and verification. Generating k tokens with the target model requires k sequential forward passes. But *verifying* k tokens requires just one forward pass, because you can feed all k tokens as input and check them all in parallel – the same way prefill processes multiple tokens at once. The early related work on this idea is Blockwise Parallel Decoding (Stern et al. 2018).

Acceptance rate and expected speedup

The **acceptance rate** α is the probability that the target model accepts a draft token. It depends on how well the draft model approximates the target model's distribution for the current context and task. Typical acceptance rates range from 0.6 to 0.9, depending on the draft model quality and the generation temperature.

The expected number of tokens generated per verification cycle is:

$$E[\text{tokens per cycle}] = \frac{1 - \alpha^{k+1}}{1 - \alpha}$$

where k is the number of draft tokens. For $\alpha = 0.8$ and $k = 4$, this gives about 3.4 tokens per cycle on average.

The speedup depends on the relative cost of the draft and target models. If the draft model runs at c times the speed of the target model (so a draft forward pass takes $1/c$ the time of a target forward pass), the wall-clock speedup is approximately:

$$\text{speedup} \approx \frac{E[\text{tokens per cycle}]}{1 + k/c}$$

The denominator accounts for the cost of the k draft passes (each costing $1/c$ of a target pass) plus the one verification pass. For $c = 10$ (the draft model is 10x faster), $k = 4$, and $\alpha = 0.8$, the speedup is roughly $3.4/1.4 \approx 2.4$x.

A crucial property of speculative decoding is that it produces *exactly* the same distribution as the target model. The acceptance/rejection mechanism is designed so that the output distribution is identical to what you'd get from running the target model autoregressively. This is not an approximation – it's a mathematically exact speedup.

Tree-based speculative decoding

Instead of generating a single chain of k draft tokens, the draft model can produce a **tree of candidates**. At each position, the draft model proposes multiple alternatives, branching into different possible continuations.

The target model verifies the entire tree in a single forward pass using a carefully constructed attention mask. The longest accepted path through the tree becomes the output. Because the tree explores multiple branches, it has a higher chance of finding a long accepted sequence than a single chain does, especially when individual token acceptance rates are moderate.

The cost is that the tree contains more total tokens than a single chain, so the verification forward pass does more work. But since the verification step is typically memory-bandwidth-bound (it's essentially a prefill over the tree tokens), the marginal cost of additional tree tokens is often small.

Inference with reference

Inference with reference (Yang et al. 2023) is a specialized form of speculative decoding where the "draft" comes from an existing text – a retrieved document, a cached previous response, or template text. If the target model's likely output overlaps significantly with the reference text, you can propose long spans of reference tokens as candidates and verify them in a single forward pass.

This works especially well for tasks like text editing, summarization (where the output may quote the input), or regenerating a cached response with minor modifications. The acceptance rate can be very high – often above 0.95 for close matches – making this one of the most efficient speculative decoding variants when it applies.

Speculative speculative decoding

Standard speculative decoding has a subtle inefficiency: the draft model must wait for the target model to complete verification before it can start drafting the next batch of tokens. This is because the draft model needs to know which tokens were accepted so it can condition on the correct prefix.

Speculative speculative decoding (Kumar et al. 2026) eliminates this dependency. The draft model speculatively starts generating the next batch of candidates *before* the target model finishes verifying the current batch. It does this by assuming all current draft tokens will be accepted and conditioning on that optimistic prefix. If some tokens are rejected, the speculative draft work is discarded and redone – but when acceptance rates are high, the speculation pays off, and the draft and target models can operate in a pipelined fashion with better hardware utilization.

When speculative decoding helps vs. hurts

Speculative decoding is not a universal win. Several factors determine whether it helps:

Temperature: at low temperatures (greedy or near-greedy sampling), the draft model's predictions are more likely to match the target model, giving higher acceptance rates. At high temperatures, the target model's distribution is more spread out, making it harder for the draft to guess correctly. Speculative decoding works best for deterministic tasks like code generation and factual Q&A.

Draft model quality: a better draft model means higher acceptance rates and more tokens per cycle. But better draft models are also larger and slower, which cuts into the speedup. There's a sweet spot where the draft model is good enough to maintain high acceptance rates but small enough to be much faster than the target.

Batch size: this is the most important practical consideration. Speculative decoding's benefit comes from replacing sequential target-model forward passes with one parallel verification pass. But in high-throughput serving with large batch sizes, the target model's forward passes are already efficient – the GPU is well-utilized because it's processing many requests at

once. Adding draft model overhead on top of an already-efficient pipeline can actually hurt throughput. Speculative decoding shines in low-batch-size, latency-sensitive scenarios.

Multi-token prediction

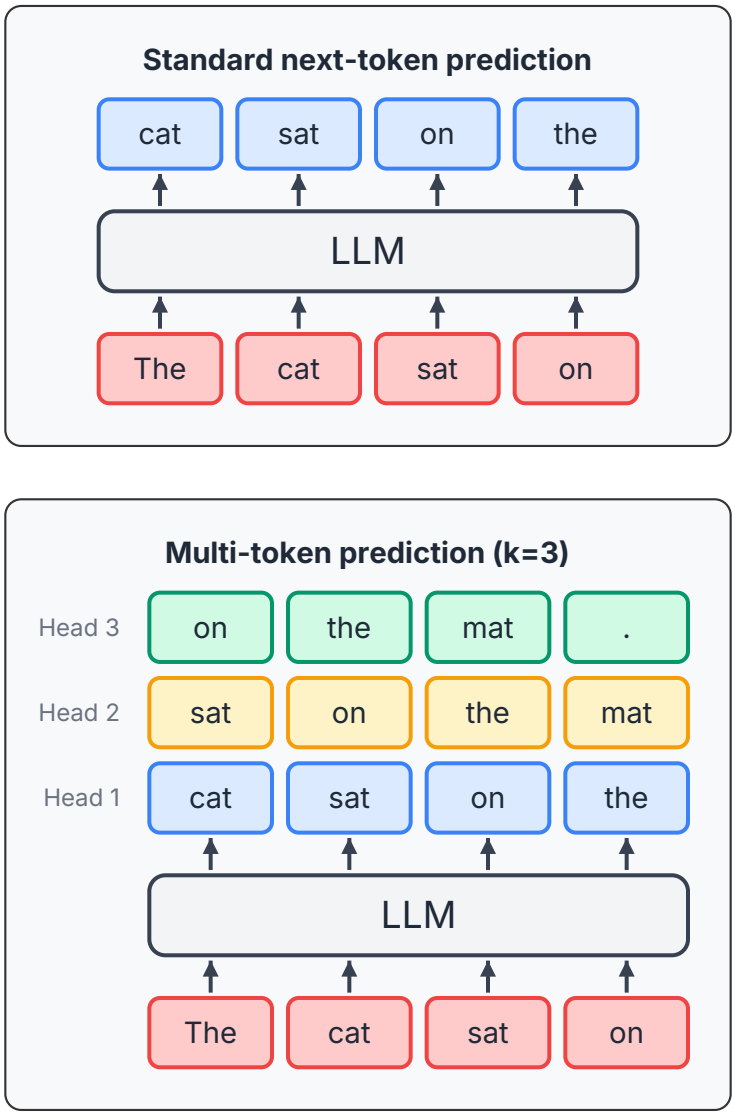


Figure 6.7: Multi-token prediction with $k = 3$ predictions for each token position

Multi-token prediction (MTP) (Gloeckle et al. 2024) modifies the model architecture to predict multiple future tokens at each position. In addition to the standard next-token prediction head, the model has auxiliary heads that predict the token 2, 3, or k steps ahead.

During training, all heads are trained simultaneously with a shared loss. A comparison of standard decoding and multi-token prediction is shown in Figure 6.7.

At inference time, the auxiliary heads generate candidate tokens for positions beyond the next token. These candidates are then verified – either through additional forward passes or by treating them as draft tokens in a speculative decoding framework. When the candidates are accepted, the model has effectively generated multiple tokens from a single forward pass through the main body of the network.

DeepSeek-V3 uses MTP with two prediction heads, and Meta's Llama architecture has explored similar approaches. The key advantage of MTP over external draft models (covered below) is that the draft candidates come from the model itself, so they tend to have high acceptance rates. The cost is a modest increase in model size and training complexity from the auxiliary heads.

6.6 Parallel Decoding

Speculative decoding uses a separate draft model (or separate heads) to propose candidates. **Parallel decoding** methods take a different approach: they generate multiple future tokens using the base model itself, without any auxiliary model.

Lookahead decoding

Lookahead decoding (Fu et al. 2024) reformulates autoregressive generation as a fixed-point problem. Instead of generating tokens one at a time, it maintains a window of **W** future token positions, initialized with guesses. Then it iteratively refines all positions in parallel using **Jacobi iteration**: at each step, every position computes what its token should be given the current values of all other positions. When the sequence converges – that is, when a contiguous run of positions doesn't change between iterations – those tokens are accepted.

The intuition is that many tokens in a sequence are relatively predictable given their context. Common phrases, syntactic patterns, and function words often converge quickly. The tokens that take many iterations to converge are the "surprising" ones – the genuinely creative or informative tokens.

Lookahead decoding requires no draft model, no extra parameters, and no changes to the base model. The tradeoff is that each Jacobi iteration involves a forward pass over **W** token positions, which is more expensive than a single-token decode step. The method wins when enough tokens converge quickly to offset the cost of processing the wider window.

Medusa

Medusa (Cai et al. 2024) adds multiple lightweight **decoding heads** to the base model. The standard model has one head that predicts the next token. Medusa adds k additional heads, where head i predicts the token $i+1$ positions ahead. These heads are small (typically just a single linear layer or a small MLP) and are trained on top of the frozen base model.

At inference time, all $k+1$ heads produce predictions simultaneously from the same hidden state. The candidates from all heads are organized into a tree structure and verified using the tree-based attention approach described in Section 6.5. The longest accepted path through the tree gives the output.

Medusa has a practical advantage over speculative decoding with a separate draft model: since the heads share the base model's forward pass, there's no need to run a separate model. The additional compute for the lightweight heads is small compared to the main model's forward pass. The disadvantage is that the heads need to be trained for each specific base model, adding a fine-tuning step.

Tree-based verification as a unifying framework

Looking across the techniques in these last two sections, a common pattern emerges. Multi-token prediction, speculative decoding with a draft tree, Medusa, and even lookahead decoding all follow the same high-level structure:

1. **Propose**: generate multiple candidate tokens using some cheap mechanism (draft model, auxiliary heads, Jacobi iteration, or reference text)
2. **Organize**: arrange candidates into a tree (or chain) of possible continuations
3. **Verify**: run the target model on the full tree in a single forward pass, using the causal mask to check all candidates in parallel
4. **Accept**: take the longest prefix (or longest path through the tree) where the target model agrees with the proposals

The differences between techniques lie entirely in step 1 – how candidates are generated. The verification and acceptance steps are the same across all methods. This unifying view makes it easier to reason about the tradeoffs: the quality and cost of the proposal mechanism determine the acceptance rate and overhead, while the verification step is always a single forward pass whose cost depends on the total number of candidate tokens.

This also explains why these techniques interact with batch size the same way. The verification step is essentially a small prefill – it processes multiple tokens in parallel. At large batch sizes, the GPU is already busy with many requests, so the marginal cost of the extra verification tokens is higher relative to the benefit. At small batch sizes, the GPU has spare capacity, and the extra tokens come nearly for free.

The techniques in this chapter operate at the level of individual requests – optimizing attention kernels, managing KV cache memory, and breaking the sequential decoding constraint. In Chapter 7, we'll move up a level and consider what happens when a single GPU isn't enough: how to distribute the model across multiple devices while keeping communication overhead manageable.

6.7 Further Reading

Attention kernels. The FlashAttention series of papers (Dao et al. 2022; Dao 2023; Shah et al. 2024) is essential reading for understanding how IO-aware algorithm design can outperform naive implementations even when the naive version does fewer FLOPs. The original FlashAttention paper is notable for its clear presentation of the tiling strategy and the online softmax trick. FlashAttention-2 (Dao 2023) improves parallelism across the sequence dimension and reduces non-matmul FLOPs. FlashAttention-3 (Shah et al. 2024) targets Hopper-generation GPUs specifically, exploiting asynchronous memory transfers and FP8 computation. Tri Dao's blog posts accompanying these papers are also worth reading for accessible explanations of the algorithms.

KV cache management. The PagedAttention paper (Kwon et al. 2023) is the key reference for understanding KV cache fragmentation and the virtual-memory solution. The paper's insight – that contiguous pre-allocation wastes 60-80% of KV cache memory in practice – motivates the entire paged approach. For KV cache compression through token eviction, Keyformer (Adnan et al. 2024) introduces score-based key token selection, and NSA (Yuan et al. 2025) combines sparse attention with compressed inter-block summaries.

Prefix caching. SGLang's RadixAttention (Zheng et al. 2023) generalized prefix caching from simple exact-match lookups to a radix-tree-based system that automatically identifies and shares common prefixes across requests. The paper is worth reading both for the caching mechanism and for its treatment of multi-call LLM programs, which create particularly large prefix-sharing opportunities.

Speculative decoding. The two foundational papers – Leviathan et al. (2022) and C. Chen et al. (2023) – were developed independently and published almost simultaneously. Both prove that speculative decoding preserves the target model's output distribution exactly, which is a surprising and important property. Leviathan et al. provide a cleaner theoretical treatment, while Chen et al. (from DeepMind) include more practical implementation details. For tree-based speculative decoding, Cai et al. (2024) extends the idea with multiple lightweight decoding heads and tree-structured verification. Stern et al. (2018) is an earlier precursor that explored the same idea of draft-then-verify in the context of machine translation. For lookahead decoding as an alternative that requires no draft model, see Fu et al. (2024).

7 Scaling Across Hardware

Everything we've covered so far in this book operates on a single GPU. That works fine for smaller models, but the largest LLMs today — with hundreds of billions of parameters — simply don't fit in the memory of a single device. Even when a model does fit, you may want to spread the work across multiple GPUs. This chapter is about how to do that.

The parallelism techniques used for inference are mostly the same ones developed for training. The good news is that they're easier to implement for inference, because they only have to deal with the forward pass — no activations to store for backpropagation, no gradient calculations, and no optimizer state. The core challenge here is that every time we split work across devices, we introduce communication overhead between devices, and that communication can be much slower than communication within one device.

We'll start by understanding the communication fabric, then work through each parallelism strategy, and finish by analyzing when communication becomes the bottleneck.

7.1 Interconnects and Multi-GPU Topology

Before we can reason about the cost of distributing work, we need to know how fast data moves between devices. The answer varies enormously depending on which devices are talking to each other. Figure 7.1 shows the different communication paths between GPUs in a cluster of two nodes, each with 8 H100 GPUs, and the typical bandwidths for each path.

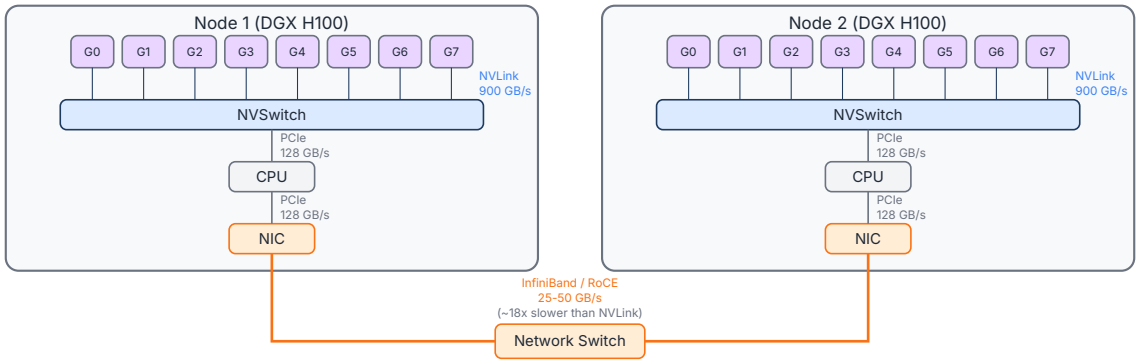


Figure 7.1: Intra-node and inter-node connections

Intra-node: NVLink and NVSwitch

Within a single server node, NVIDIA GPUs communicate over **NVLink**, a high-bandwidth point-to-point interconnect. On the H100, each GPU has 18 NVLink connections providing 900 GB/s of bidirectional bandwidth (NVIDIA 2023b). That's roughly 7x the bandwidth of PCIe Gen5.

In an 8-GPU node like the NVIDIA DGX H100, an **NVSwitch** fabric connects all GPUs in a full-mesh topology, so any GPU can communicate with any other GPU at the full NVLink bandwidth without going through intermediate hops. This is critical for collective operations like all-reduce, where every GPU needs to exchange data with every other GPU.

PCIe

PCIe (Peripheral Component Interconnect Express) is the general-purpose interconnect between the CPU and all attached devices, including GPUs. PCIe Gen5 x16 provides about 128 GB/s of bidirectional bandwidth – roughly 7x slower than NVLink. PCIe is used for CPU-GPU data transfers (loading input tokens, returning results) and as a fallback communication path between GPUs when NVLink isn't available.

For inference, PCIe bandwidth matters in two scenarios: KV cache offloading to CPU memory (discussed in Chapter 5), and consumer/workstation GPU setups that lack NVLink entirely. In those configurations, inter-GPU communication is bottlenecked at PCIe speeds, which severely limits the parallelism strategies that are practical.

Multi-node networking

When you need more GPUs than fit in a single node, communication moves to the network. The two dominant technologies are **InfiniBand** and **RDMA over Converged Ethernet (RoCE)**.

NVIDIA's HDR InfiniBand provides 200 Gb/s (25 GB/s) per port, and NDR doubles that to 400 Gb/s (50 GB/s) per port. With multiple ports bonded together, a node can achieve aggregate network bandwidth in the hundreds of GB/s – but this is still far below the 900 GB/s available over NVLink within the node. Network latency is also significantly higher: a few microseconds for InfiniBand versus roughly 100 nanoseconds for NVLink.

This bandwidth gap is the fundamental reason why multi-node inference is hard. Strategies that require frequent all-to-all communication – like tensor parallelism – work beautifully within a node over NVLink but become impractical across nodes.

Bandwidth comparison

Table 7.1: Interconnect bandwidth and latency comparison for common GPU infrastructure.

Interconnect	Bandwidth (bidirectional)	Typical latency	Scope
NVLink (H100)	900 GB/s per GPU	~100 ns	Intra-node
PCIe Gen5 x16	128 GB/s	~1 s	Intra-node
InfiniBand NDR	50 GB/s per port	~1-2 s	Multi-node
InfiniBand HDR	25 GB/s per port	~1-2 s	Multi-node
RoCE v2	25-50 GB/s per port	~2-5 s	Multi-node

Beyond NVIDIA

While this book focuses on NVIDIA GPUs, it's worth noting the competitive landscape. **AMD's MI300X** offers 192 GB of HBM3 memory – significantly more than the H100's 80 GB – connected via AMD's Infinity Fabric with bandwidth comparable to NVLink. **Google's TPU v4 and v5** use a custom 3D torus interconnect (ICI) that provides high-bandwidth, low-latency communication between chips without needing an explicit switch fabric. **AWS Inferentia** and **Trainium** chips use NeuronLink for inter-chip communication. **Cerebras** takes a radically different approach with wafer-scale integration, eliminating inter-chip communication entirely within a single wafer. Each of these platforms has different communication characteristics, but the fundamental tradeoffs between computation, memory, and communication apply universally.

7.2 Parallelism Overview and Data Parallelism

There are several ways to distribute inference across multiple devices. Each strategy partitions a different dimension of the problem:

- **Data parallelism (DP)**: replicate the entire model on each device; split requests across replicas
- **Tensor parallelism (TP)**: split individual layers across devices
- **Pipeline parallelism (PP)**: assign different layers to different devices
- **Expert parallelism (EP)**: assign different MoE experts to different devices
- **Sequence/context parallelism (SP/CP)**: split the sequence dimension across devices

In practice, production systems combine multiple strategies. A common pattern is TP within a node (where NVLink provides the bandwidth for frequent all-reduce operations) and PP across nodes (where only point-to-point activations need to cross the network).

Data Parallelism

Data parallelism is the simplest approach: place a complete copy of the model on each GPU and route different requests to different replicas. There's no communication between replicas during inference – each replica processes its requests independently.

DP scales throughput linearly with the number of replicas. If one GPU can serve 50 tokens per second, eight GPUs can serve 400 tokens per second. However, DP does nothing for single-request latency, and it requires each device to have enough memory to hold the full model. For a 70B parameter model in FP16, that's 140 GB of weights alone – already beyond the capacity of a single H100 (80 GB HBM3).

When the model fits on a single device, DP is the go-to strategy for scaling throughput. When it doesn't, we need the techniques that follow.

7.3 Tensor Parallelism (TP)

Tensor parallelism splits individual weight matrices across devices, so each GPU computes a slice of every layer. This is the most communication-intensive parallelism strategy, but it directly reduces per-device memory and enables serving models that don't fit on a single GPU.

Column-parallel and row-parallel linear layers

The core idea is to partition each linear layer's weight matrix along either its columns or its rows.

In a **column-parallel** linear layer, the weight matrix W is split along the output dimension. If we have **T** GPUs, each GPU holds W_i with shape $[d_{in}, d_{out}/T]$. Each GPU receives the full input activation x and computes xW_i locally, producing a partial output of shape $[d_{out}/T]$. No communication is needed yet – each GPU has an independent slice of the result.

In a **row-parallel** linear layer, the weight matrix is split along the input dimension. Each GPU holds W_i with shape $[d_{in}/T, d_{out}]$ and receives only its slice of the input x_i. Each GPU computes x_iW_i, producing a partial result of the full output dimension. To get the correct output, we need an **all-reduce** operation to sum the partial results across all GPUs.

The standard pattern is to pair column-parallel and row-parallel layers so that only one all-reduce is needed per pair. In a transformer block:

1. The QKV projection and the MLP up-projection use column-parallel splits
2. The attention output projection and the MLP down-projection use row-parallel splits
3. An all-reduce follows each row-parallel layer

This means each transformer layer requires **two all-reduce operations** – one after the attention block and one after the MLP block.

Gated MLP under TP

We start with the simpler FFN block and show attention next. Figure 7.2 shows what 2-way tensor parallelism looks like for the gated MLP, which is now the more common form of FFN.

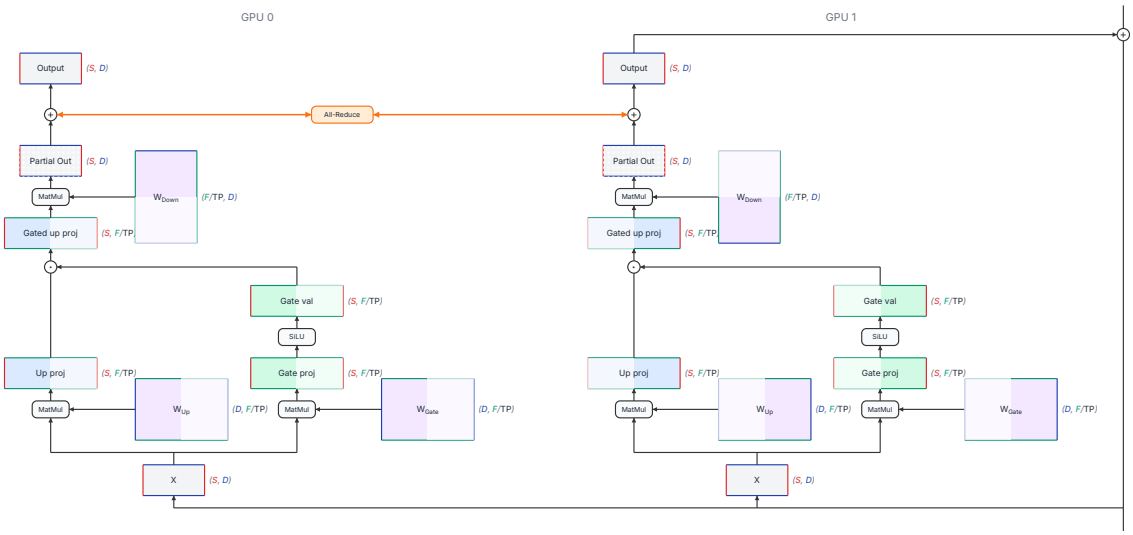


Figure 7.2: Tensor parallel MLP calculation

Multi-head attention under TP

Attention is naturally suited to tensor parallelism because the heads are independent. With **H** attention heads split across **T** GPUs, each GPU handles **H**/**T** heads. The Q, K, and V projection matrices are split column-parallel by head, each GPU computes attention for its local heads, and the output projection is row-parallel with an all-reduce to combine results. This effect is depicted in Figure 7.3 with 2-way tensor parallelism for the attention layer.

KV cache is also sharded: each GPU only stores the K and V tensors for its local heads. With GQA or MQA (discussed in Section 4.4), there are fewer KV heads than query heads, so the KV cache per device is even smaller. For example, with 8 KV head groups split across 8 GPUs with TP=8, each GPU holds just one KV head group – the minimum possible.

Communication cost

The all-reduce operation sums tensors across all **T** GPUs. Using the ring all-reduce algorithm, the communication volume per all-reduce is approximately:

$$V_{\text{all-reduce}} = 2 \times \frac{T-1}{T} \times D \times \text{bytes_per_element}$$

where the factor of 2 accounts for the reduce-scatter and all-gather phases. With two all-reduces per transformer layer and **L** layers, the total communication per forward pass is 2**L** all-reduce operations.

For a 70B model with **D** = 8192 and TP=8 on H100 NVLink (900 GB/s), each all-reduce for a single-token decode step moves roughly 2 × 8192 × 2 bytes 32 KB. That completes in

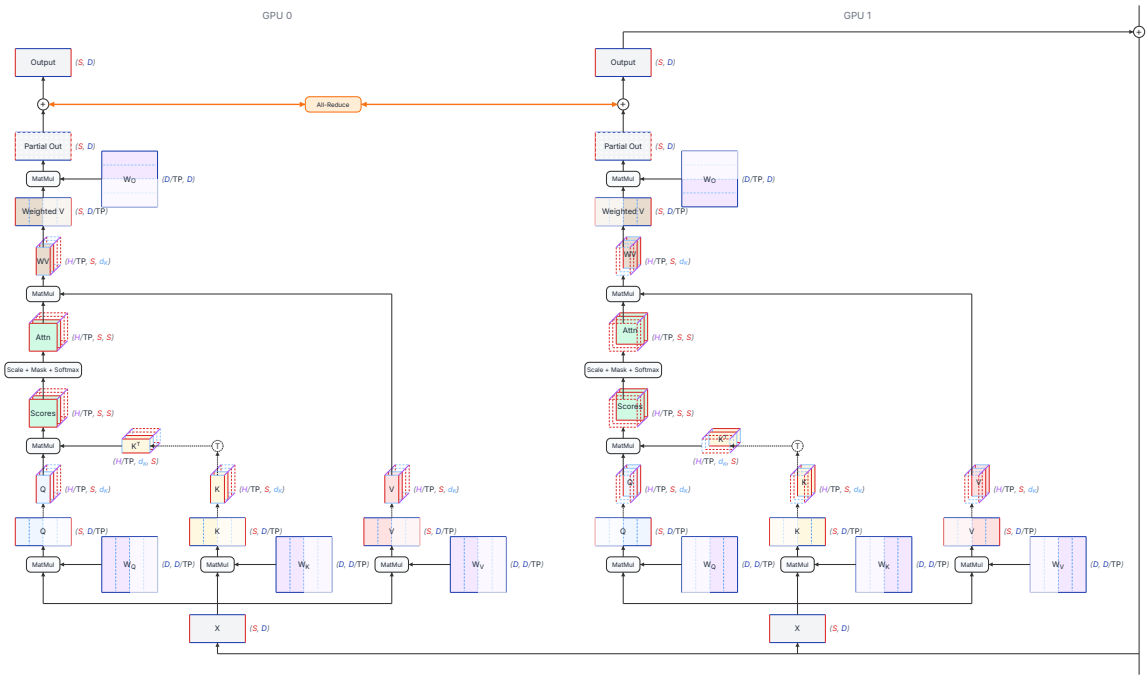


Figure 7.3: Tensor parallel attention calculation

well under a microsecond – negligible. But for prefill with thousands of tokens, or with larger batch sizes, the communication volume scales linearly with b and can start to matter.

The key insight is that TP requires **high-bandwidth, low-latency** communication every layer, making it impractical across nodes where network bandwidth is 10-30x lower than NVLink.

7.4 Pipeline Parallelism (PP)

Pipeline parallelism takes a different approach: instead of splitting each layer across devices, it assigns entire layers to different devices. If you have 80 transformer layers and 4 GPUs, each GPU gets 20 consecutive layers. This is depicted in Figure 7.4.

How it works

The forward pass flows through the pipeline stages sequentially. GPU 0 processes the input through layers 0-19 and sends the activation tensor to GPU 1, which runs layers 20-39, and so on. The communication pattern is simple point-to-point transfers between adjacent stages – no all-reduce needed.

The activation tensor passed between stages has shape **(S, D)**, which is much smaller than the all-reduce traffic in TP. This low communication volume makes PP well-suited for **multi-node** deployment, where network bandwidth is limited.

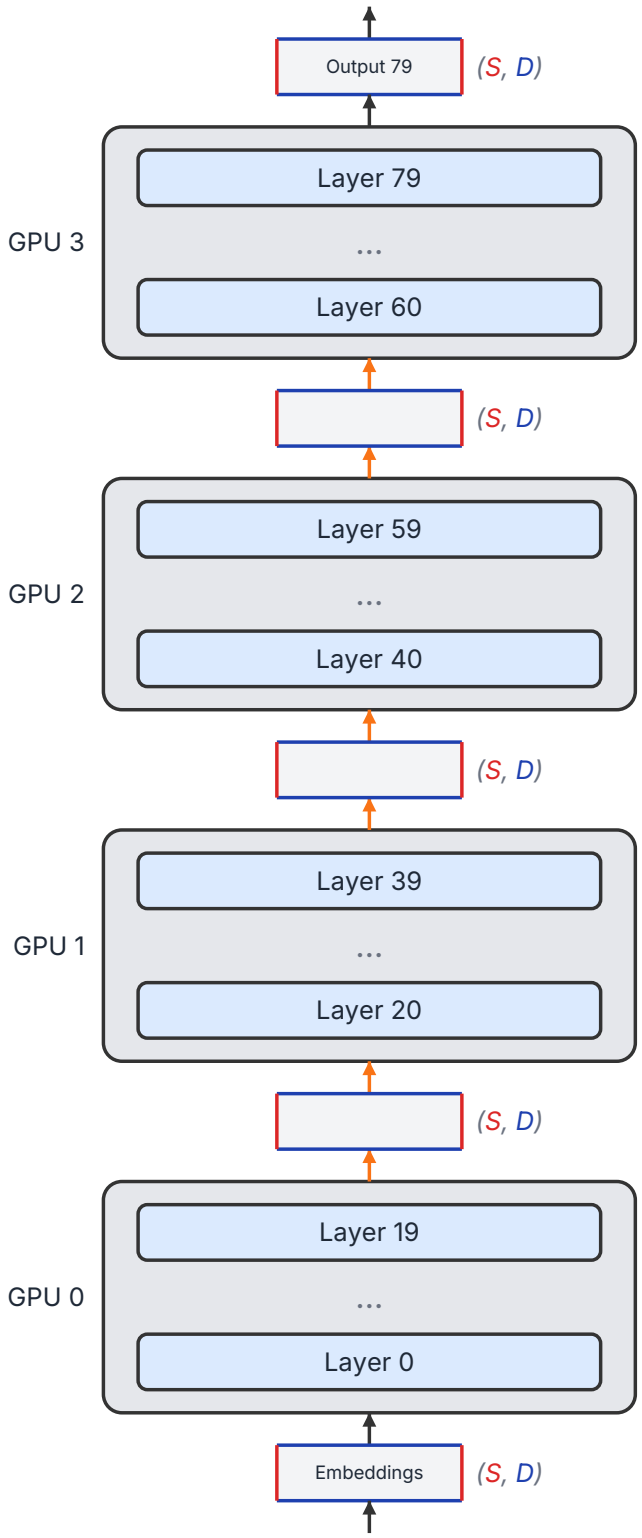


Figure 7.4: Pipeline parallelism of an 80-layer model across 4 GPUs. There is minimal communication between GPUs.

The pipeline bubble

The downside of PP is the **pipeline bubble**. When processing a single request, only one stage is active at a time – the other stages sit idle, waiting for their turn. With **P** pipeline stages, the bubble fraction is roughly $(P - 1)/P$. With 4 stages, 75% of the hardware is idle at any given moment.

Micro-batching is the standard solution. Instead of sending the entire batch through the pipeline, break it into smaller micro-batches. While Stage 1 processes micro-batch 2, Stage 0 can start processing micro-batch 3. With enough micro-batches in flight, all stages stay busy and the bubble shrinks.

For inference, the bubble problem is less severe than in training for two reasons. First, during decode, each step generates just one token per request, so the pipeline latency per step is short. Second, with continuous batching (see Chapter 5), there are usually many concurrent requests to keep the pipeline full.

However, PP does add latency to each decode step – the activation must travel through all stages sequentially. With **P** stages, the minimum latency for a single forward pass is **P** times the per-stage compute time, plus the communication time between stages. For latency-sensitive applications, this overhead can matter.

7.5 Expert Parallelism (EP)

Mixture-of-Experts (MoE) models, like Mixtral, DeepSeek-V2, and many recent large models, replace the dense MLP block with a set of expert MLPs, where a router selects a small number of experts (typically 2) for each token. This dramatically reduces per-token compute, but it means the full set of expert weights must be in memory.

Expert parallelism assigns different experts to different GPUs. With 64 experts and 8 GPUs, each GPU holds 8 experts. When a token is routed to an expert on another GPU, the token's activation must be sent to that GPU and the result sent back.

All-to-all communication

Figure 7.5 shows the data flow for expert parallelism. The communication pattern for EP is **all-to-all**: every GPU potentially needs to send tokens to every other GPU and receive results back. Each MoE layer requires two all-to-all operations – one to dispatch tokens to the correct experts, and one to collect results.

The communication volume depends on the routing decisions, but in expectation, each GPU sends $(T - 1)/T$ of its tokens to other GPUs (since only $1/T$ of experts are local). For a model with many MoE layers – DeepSeek-V3 has MoE in every other transformer block – this all-to-all traffic adds up.

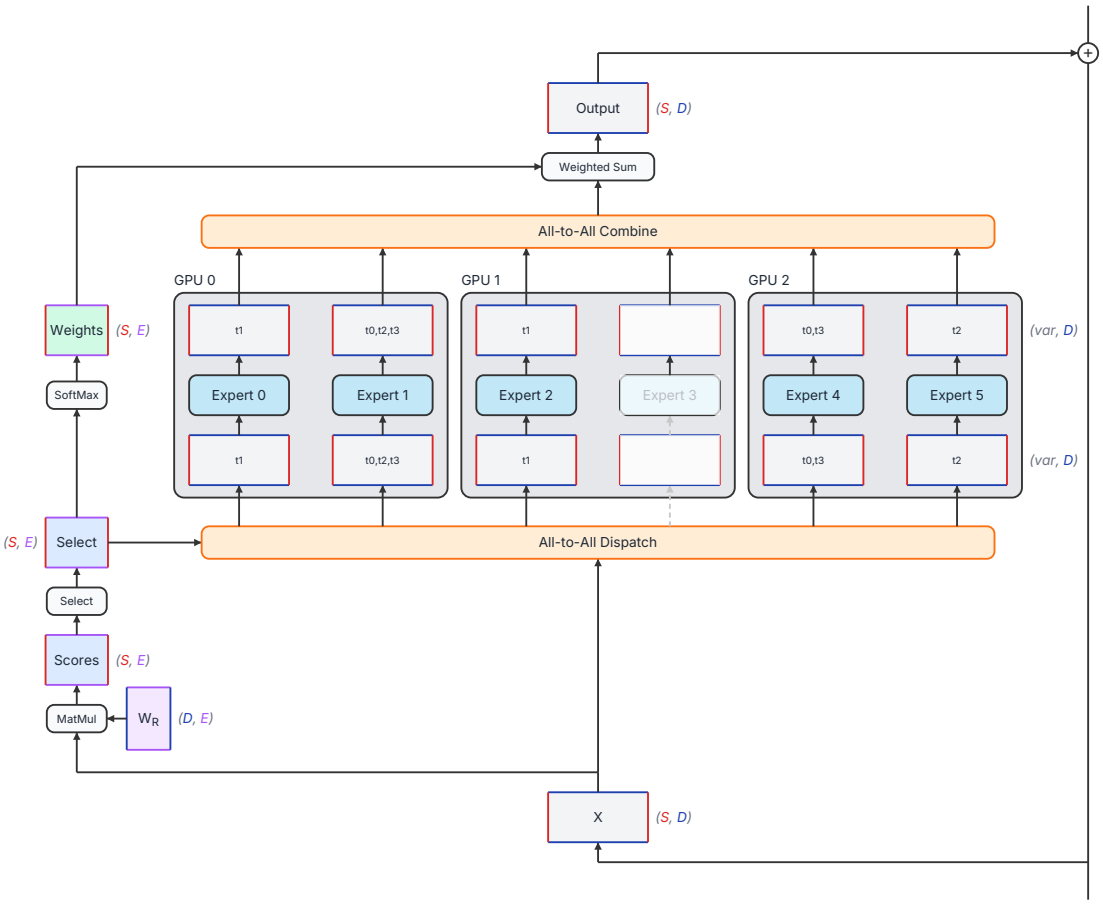


Figure 7.5: Expert parallel MLP calculation

Load balancing

A practical challenge with EP is **load imbalance**. If the router sends most tokens to a few popular experts, the GPUs holding those experts become bottlenecks while others sit idle. Training-time auxiliary losses encourage balanced routing, but inference workloads may still exhibit skewed expert utilization. Some systems address this by placing popular experts on multiple GPUs (expert replication) or by capping the number of tokens each expert processes and dropping overflow tokens.

EP pairs naturally with TP and PP. A common configuration for large MoE models is TP within a node for the attention layers and EP across nodes for the expert layers, since the all-to-all communication pattern of EP is more tolerant of network latency than the all-reduce pattern of TP.

7.6 Context / Sequence Parallelism (CP/SP)

The parallelism strategies we've covered so far partition the model. **Context parallelism** (also called **sequence parallelism**) partitions the input sequence – splitting a long sequence of tokens across multiple GPUs so that each GPU processes a subsequence.

This is primarily useful during **prefill** of long contexts. If you're processing a 128K-token prompt, the attention computation is $O(\mathrm{S}^2)$ and the KV cache for that single request may not fit on one GPU. Splitting the sequence across 4 GPUs gives each one a 32K-token chunk to process.

Ring attention

Ring self-attention (Li et al. 2021) arranges the GPUs in a logical ring. Each GPU starts with a local chunk of queries and an initial chunk of keys and values. Through a series of ring-pass steps, the K/V chunks rotate around the ring, and each GPU accumulates partial attention scores from all chunks. After $\mathbf{T}$ steps (where $\mathbf{T}$ is the number of GPUs), every GPU has computed attention over the full sequence. This pattern is depicted in Figure 7.6 for $\mathbf{T} =$ 4 GPUs.

The ring communication pattern overlaps neatly with computation: while a GPU computes attention with the current K/V chunk, it simultaneously sends that chunk to its neighbor and receives the next chunk. With enough computation per chunk, the communication is fully hidden.

DistFlashAttn (D. Li et al. 2023) combines ring attention with FlashAttention's IO-aware tiling (see Section 6.1), getting both the distributed memory benefit and the HBM traffic reduction of FlashAttention.

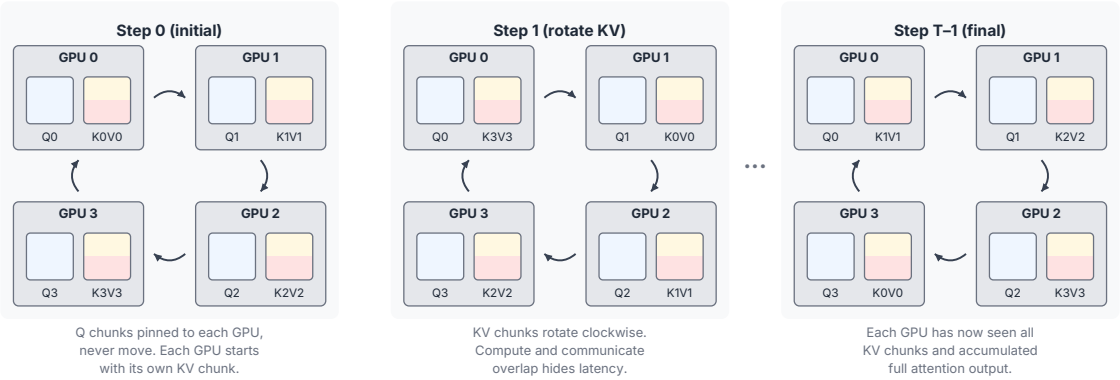


Figure 7.6: Ring attention calculation

NVIDIA sequence parallelism

NVIDIA's variant of **sequence parallelism** is designed to work alongside tensor parallelism. In a TP-sharded model, operations that are not parallelized by TP – like LayerNorm and dropout – are replicated across all GPUs, which wastes memory. NVIDIA SP partitions the sequence dimension for these operations, so each GPU handles a fraction of the sequence during LayerNorm and other non-TP layers, then all-gathers the full sequence before the TP-parallel matrix multiplications.

DeepSpeed Ulysses

DeepSpeed Ulysses (Jacobs et al. 2023) takes a different approach. Instead of passing K/V chunks around a ring, it uses all-to-all communication to transpose the Q, K, and V tensors from a sequence-partitioned layout to a head-partitioned layout. Each GPU then computes full-sequence attention for its subset of heads – equivalent to tensor parallelism's head-based split, but derived from a sequence-partitioned starting point. After attention, another all-to-all transposes back.

The advantage of this approach is that each GPU computes standard FlashAttention on its local heads without the complexity of incremental partial-attention accumulation. The cost is two all-to-all operations per attention layer.

FlashDecoding

FlashDecoding is worth mentioning here even though it operates within a single GPU. It parallelizes the attention computation across the KV sequence dimension – splitting the cached keys and values across multiple thread blocks that compute partial attention in parallel, then reducing the results. This addresses the problem that during decode, the single query token produces very little parallelism across the head dimension, leaving most of the GPU idle.

FlashDecoding gives the GPU more work to do in parallel by splitting the "long" dimension – the cached sequence.

When sequence parallelism helps

Context parallelism is most effective for **long-context prefill**, where the sequence length creates both a compute and memory bottleneck. For short-context decode – the common case for most serving workloads – the sequence dimension is short and there's little benefit to splitting it further. The overhead of the extra communication typically outweighs any gains.

7.7 Communication Cost Analysis

With all the parallelism strategies laid out, let's compare their communication patterns and costs.

Table 7.2: Communication patterns and costs for each parallelism strategy. **T** = number of devices, d = model dimension, b = batch size in tokens, S = sequence length.

Strategy	Collective op	Ops per layer	Communication volume per op	Best interconnect
TP	All-reduce	2	$2 \times \frac{T-1}{T} \times d \times b$ bytes	NVLink
PP	Point-to-point	1	$d \times b$ bytes (activation)	Network OK
EP	All-to-all	2	Routing-dependent; $\sim\frac{T-1}{T}$ of tokens	NVLink or network
CP/SP	Ring-pass or all-to-all	**T** or 2	$O(S/T \times d)$ per step	NVLink

The key observation is that **TP is the most communication-hungry** strategy, requiring two all-reduce operations per layer. This is why TP is confined to within a single node over NVLink. PP has the lightest communication – just a single activation tensor between adjacent stages – making it the natural choice for spanning nodes.

Combined TP + PP

The most common multi-node configuration is **TP within a node, PP across nodes**. For example, serving a 70B model on 2 nodes of 8 GPUs each: use TP=8 within each node (splitting layers across all 8 GPUs via NVLink), and PP=2 across the two nodes (first half of layers on node 1, second half on node 2). The frequent all-reduces stay on the fast NVLink fabric, and only the occasional activation transfer crosses the network.

Communication-bound regime

When does communication become the bottleneck? During decode with small batch sizes, the per-layer computation is tiny (a few matrix-vector multiplications), and each layer's all-reduce – even if it's only transferring tens of KB – has a latency floor that can dominate. The all-reduce latency depends on both bandwidth and the number of sequential steps in the algorithm.

A useful rule of thumb: if the communication time per layer exceeds the compute time per layer, you've over-parallelized. Adding more TP devices beyond this point will increase latency rather than decrease it, because you're spending more time synchronizing than computing. This is why TP=8 is a common sweet spot on 8-GPU nodes – it matches the NVLink connectivity – and going to TP=16 across nodes rarely makes sense.

7.8 Multi-Node Inference

When a model is too large for a single node – or when you need more throughput than one node can provide – you're in multi-node territory. This introduces the network as a hard constraint on performance.

Network latency as a floor

During decode, each token generation requires a full forward pass through the model. If that forward pass spans two nodes connected by InfiniBand, each pipeline stage boundary (or TP all-reduce, if you're running TP across nodes) adds network latency. InfiniBand NDR round-trip latency is roughly 2-5 s, compared to ~100 ns for NVLink. This latency is per layer for TP or per stage for PP, and it accumulates across the forward pass.

For a model using PP=2 across nodes, the activation transfer between stages adds a few microseconds per decode step. That's usually acceptable. But for TP across nodes, where you need two all-reduces per layer across potentially 80+ layers, the cumulative latency can add milliseconds to each token, directly increasing TPOT (see Section 3.2).

Practical deployment patterns

In practice, multi-node inference for large models follows a few common patterns:

- **TP within node, PP across nodes**: the standard approach, as discussed above. Minimizes cross-node communication.
- **TP within node, EP across nodes for MoE**: expert parallelism's all-to-all pattern is more latency-tolerant than TP's all-reduce, making it more viable across nodes.
- **DP across nodes**: when the model fits on a single node, the simplest scale-out is running independent replicas. No cross-node communication during inference at all.

> **Note**
>
> The distinction between what's practical within a node versus across nodes is driven almost entirely by the bandwidth gap in Table 7.1. NVLink provides 900 GB/s; the network provides 50 GB/s. That 18x gap is why the parallelism strategy changes at the node boundary.

Memory and KV cache considerations

One often-overlooked benefit of distributing a model across more GPUs is that it frees up memory for the **KV cache**. If a 70B FP16 model uses 140 GB across 8 GPUs (17.5 GB per GPU), that leaves about 62 GB per GPU on H100 for KV cache and activations. With quantized weights (e.g., INT4), the model only consumes ~35 GB total, leaving even more room. More KV cache memory means you can serve larger batches or longer sequences, directly improving throughput.

This is a genuine advantage of tensor parallelism beyond just enabling large models: by spreading weight memory across more devices, you increase the aggregate memory available for serving. Combined with KV cache sharding from GQA/MQA models (Section 4.4) and PagedAttention (Section 6.3), multi-GPU setups can serve substantially more concurrent requests than the raw parameter count might suggest.

The next chapter (Chapter 8) will discuss how all of these techniques come together in production serving systems, including how frameworks like vLLM and TensorRT-LLM configure parallelism strategies automatically based on available hardware.

7.9 Further Reading

Tensor parallelism. Megatron-LM (Shoeybi et al. 2019) is the foundational paper for tensor parallelism in transformers. It introduces the column-parallel and row-parallel linear layer decompositions used by essentially every inference framework today. The follow-up paper (Narayanan et al. 2021) extends the analysis to combined tensor and pipeline parallelism, with a clear treatment of how communication costs scale with model size and parallelism degree. Both papers are written from a training perspective, but the forward-pass partitioning is identical for inference.

Pipeline parallelism. GPipe (Huang et al. 2019) introduced micro-batching to reduce the pipeline bubble. While the original context is training, the concept of splitting a model into stages across devices and using micro-batches to keep all stages busy applies directly to inference. For inference specifically, the pipeline parallelism discussion in Pope et al. (2022) provides a good analysis of how bubble fraction and inter-stage latency affect throughput and per-token latency.

Sequence and context parallelism. Ring attention (Li et al. 2021) and DistFlashAttn (D. Li et al. 2023) extend the FlashAttention tiling strategy across multiple devices, enabling

attention over sequences that are too long to fit in a single GPU's memory. DeepSpeed Ulysses (Jacobs et al. 2023) takes a different approach, partitioning along the head dimension rather than the sequence dimension. Korthikanti et al. (2022) covers NVIDIA's sequence parallelism technique, which partitions the non-tensor-parallel regions (LayerNorm, dropout) across devices to reduce activation memory.

Multi-node inference. The NVIDIA DGX H100 architecture whitepaper (NVIDIA 2023a) is useful for understanding the topology and bandwidth hierarchy of a modern multi-GPU node. For multi-node deployment patterns, the AlpaServe paper (Z. Li et al. 2023) explores how to automatically determine parallelism strategies across heterogeneous clusters, balancing latency constraints against hardware utilization.

8 Production LLM Serving Systems

Now that we understand LLM inference and many of the optimization techniques used, we can discuss some of the popular model serving frameworks and the implementation choices they have made for serving LLMs in the real world. Some of these systems are in production across many continents, scaling to thousands of concurrent users or more. This is not a usage tutorial for any particular framework. The goal is to understand which optimization combinations each system makes, why those choices matter, and what operational concerns arise once you move from benchmarking a single model on a single GPU to serving real traffic.

8.1 Serving Frameworks

The open-source LLM serving landscape has converged around a handful of frameworks, each with a different philosophy about what to optimize for. All of them implement continuous batching (Section 5.1), support tensor parallelism (Section 7.3), and integrate FlashAttention (Section 6.1). Where they differ is in their core innovation and the tradeoffs they make.

vLLM

vLLM is built around PagedAttention (Kwon et al. 2023), the virtual-memory-inspired KV cache management system we covered in Section 6.3. By breaking the KV cache into fixed-size pages that don't need to be physically contiguous, vLLM nearly eliminates the memory fragmentation that plagued earlier systems. This is what allows it to pack more concurrent requests into the same GPU memory budget.

Beyond PagedAttention, vLLM implements continuous batching, chunked prefill (Section 5.2), prefix caching (Section 6.4), speculative decoding (Section 6.5), and multi-LoRA serving. It supports NVIDIA, AMD, and other hardware backends, making it one of the most broadly portable options. vLLM has become the default choice for many deployments, largely because of its active open-source community and broad model support.

TensorRT-LLM

TensorRT-LLM takes a different approach. Rather than optimizing memory management at the Python level, it focuses on squeezing maximum throughput from NVIDIA hardware through aggressive kernel fusion and compilation (Section 6.2). Models are compiled into

optimized execution plans that fuse operations across layers, eliminate unnecessary memory traffic, and exploit NVIDIA-specific hardware features like FP8 on Hopper GPUs.

The tradeoff is narrower hardware support – TensorRT-LLM runs only on NVIDIA GPUs – and a heavier compilation step before serving begins. But on NVIDIA hardware, it often achieves the highest raw throughput, particularly for large-batch, throughput-oriented workloads. It implements the same core techniques (continuous batching, paged KV cache, tensor parallelism) but through a compiled execution path rather than an interpreted one.

SGLang

SGLang (Zheng et al. 2023) innovates in two directions. First, its **RadixAttention** system extends prefix caching beyond simple exact-match lookups. It organizes the KV cache as a radix tree keyed by token sequences, enabling automatic, fine-grained prefix sharing across requests (Section 6.4). When many requests share system prompts, few-shot examples, or RAG context, this can dramatically reduce redundant prefill computation.

Second, SGLang provides a programming model for **multi-call LLM programs** – workflows where a single user interaction involves multiple LLM calls with branching logic, structured output constraints, or iterative refinement. By co-designing the serving engine with this programming model, SGLang can optimize across calls in ways that a stateless API cannot.

DeepSpeed-Inference

DeepSpeed-Inference (Aminabadi et al. 2022) focuses on scaling to very large models across many devices and nodes. Its ZeRO-Inference system distributes model weights across GPUs and even to NVMe storage, enabling inference on models that don't fit in aggregate GPU memory. This makes it particularly useful for multi-node inference and extremely large models.

DeepSpeed-Inference also implements custom CUDA kernels for fused operations, but its primary strength is the multi-GPU and multi-node orchestration layer rather than single-device kernel optimization.

Framework comparison

The following table summarizes the key differences across these frameworks. Keep in mind that all four are actively developed, so specific feature gaps tend to close over time.

Table 8.1: Framework comparison across major open-source LLM serving systems.

Feature	vLLM	TensorRT-LLM	SGLang	DeepSpeed-Inference
Core innovation	PagedAttention	Kernel fusion / compilation	RadixAttention + structured gen	ZeRO-Inference / NVMe offload

Feature	vLLM	TensorRT-LLM	SGLang	DeepSpeed-Inference
KV cache management	Paged (virtual memory)	Paged	Radix tree	Contiguous + offload
Continuous batching	Yes	Yes	Yes	Yes
Speculative decoding	Yes	Yes	Yes	Limited
Multi-LoRA	Yes	Yes	Yes	No
Hardware support	NVIDIA, AMD, others	NVIDIA only	NVIDIA, AMD	NVIDIA
Best suited for	General-purpose serving	Max throughput on NVIDIA	Prefix-heavy / multi-call	Very large models, multi-node

Note

CPU and hybrid inference engines like llama.cpp and Apple MLX serve an important role for local and edge deployment, but they operate under fundamentally different constraints – limited memory bandwidth, no HBM, CPU-only or unified-memory architectures. Closed-source serving systems from cloud providers are also out of scope. This chapter focuses on GPU-based serving at scale.

8.2 Multi-LoRA Serving

In many production deployments, you don't just serve one model – you serve many fine-tuned variants of the same base model. Different customers, tasks, or use cases each have their own adapter, and loading a separate full model for each one is wildly impractical. **Multi-LoRA serving** solves this by keeping a single copy of the base model weights in GPU memory and hot-swapping lightweight LoRA adapters at request time.

A LoRA adapter modifies the base model by adding low-rank update matrices to specific layers – typically the attention projections. These adapters are small, often just tens of megabytes compared to the base model's tens of gigabytes. This size difference is what makes the whole approach work: you can keep the base model loaded and swap adapters in and out without the memory cost of maintaining multiple full model copies.

The serving framework needs to handle three things for multi-LoRA to work well:

1. **Adapter routing**: Each incoming request specifies which adapter to use. The system must route the request to the correct adapter and apply it during the forward pass.
2. **Adapter memory management**: Popular adapters can be kept resident in GPU memory. Less frequently used adapters are loaded on demand from CPU memory or

storage. The system needs an eviction policy – typically LRU – to manage which adapters are resident.

3. **Batching across adapters**: Requests using different adapters can still be batched together for the base model computation. The adapter-specific computation (the low-rank matrix multiplies) is applied per-request. This means you get most of the batching benefits of shared base weights while still serving personalized models.

Both vLLM and SGLang support multi-LoRA serving, with adapter management integrated into their scheduling and memory systems.

8.3 Scheduling and Orchestration

The scheduling strategies from Chapter 5 operate at the level of a single serving instance – one model on one set of GPUs. In production, you often need to coordinate across a fleet of instances, potentially running different models on different hardware.

Multi-model routing

Many applications route requests to models of different sizes based on the task complexity. Simple classification tasks go to a smaller, cheaper model. Complex reasoning tasks go to a larger, more capable model. This **quality-tier routing** can significantly reduce cost while maintaining quality where it matters, but it requires a routing layer that can classify incoming requests and direct them appropriately.

Load balancing across replicas

When you scale out with data parallelism – running multiple identical copies of the same model – you need a load balancer that understands LLM workload characteristics. Naive round-robin doesn't work well because requests vary enormously in cost. A request with a 10,000-token prompt and 2,000-token output takes orders of magnitude more resources than a 100-token prompt with a 50-token output. Load balancers that account for estimated request cost (based on input length and expected output length) distribute work more evenly.

Disaggregated prefill at fleet level

In Section 5.2, we discussed disaggregating prefill and decode to separate GPU pools (Zhong et al. 2024). At fleet scale, this becomes an orchestration problem: how many prefill instances versus decode instances should you run, and how do you transfer KV caches between them? The optimal ratio depends on your workload's input/output length distribution (Section 3.3). Prefill-heavy workloads (long prompts, short outputs) need more prefill capacity. Decode-heavy workloads need more decode capacity. Production systems typically monitor queue depths on both pools and auto-scale the ratio.

Heterogeneous hardware

Large deployments often mix GPU generations – perhaps H100s for the most latency-sensitive traffic and A100s or older GPUs for batch workloads. The orchestration layer must route requests based on both the model's requirements and each hardware pool's capabilities and current load.

8.4 Memory Management and Preemption

We covered the metrics around preemption in Section 3.2, and the paged KV cache in Section 6.3. In production, memory management becomes one of the most critical operational concerns, because running out of KV cache space under load can cascade into system-wide degradation.

Multi-tenant memory pressure

When many concurrent requests share the same GPU, the KV cache dominates memory usage. Each active request's cache grows with every decode step, and different requests may have very different sequence lengths. The serving framework must track per-request memory consumption and make admission decisions: can we accept this new request, or will it push us over our memory budget?

PagedAttention helps by eliminating fragmentation – memory is allocated in pages and freed when requests complete. But even with perfect paging, the total KV cache demand can exceed available memory when the system is under heavy load or processing long-context requests.

Preemption policies

When memory runs out, something has to give. The system **preempts** one or more active requests, freeing their KV cache to make room for others. The preempted request either has its KV cache swapped to CPU memory (to be restored later) or is simply evicted and must recompute its KV cache from scratch when it resumes.

The choice between swap and recompute involves a tradeoff:

- **Swap** preserves the computation already done but requires CPU memory and PCIe bandwidth to move the KV cache. For long sequences with large KV caches, the transfer time can be significant.
- **Recompute** avoids the transfer cost but wastes the prefill work. For short sequences, recompute is often cheaper than swap.

Production systems typically use **priority-based preemption** – lower-priority requests are evicted first, and SLA-aware policies protect requests that are close to completing or that belong to higher-priority tiers.

Graceful degradation

Under sustained overload, the goal shifts from maximizing throughput to avoiding cascading failures. **Request shedding** – rejecting new requests at the admission layer when the system is at capacity – is blunt but effective. More sophisticated approaches include dynamically reducing maximum sequence lengths, lowering batch sizes, or routing overflow traffic to a degraded service tier that uses a smaller, faster model.

8.5 Monitoring and Benchmarking

You can't optimize what you don't measure. Production serving requires continuous monitoring of the metrics we introduced in Section 3.2, with particular attention to tail latencies and resource utilization.

Key production metrics

The metrics that matter most in production go beyond averages:

Table 8.2: Key production metrics and their diagnostic value.

Metric	What it tells you	Warning threshold
P99 TTFT	Worst-case user wait for first token	Exceeds SLA
P99 TPOT	Worst-case streaming speed	Users perceive lag
GPU utilization (MFU/MBU)	Hardware efficiency	Below 50% suggests misconfiguration
Queue depth	Request backlog	Sustained growth means under-provisioned
Preemption rate	Memory pressure frequency	Any sustained preemption suggests need for more capacity
KV cache hit rate	Prefix caching effectiveness	Low rate with shared prefixes means caching misconfigured
Goodput	Useful throughput within SLA	Gap between throughput and goodput means SLA violations

Percentile latencies (P50, P90, P99) are essential because averages hide tail behavior. A system can have a perfectly acceptable P50 TTFT of 200ms while its P99 is 5 seconds – meaning one in a hundred users waits 25x longer than the median user. Production SLAs are typically defined at the P99 level.

Instrumentation

Most production deployments use **Prometheus** (Prometheus Authors 2024) for metrics collection and **Grafana** (Grafana Labs 2024) for dashboarding. vLLM, SGLang, and TensorRT-LLM all expose Prometheus-compatible metrics endpoints. For distributed tracing across the full request lifecycle – from the load balancer through the serving engine to response delivery – **OpenTelemetry** (OpenTelemetry Authors 2024) provides a standard instrumentation layer.

Benchmarking

Before deploying a new configuration, you need reproducible benchmarks that reflect your actual workload. Tools like **LLMPerf** generate synthetic traffic with configurable input/output length distributions and concurrency levels (Kadous et al. 2023). The critical mistake in benchmarking is using a uniform workload (e.g., all requests with 512 input tokens and 128 output tokens) when your production traffic has high variance. Benchmark with a distribution that matches production, or you'll be surprised by how differently your system behaves under real load.

8.6 Production Failure Modes

Understanding how systems fail is just as important as understanding how they work. Several failure modes are specific to LLM serving, and recognizing them early can prevent cascading outages.

OOM cascades

This is the most dangerous failure pattern. A single unusually long request consumes a large portion of the KV cache, triggering preemption of other requests. Those preempted requests eventually resume and recompute their KV caches, creating a burst of prefill work that competes with ongoing decode steps. This increases latency for all active requests, causing SLA violations, which may trigger retries from the client, which adds more load. The feedback loop can take down an entire serving instance.

Mitigation: Set hard limits on maximum sequence length per request. Use admission control to reject requests that would push memory usage above a safe threshold. Monitor preemption rate as an early warning signal.

Head-of-line blocking

A request with a very long prompt ties up GPU compute during its prefill phase. If the system uses naive scheduling, shorter requests queue behind it and experience inflated TTFT. This is particularly damaging in mixed-workload deployments where most requests are short but occasional long-context requests arrive.

Mitigation: Chunked prefill (Section 5.2) breaks long prefills into smaller pieces that are interleaved with decode steps, preventing any single prefill from monopolizing the GPU. Priority scheduling (Section 5.4) can also help by ensuring short requests aren't starved.

Thundering herd

A burst of simultaneous requests – perhaps triggered by a traffic spike, a retry storm, or a batch job – overwhelms the KV cache allocator and scheduler. The system attempts to admit all requests at once, runs out of memory, and begins preempting aggressively, which compounds the problem.

Mitigation: Admission control with request queuing and rate limiting. Gradually ramp admitted requests rather than accepting the full burst. Backpressure signals to upstream load balancers allow the fleet to absorb traffic spikes across replicas.

Graceful degradation strategies

When a system is under sustained pressure, you have several levers to pull before things break:

1. **Request shedding**: reject low-priority requests at the edge, returning a "service unavailable" rather than degrading quality for everyone
2. **Dynamic batch policy adjustment**: temporarily reduce maximum batch sizes to lower memory pressure, trading throughput for stability
3. **Quality-tier routing**: redirect overflow traffic to smaller, cheaper models that can handle the load, accepting lower quality to maintain availability
4. **Speculative decoding toggle**: disable speculative decoding under memory pressure, since draft tokens consume KV cache space

The common thread across all of these strategies is that controlled, deliberate degradation is vastly preferable to uncontrolled cascading failure. A system that gracefully sheds 10% of requests under a load spike is far better than one that falls over entirely and drops 100%.

Production LLM serving sits at the intersection of systems engineering and machine learning optimization. The techniques from the preceding chapters provide the building blocks, but assembling them into a reliable, efficient serving system requires understanding the operational realities – memory pressure, failure modes, monitoring, and orchestration – that don't show up in benchmark results. The frameworks surveyed here each represent a different set of answers to these challenges, and the right choice depends on your specific workload, hardware, and reliability requirements.

8.7 Further Reading

Framework documentation. Each of the major frameworks has comprehensive documentation that goes well beyond what we've covered here. The vLLM docs (vLLM Team 2024) include guides on configuring PagedAttention parameters, enabling speculative decoding, and tuning scheduler policies. The TensorRT-LLM docs (NVIDIA 2024b) cover the compilation pipeline and how to exploit FP8 and custom kernels on NVIDIA hardware. SGLang's docs (SGLang Team 2024) are particularly useful for understanding the RadixAttention cache and the structured generation programming model. DeepSpeed-Inference (Microsoft DeepSpeed Team 2024) covers multi-node deployment and NVMe offloading.

Multi-LoRA serving. The original LoRA paper (Hu et al. 2021) explains the low-rank adaptation technique and why the adapter weights are small enough to swap efficiently. For serving multiple LoRA adapters simultaneously, the S-LoRA paper (Sheng et al. 2023) introduces the idea of a unified base model with dynamically loaded adapters, including a custom CUDA kernel for batched LoRA computation across requests using different adapters. Punica (L. Chen et al. 2023) takes a similar approach with a focus on the GPU kernel design for multi-adapter batching.

Benchmarking. If you're evaluating frameworks for a specific deployment, LLMPerf (Anyscale 2024b) provides standardized benchmarking scripts that measure the metrics from Section 3.2 across different frameworks and hardware configurations. For understanding how to interpret benchmark results and avoid common pitfalls, the Anyscale metrics blog post (Kadous et al. 2023) is a practical companion.

System design. For the broader context of how LLM serving fits into production ML systems, Miao et al. (2023) surveys the full stack from model optimization through serving infrastructure. For fleet-level scheduling across multiple model replicas, Wu et al. (2023) covers how to route requests across heterogeneous hardware to maximize goodput under latency constraints.

References

Adnan, Muhammad, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. "Keyformer: KV Cache Reduction Through Key Tokens Selection for Efficient Generative Inference." *MLSys*. https://proceedings.mlsys.org/paper_files/paper/2024/hash/48fecef47b19fe501d27d338b6d52582-Abstract-Conference.html.

Agrawal, Amey, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. "Sarathi: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills." *arXiv Preprint arXiv:2308.16369*. https://arxiv.org/abs/2308.16369.

Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." *arXiv Preprint arXiv:2305.13245*. https://arxiv.org/abs/2305.13245.

Alammar, Jay. 2018. "The Illustrated Transformer." https://jalammar.github.io/illustrated-transformer/.

Aminabadi, Reza Yazdani, Samyam Rajbhandari, Ammar Ahmad Awan, et al. 2022. "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale." *arXiv Preprint arXiv:2207.00032*. https://arxiv.org/abs/2207.00032.

Anyscale. 2024a. "How Continuous Batching Enables 23x Throughput in LLM Inference While Reducing P50 Latency." https://www.anyscale.com/blog/continuous-batching-llm-inference.

Anyscale. 2024b. "LLMPerf: A Tool for Benchmarking LLM Inference." https://github.com/ray-project/llmperf.

Austin, Jacob, Sholto Douglas, Roy Frostig, et al. 2025. "How to Scale Your Model." https://jax-ml.github.io/scaling-book/.

Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. "Layer Normalization." *arXiv Preprint arXiv:1607.06450*. https://arxiv.org/abs/1607.06450.

Beltagy, Iz, Matthew E. Peters, and Arman Cohan. 2020. "Longformer: The Long-Document Transformer." *arXiv Preprint arXiv:2004.05150*. https://arxiv.org/abs/2004.05150.

Bloem, Peter. 2019. "Transformers from Scratch." https://peterbloem.nl/blog/transformers.

Brandon, William, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan Kelly. 2024. "Reducing Transformer Key-Value Cache Size with Cross-Layer Attention." *arXiv Preprint arXiv:2405.12981*. https://arxiv.org/abs/2405.12981.

Cai, Tianle, Yuhong Li, Zhengyang Geng, et al. 2024. "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads." *arXiv Preprint arXiv:2401.10774*. https://arxiv.org/abs/2401.10774.

Chen, Carol. 2022. "Transformer Inference Arithmetic." https://kipp.ly/transformer-inference-arithmetic/.

Chen, Charlie, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. "Accelerating Large Language Model Decoding with Speculative Sampling." *arXiv Preprint arXiv:2302.01318*. https://arxiv.org/abs/2302.01318.

Chen, Lequn, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. "Punica: Multi-Tenant LoRA Serving." *arXiv Preprint arXiv:2310.18547*. https://arxiv.org/abs/2310.18547.

Chips and Cheese. 2023. "Nvidia's H100: Funny L2, and Tons of Bandwidth." https://chipsandcheese.com/2023/07/02/nvidias-h100-funny-l2-and-tons-of-bandwidth/.

Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, et al. 2022. "PaLM: Scaling Language Modeling with Pathways." *arXiv Preprint arXiv:2204.02311*. https://arxiv.org/abs/2204.02311.

Dao, Tri. 2023. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." *arXiv Preprint arXiv:2307.08691*. https://arxiv.org/abs/2307.08691.

Dao, Tri, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *arXiv Preprint arXiv:2205.14135*. https://arxiv.org/abs/2205.14135.

Dettmers, Tim et al. 2021. "Bitsandbytes: Accessible Large Language Models via k-Bit Quantization for PyTorch." https://github.com/bitsandbytes-foundation/bitsandbytes.

Dettmers, Tim, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. "LLM.int8(): 8-Bit Matrix Multiplication for Transformers at Scale." *arXiv Preprint arXiv:2208.07339*. https://arxiv.org/abs/2208.07339.

Frantar, Elias, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers." *arXiv Preprint arXiv:2210.17323*. https://arxiv.org/abs/2210.17323.

Fu, Yichao, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. "Break the Sequential Dependency of LLM Inference Using Lookahead Decoding." *arXiv Preprint arXiv:2402.02057*. https://arxiv.org/abs/2402.02057.

Gloeckle, Fabian, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. 2024. "Better & Faster Large Language Models via Multi-Token Prediction." *arXiv Preprint arXiv:2404.19737*. https://arxiv.org/abs/2404.19737.

Grafana Labs. 2024. "Grafana Documentation." https://grafana.com/docs/grafana/latest/.

Grattafiori, Aaron, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. "The Llama 3 Herd of Models." *arXiv Preprint arXiv:2407.21783*. https://arxiv.org/abs/2407.21783.

Gu, Albert, and Tri Dao. 2023. "Mamba: Linear-Time Sequence Modeling with Selective State Spaces." *arXiv Preprint arXiv:2312.00752*. https://arxiv.org/abs/2312.00752.

Gu, Albert, and Tri Dao. 2024. "GPUs Go Brrr." https://hazyresearch.stanford.edu/blog/2024-05-12-tk.

Gu, Yuxian, Li Dong, Furu Wei, and Minlie Huang. 2024. "MiniLLM: On-Policy Distillation of Large Language Models." *arXiv Preprint arXiv:2306.08543*. https://arxiv.org/abs/2306.08543.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. "Distilling the Knowledge in a Neural Network." *arXiv Preprint arXiv:1503.02531*. https://arxiv.org/abs/1503.02531.

Hu, Edward J., Yelong Shen, Phillip Wallis, et al. 2021. "LoRA: Low-Rank Adaptation of Large Language Models." *arXiv Preprint arXiv:2106.09685*. https://arxiv.org/abs/2106.09685.

Huang, Yanping, Youlong Cheng, Ankur Bapna, et al. 2019. "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism." *arXiv Preprint arXiv:1811.06965*. https://arxiv.org/abs/1811.06965.

Jacobs, Sam Ade, Masahiro Tanaka, Chengming Zhang, et al. 2023. "DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models." *arXiv Preprint arXiv:2309.14509*. https://arxiv.org/abs/2309.14509.

Jiang, Albert Q., Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, et al. 2024. "Mixtral of Experts." *arXiv Preprint arXiv:2401.04088*. https://arxiv.org/abs/2401.04088.

Kadous, Waleed, Kyle Huang, Wendi Ding, Liguang Xie, Avnish Narayan, and Ricky Xu. 2023. "Reproducible Performance Metrics for LLM Inference." https://www.anyscale.com/blog/reproducible-performance-metrics-for-llm-inference.

Karpathy, Andrej. 2023. "Let's Build GPT: From Scratch, in Code, Spelled Out." https:

//www.youtube.com/watch?v=kCc8FmEb1nY.

Kazemnejad, Amirhossein, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. 2023. "The Impact of Positional Encoding on Length Generalization in Transformers." *arXiv Preprint arXiv:2305.19466*. https://arxiv.org/abs/2305.19466.

Kimi Team, Yu Zhang, Zongyu Lin, et al. 2025. "Kimi Linear: An Expressive, Efficient Attention Architecture." *arXiv Preprint arXiv:2510.26692*. https://arxiv.org/abs/2510.26692.

Korthikanti, Vijay Anand, Jared Casper, Sangkug Lym, et al. 2022. "Reducing Activation Recomputation in Large Transformer Models." *arXiv Preprint arXiv:2205.05198*. https://arxiv.org/abs/2205.05198.

Kumar, Tanishq, Tri Dao, and Avner May. 2026. "Speculative Speculative Decoding." *arXiv Preprint arXiv:2603.03251*. https://arxiv.org/abs/2603.03251.

Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, et al. 2023. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *arXiv Preprint arXiv:2309.06180*. https://arxiv.org/abs/2309.06180.

Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2022. "Fast Inference from Transformers via Speculative Decoding." *arXiv Preprint arXiv:2211.17192*. https://arxiv.org/abs/2211.17192.

Li, Dacheng, Rulin Shao, Anze Xie, et al. 2023. "DistFlashAttn: Distributed Memory-Efficient Attention for Long-Context LLMs Training." *arXiv Preprint arXiv:2310.03294*. https://arxiv.org/abs/2310.03294.

Li, Shenggui, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. 2021. "Sequence Parallelism: Long Sequence Training from System Perspective." *arXiv Preprint arXiv:2105.13120*. https://arxiv.org/abs/2105.13120.

Li, Zhuohan, Lianmin Zheng, Yinmin Zhong, et al. 2023. "AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving." *arXiv Preprint arXiv:2302.11665*. https://arxiv.org/abs/2302.11665.

Lin, Ji, Jiaming Tang, Haotian Tang, et al. 2023. "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration." *arXiv Preprint arXiv:2306.00978*. https://arxiv.org/abs/2306.00978.

Liu, Aixin et al. 2024. "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model." *arXiv Preprint arXiv:2405.04434*. https://arxiv.org/abs/2405.04434.

Miao, Xupeng, Gabriele Oliaro, Zhihao Zhang, et al. 2023. "Towards Efficient Generative

Large Language Model Serving: A Survey from Algorithms to Systems." *arXiv Preprint arXiv:2312.15234*. https://arxiv.org/abs/2312.15234.

Microsoft DeepSpeed Team. 2024. "DeepSpeed-Inference Documentation." https://www.deepspeed.ai/inference/.

Narayanan, Deepak, Mohammad Shoeybi, Jared Casper, et al. 2021. "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM." *arXiv Preprint arXiv:2104.04473*. https://arxiv.org/abs/2104.04473.

NVIDIA. 2022. "NVIDIA Hopper Architecture in-Depth." https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/.

NVIDIA. 2023a. "DGX Platform Solution Overview." https://resources.nvidia.com/en-us-dgx-systems.

NVIDIA. 2023b. "NVIDIA H100 Tensor Core GPU." https://www.nvidia.com/en-us/data-center/h100/.

NVIDIA. 2024a. "CUDA Programming Guide." https://docs.nvidia.com/cuda/cuda-programming-guide/.

NVIDIA. 2024b. "TensorRT-LLM Documentation." https://nvidia.github.io/TensorRT-LLM/.

OpenTelemetry Authors. 2024. "OpenTelemetry Documentation." https://opentelemetry.io/docs/.

Patel, Pratyush, Esha Choukse, Chaojie Zhang, et al. 2024. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting." *arXiv Preprint arXiv:2311.18677*. https://arxiv.org/abs/2311.18677.

Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, et al. 2022. "Efficiently Scaling Transformer Inference." *arXiv Preprint arXiv:2211.05102*. https://arxiv.org/abs/2211.05102.

Press, Ofir, Noah A. Smith, and Mike Lewis. 2021. "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation." *arXiv Preprint arXiv:2108.12409*. https://arxiv.org/abs/2108.12409.

Prometheus Authors. 2024. "Prometheus Documentation." https://prometheus.io/docs/.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. "Language Models Are Unsupervised Multitask Learners." *OpenAI Blog*. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_

multitask_learners.pdf.

Raschka, Sebastian. 2025. "LLM Architecture Gallery." https://sebastianraschka.com/llm-architecture-gallery/.

Rush, Alexander. 2018. "The Annotated Transformer." https://nlp.seas.harvard.edu/annotated-transformer/.

SGLang Team. 2024. "SGLang Documentation." https://docs.sglang.ai/.

Shah, Jay, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision." *arXiv Preprint arXiv:2407.08608*. https://arxiv.org/abs/2407.08608.

Shazeer, Noam. 2019. "Fast Transformer Decoding: One Write-Head Is All You Need." *arXiv Preprint arXiv:1911.02150*. https://arxiv.org/abs/1911.02150.

Shazeer, Noam. 2020. "GLU Variants Improve Transformer." *arXiv Preprint arXiv:2002.05202*. https://arxiv.org/abs/2002.05202.

Sheng, Ying, Shiyi Cao, Dacheng Li, et al. 2023. "S-LoRA: Serving Thousands of Concurrent LoRA Adapters." *arXiv Preprint arXiv:2311.03285*. https://arxiv.org/abs/2311.03285.

Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." *arXiv Preprint arXiv:1909.08053*. https://arxiv.org/abs/1909.08053.

Spheron. 2025. "vLLM Vs TensorRT-LLM Vs SGLang: Benchmarks on H100." https://www.spheron.network/blog/vllm-vs-tensorrt-llm-vs-sglang-benchmarks/.

Stern, Mitchell, Noam Shazeer, and Jakob Uszkoreit. 2018. "Blockwise Parallel Decoding for Deep Autoregressive Models." *arXiv Preprint arXiv:1811.03115*. https://arxiv.org/abs/1811.03115.

Su, Jianlin, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. "RoFormer: Enhanced Transformer with Rotary Position Embedding." *arXiv Preprint arXiv:2104.09864*. https://arxiv.org/abs/2104.09864.

Touvron, Hugo, Thibaut Lavril, Gautier Izacard, et al. 2023. "LLaMA: Open and Efficient Foundation Language Models." *arXiv Preprint arXiv:2302.13971*. https://arxiv.org/abs/2302.13971.

Touvron, Hugo, Louis Martin, Kevin Stone, et al. 2023. "Llama 2: Open Foundation and Fine-Tuned Chat Models." *arXiv Preprint arXiv:2307.09288*. https://arxiv.org/abs/2307.09288.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. "Attention Is All You Need." *arXiv Preprint arXiv:1706.03762*. https://arxiv.org/abs/1706.03762.

vLLM Team. 2024. "vLLM Documentation." https://docs.vllm.ai/.

Weng, Lilian. 2023. "The Transformer Family Version 2.0." https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/.

Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76. https://dl.acm.org/doi/pdf/10.1145/1498765.1498785.

Wu, Bingyang, Yinmin Zhong, Zili Zhang, et al. 2023. "Fast Distributed Inference Serving for Large Language Models." *arXiv Preprint arXiv:2305.05920*. https://arxiv.org/abs/2305.05920.

Xiao, Guangxuan, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models." *arXiv Preprint arXiv:2211.10438*. https://arxiv.org/abs/2211.10438.

Yang, An, Anfeng Li, Baosong Yang, et al. 2025. "Qwen3 Technical Report." *arXiv Preprint arXiv:2505.09388*. https://arxiv.org/abs/2505.09388.

Yang, Nan, Tao Ge, Liang Wang, et al. 2023. "Inference with Reference: Lossless Acceleration of Large Language Models." *arXiv Preprint arXiv:2304.04487*. https://arxiv.org/abs/2304.04487.

Yu, Gyeong-In, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. "Orca: A Distributed Serving System for Transformer-Based Generative Models." *OSDI*. https://www.usenix.org/conference/osdi22/presentation/yu.

Yuan, Jingyang et al. 2025. "Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention." *arXiv Preprint arXiv:2502.11089*. https://arxiv.org/abs/2502.11089.

Zandieh, Amir, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2025. "TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate." *ICLR*. https://arxiv.org/abs/2504.19874.

Zhang, Biao, and Rico Sennrich. 2019. "Root Mean Square Layer Normalization." *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/1910.07467.

Zheng, Lianmin, Liangsheng Yin, Zhiqiang Xie, et al. 2023. "SGLang: Efficient Execution of Structured Language Model Programs." *arXiv Preprint arXiv:2312.07104*. https://arxiv.org/abs/2312.07104.

Zhong, Yinmin, Shengyu Liu, Junda Chen, et al. 2024. "DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving." *arXiv Preprint arXiv:2401.09670*. https://arxiv.org/abs/2401.09670.

Zhu, Xunyu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2024. "A Survey on Model Compression for Large Language Models." *arXiv Preprint arXiv:2308.07633*. https://arxiv.org/abs/2308.07633.